

Checkpoint — Análise de Data Drift em Seq2Seq para Predição de Trajetórias no RoboCup SSL

Iniciação Científica — Relatório técnico para validação do orientador (Meses 1–10)

3 de julho de 2026

Resumo

Este documento consolida dez meses de trabalho sobre *data drift* (deriva dos dados — o fenômeno em que dados reais mudam ao longo do tempo, fazendo o modelo perder precisão) em um modelo **Seq2Seq** (*Sequence-to-Sequence* — rede neural que prevê trajetórias futuras a partir de trajetórias passadas) de predição de trajetórias de robôs no **RoboCup Small Size League (SSL)** (competição mundial de futebol robótico com robôs miniatura autônomos). O modelo foi treinado em dados de 2019 e avaliado contra dados de 2021–2025; mostramos que ele degrada (**ADE** [Erro Médio de Deslocamento, em milímetros] $\uparrow$ de 4,53 mm em 2019 para 7,58 mm em 2021 no horizonte 30 $\rightarrow$ 15 [o modelo recebe 30 instantes passados e prevê 15 futuros]) e investigamos a natureza desse drift, em seguida convertendo a evidência em um detetor online operacional (um algoritmo que monitora o fluxo de dados em tempo real e dispara alertas quando detecta mudanças).

A contribuição do trabalho está organizada em sete camadas sucessivas: (i) **degradação e diagnóstico granular** (Mês 1): reescrita do pipeline para operar por jogo, por janela e por trajetória individual; (ii) **detecção formal no stream** (Mês 2) com **ADWIN** (detetor adaptativo de janela), **Page-Hinkley** (detetor de aumento acumulado), **KSWIN** (detetor baseado no teste KS) e **Pelt** (detetor offline de pontos de ruptura); (iii) **covariate shift e explicação** (Mês 3) via **importance weighting LSIF** (técnica que quantifica quanto do erro é explicado pela mudança na distribuição dos dados de entrada) e **fine-tuning** (retreinamento parcial do modelo); (iv) **robustez e validação** (Meses 4–6) com baseline completo, *sample-size sweep* (teste de sensibilidade ao tamanho da amostra de referência), calibração por **ARL** (Comprimento Médio de Corrida — métrica padrão de calibração de detectores), **BOCPD** (detecção bayesiana online de changepoints) e sensibilidade ao parâmetro γ do Pelt; (v) **anti-forgetting** (Mês 7) com **EWC** [17] (*Elastic Weight Consolidation* — técnica que protege parâmetros importantes durante retreinamento); (vi) **revisão bibliográfica consolidada** (Mês 8); (vii) **otimização dos detectores online** (Mês 9) em quatro fases: suavização do sinal ADE (22 experimentos), *grid search* (busca exaustiva em grade de 174 combinações $\times$ 2 modelos), ensemble temporal AND/OR (combinação de múltiplos detectores) e consolidação científica com métricas corrigidas; (viii) **fechamento das pendências da revisão** (Mês 10): latência de detecção por ano, validação da hipótese de independência do stream, ensemble AND de mesma escala temporal e EWC no regime $\lambda \geq 10^5$.

Achados principais com suporte numérico:

- Covariate shift é direcional e estatisticamente significativo em todos os anos ($p < 10^{-3}$ pelo classifier two-sample test [16]), com **accel_p99** (percentil 99 da aceleração — os momentos de aceleração mais extrema) como dimensão dominante. Triangulação com três estimadores delimita a fração do excesso de ADE explicável: LSIF clássico (59–72 %, ESS 0,58–0,79 — pesos com cauda pesada), RuLSIF [15] com $\alpha=0,1$ (9–23 %, ESS 0,94–0,996 — estimador limitado em $1/\alpha=10$, mais conservador), e cobertura do classifier (22–51 %, árbitro independente). O sinal qualitativo (covariate shift dominado por aceleração, 2021 mais severo que 2024) é robusto aos três estimadores; a banda 9–72 % reflete a sensibilidade do recovery à cauda de **accel_p99**.

- Fine-tuning leve (retreinamento parcial do modelo) é prejudicado por *catastrophic interference* (esquecimento catastrófico — o modelo melhora nos dados novos mas piora nos dados antigos): `cat_ratio` (razão de esquecimento) = 0,82 no Mês 3 reduzido a 0,44 no Mês 7 com **EWC** (regularização que protege parâmetros importantes) + `lr` conservador (taxa de aprendizado baixa = 10^{-5}).
- O drift é **intra-divisão** (ocorre mesmo dentro da Divisão A, sem mudança de categoria) e **predominantemente gradual** (Pelt mBIC [critério de informação bayesiano modificado] aponta $K^*=0$ pontos de ruptura abrupta ao nível de 36 jogos — não há quebra súbita, o drift é contínuo).
- Sensibilidade ao tamanho do baseline é muito grande: ADE_{2019} vai de 4,53 mm ($n=6$) a 6,20 mm ($n=1$), invertendo o sinal do excesso em quatro dos cinco anos. Isso justifica retroativamente a decisão de adotar o baseline completo.
- **Otimização do Mês 9.** Sobre o stream de 288 k trajetórias, identifica-se um detetor primário operacional: **ADWIN com robust-z** $w=200$ ($\delta=10^{-7}$, `min_window`=600, `cooldown`=200, `max_window`=2000) atinge $FAR_{2019}=0,042/1k$ com cobertura 5/5 anos pós-2019 (IC Wilson 95 % = $[0,011; 0,152]/1k$ para $k=2$ alarmes em $n=48\,000$). A validação com o ADWIN exato [2] (implementação `river`, re-grid independente) confirma o mesmo quadro qualitativo com $FAR_{2019}=0,125/1k$ e cobertura 5/5. O ensemble AND temporal com Page-Hinkley agregado é **estruturalmente inviável** a este nível (desalinhamento $\Delta_{\min}=214$ trajetórias entre os alarmes mais próximos — maior que a janela $W_{\text{coincidence}}=200$ operacionalmente defensável; cobertura $\geq 2/5$ exigiria $W \geq 10\,000$).
- **Fechamento das pendências (Mês 10).** (a) *Latência*: o detetor primário dispara com atraso de 0,3–2,8 jogos após a fronteira do ano; o canal OR reduz o pior caso para $\leq 0,4$ jogo em 4/5 anos. (b) *Independência*: a autocorrelação intra-jogo é real ($\rho_1 \approx 0,54$, $n_{\text{eff}} \approx 16\%$ de n), mas permutações mostram que ela *infla* a FAR (streams i.i.d.-izados de 2019 produzem zero alarmes) — a admissibilidade não é artefato da dependência serial. (c) *Ensemble de mesma escala*: alinhar as escalas derruba $\Delta_{\min}$ de 214 para 3 trajetórias; o AND(ADE , `accel_p95`) atinge $FAR=0,021/1k$ mas cobre 3/5 anos — o standalone segue recomendado. (d) *EWC*: $\lambda \in \{10^5, 10^6, 10^7\}$ não altera `cat_ratio` (0,449–0,452 vs. 0,444 em $\lambda=0$) — resultado negativo definitivo em 8 ordens de grandeza.

O documento incorpora todas as figuras geradas pelo pipeline — *incluindo galerias completas* das 22 configurações de suavização e dos heatmaps das três frentes do grid search — para permitir comparação lado a lado e seleção posterior dos candidatos finais ao paper. Limitações onde os experimentos não foram conclusivos são explicitamente sinalizadas (Divisão B ausente; ensemble AND inviável; KSWIN inadmissível).

Conceitos Fundamentais para o Leitor

Esta seção apresenta, em linguagem acessível, todos os conceitos e termos técnicos usados neste relatório. O objetivo é que qualquer pessoa com boa capacidade de raciocínio — mesmo sem formação prévia em aprendizado de máquina ou estatística — consiga acompanhar os resultados e conclusões sem precisar de fontes externas.

O que é o RoboCup SSL?

O **RoboCup Small Size League (SSL)** é uma competição internacional de futebol robótico em que duas equipes de pequenos robôs autônomos (com cerca de 15 cm de diâmetro) disputam partidas em um campo verde monitorado por câmeras suspensas no teto. Uma câmera filmando de cima envia a posição de cada robô ao computador central 60 vezes por segundo; o computador toma decisões táticas e envia comandos de movimento sem fio para cada robô individualmente.

O sistema opera a alta velocidade e gera sequências densas de posições no tempo — as *trajetórias* analisadas neste trabalho. Os dados usados aqui correspondem a partidas reais dos campeonatos mundiais de 2019 a 2025.

O que é um modelo Seq2Seq?

Um modelo **Seq2Seq** (Sequência para Sequência, do inglês *Sequence-to-Sequence*) é um tipo de rede neural profunda capaz de receber uma sequência de valores como entrada e produzir outra sequência como saída. Imagine ensinar um computador a ler os últimos segundos do movimento de um robô e prever para onde ele vai nos próximos instantes — exatamente isso faz o Seq2Seq. Neste trabalho, o modelo recebe os últimos 30 instantes de posição de um robô (o *histórico*) e prevê os próximos 15 instantes (o *futuro*). A arquitetura interna possui dois módulos principais:

- **Encoder BiLSTM:** o encoder “lê” o histórico de movimentos e o comprime em uma representação numérica compacta. A sigla **LSTM** significa *Long Short-Term Memory* (Memória de Longo e Curto Prazo): é um tipo especial de rede neural recorrente que foi projetado para lembrar informação relevante ao longo de uma sequência. Diferente de uma rede neural simples que processa cada ponto de forma independente, a LSTM mantém uma “memória interna” que pode guardar contexto de instantes passados — como uma pessoa que lembra o início de uma frase ao chegar no fim. O prefixo **Bi** significa *Bidirecional*: a rede processa a sequência *duas vezes*, uma da esquerda para a direita (passado para presente) e outra da direita para a esquerda (do presente de volta ao passado), e combina as duas perspectivas. Isso captura padrões que dependem tanto do início quanto do fim do histórico.
- **Decoder LSTM com atenção:** o decoder usa a representação criada pelo encoder para gerar a previsão passo a passo. O mecanismo de **atenção** (*attention*) permite que o decoder, ao gerar cada passo futuro, “consulte” os instantes mais relevantes do histórico de entrada — em vez de depender apenas de um vetor de contexto fixo. É análogo a uma pessoa que, ao tentar prever o final de uma história, pode voltar e reler os trechos mais importantes da narrativa.

O modelo foi **treinado** com dados de 2019, ajustando seus parâmetros internos (milhões de números) para minimizar o erro nas trajetórias daquele ano. Ao avaliar com dados de 2021 em diante, observa-se que o erro aumentou: o modelo não se adaptou automaticamente às mudanças no comportamento dos robôs ao longo dos anos.

O que é o Filtro de Kalman?

O **Filtro de Kalman** é um algoritmo clássico (desenvolvido em 1960) de estimação e predição para sistemas físicos, como o movimento de um robô. Ao contrário do Seq2Seq, ele *não aprende com dados*: seus parâmetros são definidos por equações físicas de movimento (posição, velocidade, aceleração). Neste trabalho, o Kalman serve como **referência determinística**: se o Kalman também piora em determinado período, significa que o comportamento dos robôs mudou genuinamente (e qualquer modelo sofreria); se apenas o Seq2Seq piora, a causa provável é uma falha específica do modelo treinado em capturar os novos padrões.

O que são ADE e FDE?

- **ADE** (*Average Displacement Error*, Erro Médio de Deslocamento): é a média das distâncias euclidianas (distância em linha reta) entre a posição prevista pelo modelo e a posição real do robô, calculada em cada instante de tempo dentro da janela de previsão. Um ADE de 4,53 mm significa que, em média, o modelo errou por 4,53 milímetros. Quanto *menor* o ADE, melhor o modelo. O aumento do ADE de 4,53 mm (2019) para 7,58 mm (2021) indica que o modelo passou a errar 67 % mais em termos de distância média.

- **FDE** (*Final Displacement Error*, Erro no Instante Final): é a distância entre a posição prevista *no último instante* da janela e a posição real. Enquanto o ADE mede o erro ao longo de toda a trajetória prevista, o FDE mede especificamente o erro no ponto de chegada.

A notação “horizonte 30→15” significa: o modelo recebe 30 instantes passados como entrada e prevê 15 instantes futuros como saída.

O que é Data Drift (Deriva dos Dados)?

Data drift (ou deriva dos dados) é o fenômeno em que os dados reais que chegam ao modelo ao longo do tempo diferem dos dados com que ele foi treinado originalmente. Imagine um médico que aprendeu a diagnosticar doenças estudando casos de 2019: se os padrões clínicos dos pacientes mudarem em 2023 (por exemplo, devido a novas variantes de um vírus), suas previsões baseadas no conhecimento de 2019 começarão a falhar. O mesmo acontece com redes neurais: o modelo “congela” seu conhecimento no momento do treino e não se atualiza automaticamente quando o mundo muda.

Existem dois tipos fundamentais de drift, com causas e soluções muito diferentes:

- **Covariate shift** (desvio de covariáveis): a distribuição dos dados de *entrada* $p(x)$ muda, mas a relação entre entrada e saída $p(y|x)$ permanece a mesma. No contexto deste trabalho: os robôs passaram a acelerar mais agressivamente em 2021–2025 do que em 2019 (mudança na distribuição de aceleração), mas *dada* uma certa aceleração, a trajetória resultante ainda segue as mesmas leis físicas. Este tipo de drift é **recuperável sem retreinar o modelo**: pode-se corrigir dando mais peso aos exemplos antigos que se parecem com os novos dados (importance weighting).
- **Concept drift** (deriva de conceito, também chamado *real drift*): a própria relação $p(y|x)$ muda. Exemplo: se as estratégias táticas dos robôs evoluíram radicalmente (novas manobras que não existiam em 2019), a trajetória dada uma certa velocidade seria diferente da trajetória observada em 2019. Este tipo exige retreinamento do modelo.

Este trabalho demonstra que covariate shift é direcional e significativo ($p < 10^{-3}$, classifier two-sample test [16]), principalmente na distribuição da aceleração de pico (`accel_p99`: o percentil 99 da aceleração de cada jogo). A fração do excesso de ADE atribuível a esse covariate shift, triangulada por três estimadores independentes (LSIF clássico, RuLSIF [15] e classifier coverage), situa-se em [9; 72] %, com o LSIF acusando 59–72 % e o RuLSIF (estimador limitado, mais conservador) 9–23 %; a cobertura do classifier (22–51 %) serve de árbitro independente. A componente não explicada é concept drift residual mais ruído amostral.

O que são Detectores Online de Mudança?

Um **detector online** é um algoritmo que monitora um fluxo contínuo de dados em tempo real e dispara um alarme quando detecta que algo mudou na distribuição. “Online” significa que o algoritmo processa os dados um a um, conforme chegam, sem precisar armazenar e reprocessar toda a sequência histórica. Em sistemas de produção com modelos de machine learning, detectores online são essenciais: se o modelo começa a errar mais sistematicamente, o detector pode acionar automaticamente um processo de retreinamento.

Os dois critérios de avaliação dos detectores neste trabalho são:

- **FAR** (*False Alarm Rate*, Taxa de Falsos Alarmes): número de alarmes disparados por mil trajetórias *no período de treino* (2019), onde sabe-se que não há drift real. Um detector ideal tem $FAR \approx 0$. FAR alto significa que o detector aciona alarmes desnecessariamente, levando a retreinamentos desnecessários e custosos.
- **year_coverage** (Cobertura Anual): em quantos dos 5 anos pós-2019 (2021–2025) o detector disparou pelo menos um alarme. Um bom detector deve cobrir todos os 5 anos (5/5), demonstrando que detecta drift real em todos os períodos relevantes.

O que é ARL (Average Run Length)?

ARL (Comprimento Médio de Corrida, do inglês *Average Run Length*) é o número médio de observações que o detector processa antes de disparar um alarme *por acaso*, em dados sem nenhuma mudança real (dados i.i.d. — independentes e identicamente distribuídos, como lances de uma moeda justa). Um ARL de 1000 significa que, em dados normais, o detector dispara um falso alarme a cada 1000 observações em média. Para calibrar detectores de forma rigorosa, compara-se o ARL observado nos dados de treino com o ARL teórico esperado sob ausência de drift.

O que são SNR e SNR_smooth?

SNR (*Signal-to-Noise Ratio*, Razão Sinal-Ruído) é uma métrica que compara o número de alarmes em anos com drift (“sinal”) com o número de alarmes no ano de treino 2019 (“ruído”). $SNR > 1$ indica que o detector dispara mais quando há drift do que quando não há — ou seja, está detectando algo real e não apenas ruído aleatório. Um problema surge quando há *zero* alarmes em 2019: nesse caso, SNR seria infinito, e foi atribuído o valor sentinela 9999. A versão corrigida **SNR_smooth** usa uma pseudo-contagem de Laplace ($a = 1$) para evitar esse problema, adicionando 1 alarme imaginário tanto no numerador quanto no denominador, garantindo um valor finito e estável.

O que é o Intervalo de Confiança Bootstrap?

O **bootstrap** é uma técnica estatística não-paramétrica (que não assume nenhuma forma específica para a distribuição dos dados) para estimar a incerteza de uma medida. O procedimento é:

1. Dado um conjunto de n observações reais, geram-se $B = 2.000$ amostras artificiais do mesmo tamanho, cada uma sorteada *com reposição* (o mesmo valor pode ser sorteado várias vezes).
2. Calcula-se a estatística de interesse (por exemplo, a média do ADE) em cada amostra artificial.
3. Os percentis 2,5 % e 97,5 % dessas 2.000 estimativas formam o **intervalo de confiança de 95 %** (IC 95 %).

Interpretação: se o IC de 2019 não se sobrepõe ao IC de 2021, a diferença entre os anos é estatisticamente significativa — não pode ser explicada apenas pelo acaso amostral.

O que é a Estatística de Kolmogorov–Smirnov (KS)?

A estatística **KS** compara duas distribuições de dados verificando qual é a maior diferença entre suas curvas de distribuição acumulada (**CDF**, *Cumulative Distribution Function*). Uma CDF empírica é construída ordenando todos os valores do menor para o maior: 0 % das observações são menores que o mínimo, 50 % são menores que a mediana, e 100 % são menores que o máximo. A KS mede o ponto onde as duas CDFs mais se afastam. Varia de 0 (distribuições idênticas) a 1 (distribuições completamente separadas). Valores altos indicam que as distribuições são muito diferentes, o que é um sinal de drift nas features de entrada.

O que é a Distância de Wasserstein?

A distância de **Wasserstein** (também chamada de *Earth Mover’s Distance*, distância do transportador de terra) mede o custo mínimo para transformar uma distribuição em outra. A metáfora clássica: imagine a distribuição de velocidades de 2019 como um monte de areia e a de 2021 como outro monte com forma diferente — a distância de Wasserstein é o trabalho mínimo (massa $\times$ distância) para remodelar o primeiro monte para que fique igual ao segundo. Enquanto o KS mede apenas a máxima separação em um ponto, o Wasserstein captura a *magnitude* do deslocamento em toda a distribuição. Por ser medido na mesma unidade dos dados (mm, mm/s, etc.), é mais intuitivo para interpretação física.

O que é a Correlação de Spearman (ρ)?

A correlação de **Spearman** mede se duas variáveis tendem a crescer juntas (correlação positiva) ou em sentidos opostos (correlação negativa), sem exigir que a relação seja linear. Em vez de usar os valores brutos, ela usa os *ranks* (posições no ranking ordenado): o menor valor recebe rank 1, o segundo menor recebe rank 2, e assim por diante. Isso a torna robusta a *outliers* (valores extremos). Um $|\rho| > 0,5$ entre uma feature de entrada (por exemplo, aceleração) e o erro do modelo (ADE) é um indício forte de que aquela feature “carrega” o drift — ou seja, quando a aceleração aumenta, o ADE também tende a aumentar.

O que é o IC de Wilson para Proporções?

O **intervalo de confiança de Wilson** é um método estatisticamente robusto para calcular intervalos de confiança para proporções (como a FAR), especialmente quando o número de eventos observados é muito pequeno — inclusive zero. O método clássico (baseado na distribuição normal) falha para $k = 0$ eventos, pois produziria um intervalo de largura zero, o que é enganoso (não significa que a FAR real é exatamente zero — significa apenas que não foi observado nenhum alarme naquela amostra). O Wilson usa uma fórmula que incorpora corretamente a incerteza: com $k = 0$ alarmes em $n = 48.000$ trajetórias, o limite superior do IC 95 % é 0,080/1k — ou seja, mesmo não tendo visto nenhum alarme, ainda é possível que a taxa real seja até 0,080 por mil trajetórias.

O que é Catastrophic Forgetting (Esquecimento Catastrófico)?

Quando uma rede neural treinada em dados antigos (2019) é retreinada com dados novos (2021), ela tende a “esquecer” o que aprendeu antes, sacrificando a performance nos dados antigos para melhorar nos novos. Esse fenômeno é chamado de **catastrophic forgetting** (esquecimento catastrófico) ou *catastrophic interference* (interferência catastrófica). É como forçar um estudante a memorizar um texto novo: ele pode acabar esquecendo o texto anterior. Neste trabalho, a métrica `cat_ratio` quantifica o esquecimento:

$$\text{cat_ratio} = \frac{\text{ADE}_{\text{baseline, depois do fine-tuning}}}{\text{ADE}_{\text{baseline, antes do fine-tuning}}}$$

Um `cat_ratio = 1,0` indica que o modelo não esqueceu nada; `cat_ratio = 0,82` (resultado do Mês 3) indica que o modelo passou a errar 82 % mais nos dados antigos após o ajuste nos dados novos — alta interferência catastrófica.

O que é Fine-Tuning (Ajuste Fino)?

Fine-tuning é a prática de pegar um modelo já treinado e continuá-lo treinando com dados novos, geralmente usando uma taxa de aprendizado (*learning rate*) muito menor do que na fase de treino original. É como um especialista que, após anos de formação geral, faz um curso de especialização focado: não recomeça do zero, apenas ajusta seu conhecimento existente. A taxa de aprendizado controla o quão grandes são os ajustes em cada iteração — valores muito altos causam esquecimento; valores muito baixos causam pouca adaptação.

O que é Busca em Grade (*Grid Search*)?

Grid search (busca em grade) é a estratégia de otimização que testa *todas* as combinações possíveis de valores para os hiperparâmetros de um algoritmo. É como provar todas as combinações de temperatura e tempo para assar um bolo: define-se uma grade de valores (ex.: $\delta \in \{10^{-7}, 10^{-6}, 10^{-5}\}$; `cooldown` $\in \{200, 500, 1000\}$) e testa-se cada combinação. No Mês 9, foram testadas 174 combinações de hiperparâmetros dos detectores, avaliando cada uma pelas métricas FAR e `year_coverage`. A busca em grade é exaustiva mas garante que a melhor combinação dentre as testadas será encontrada.

O que é um Ensemble de Detectores?

Um **ensemble** combina múltiplos detectores independentes para produzir uma decisão mais robusta. Existem duas lógicas principais:

- **AND** (“E”): alarme confirmado (*CONFIRMED*) apenas quando *todos* os detectores disparam dentro de uma janela temporal $W_{\text{coincidence}}$. Isso reduz falsos alarmes (dois detectores independentes raramente erram ao mesmo tempo), mas pode perder alarmes reais se os detectores operam em escalas temporais muito diferentes.
- **OR** (“OU”): alarme suspeito (*SUSPECTED*) disparado quando *qualquer* detector dispara. Isso aumenta a cobertura (detecta mais casos de drift), mas também aumenta os falsos alarmes.

A janela de coincidência $W_{\text{coincidence}}$ define o intervalo máximo (em número de trajetórias) dentro do qual dois detectores precisam disparar para serem considerados “coincidentes” no AND. O Mês 9 demonstra que o AND entre ADWIN e Page-Hinkley é estruturalmente inviável porque os dois detectores operam em escalas temporais completamente diferentes.

O que é o pré-processamento Robust-Z?

O **robust-z** é uma forma de normalização local que, para cada ponto do stream, subtrai a mediana dos últimos w pontos e divide pelo MAD (*Median Absolute Deviation*, Desvio Absoluto Mediano) dos mesmos w pontos:

$$z_t = \frac{x_t - \text{mediana}(x_{t-w:t})}{\text{MAD}(x_{t-w:t})}$$

Isso “centraliza” e “normaliza a escala” do sinal localmente — o resultado é um sinal adimensional que indica o quão “anormal” cada ponto é em relação ao contexto local recente. O MAD é mais robusto a *outliers* do que o desvio-padrão convencional: valores extremos isolados não distorcem a normalização. Com janela $w = 200$ trajetórias, o robust-z foi o único pré-processamento que reduziu a FAR do ADWIN (−51 %), porque normaliza a escala local sem comprimir o sinal de drift.

O que é a ESS (Effective Sample Size)?

Quando se aplica ponderação por importância (*importance weighting*), os pesos $\hat{w}(x_i)$ podem ser muito desiguais. A **ESS** (Tamanho Efetivo da Amostra, *Effective Sample Size*) mede o grau de “degenerência” dos pesos:

$$\text{ESS} = \frac{(\sum_i \hat{w}_i)^2}{n \sum_i \hat{w}_i^2} \in \left[\frac{1}{n}, 1 \right]$$

ESS próximo de 1 indica pesos bem distribuídos (o estimador ponderado é confiável); ESS próximo de $1/n$ indica que apenas um ou poucos exemplos dominam toda a ponderação (o estimador é instável). A regra prática usada é $\text{ESS} > 0,5$ para aceitar o estimador como válido.

Trabalhos relacionados

Predição de trajetórias. A predição de trajetórias de agentes móveis (pedestres, veículos, robôs) é um problema central em robótica e visão computacional: dado o histórico de movimentos de um agente, qual será sua posição nos próximos instantes? A dificuldade principal é que múltiplos futuros são possíveis e que os agentes interagem entre si.

O **Social-LSTM** [19] foi um marco ao introduzir *social pooling*: em vez de prever cada pedestre de forma isolada, as LSTMs de pedestres vizinhos “compartilhavam informação” por um mecanismo de pooling (agregação de estado), capturando interações sociais como evitar colisões e seguir grupos.

O **Trajectron++** [20] substituiu o pooling por grafos dinâmicos que representam explicitamente as relações entre agentes, usando variáveis latentes para modelar múltiplos modos de comportamento possíveis — produzindo distribuições sobre trajetórias em vez de uma única previsão determinística.

O **AgentFormer** [21] usa arquitetura **Transformer** — o mesmo mecanismo de atenção multi-cabeça que revolucionou o processamento de linguagem natural — de forma “agent-aware”, tratando cada agente e cada instante de tempo como um “token” que pode atender aos demais.

Os **modelos de difusão** como o MID [23] geram previsões através de um processo iterativo de refinamento: começam com ruído aleatório e progressivamente “limpam” esse ruído até obter uma trajetória coesa, produzindo trajetórias diversas e realistas.

No domínio SSL, Steuernagel and Maximo [22] aplicaram exatamente a mesma arquitetura Seq2Seq (encoder BiLSTM + atenção + decoder LSTM) com as mesmas janelas 30/15 e 60/30 aqui estudadas, tornando esse trabalho o baseline mais próximo desta dissertação e a referência direta de comparação.

Data drift e concept drift. Gama et al. [6] fornecem o survey canônico sobre drift em *data streams* (fluxos contínuos de dados). A distinção fundamental é entre *virtual drift* (mudança em $p(x)$ — covariate shift: os dados de entrada mudam mas a relação com a saída permanece) e *real drift* (mudança em $p(y|x)$ — a própria relação entre entrada e saída se altera). Os detectores se dividem em duas famílias: os *error-based*, que monitoram métricas de desempenho do modelo (como o ADE) em tempo real — ADWIN [2] e Page-Hinkley [3] —; e os *data distribution-based*, que comparam diretamente as distribuições dos dados de entrada sem usar informação de erro do modelo, como KSWIN [4]. Lu et al. [7] identificam a calibração do ARL (comprimento médio de corrida — número esperado de observações até um falso alarme) como requisito essencial para detecção confiável — exatamente o que a análise do Mês 6 confirma como lacuna nos resultados anteriores e o que o Mês 9 resolve por busca em grade direta sobre o stream completo de 288 k trajetórias.

Detecção de changepoints. Um **changepoint** (ponto de mudança) é um instante no tempo em que a distribuição estatística de um sinal muda abruptamente. Detectar changepoints é diferente de detectar drift gradual: aqui, busca-se o instante exato de ruptura, processando toda a série de uma vez (modo *offline*, em oposição aos detectores online que processam ponto a ponto).

O algoritmo **PELT** [9] (*Pruned Exact Linear Time* — Tempo Linear Exato com Poda) resolve esse problema de forma eficiente via programação dinâmica. Dada uma série de T observações, o PELT encontra o número ótimo de breakpoints K^* e suas posições minimizando a soma do custo de ajuste de cada segmento mais uma penalidade $\beta \cdot K$ (quanto maior β , menos breakpoints o algoritmo aceita — forçando parsimônia). O custo **RBF** (*Radial Basis Function*, Função de Base Radial) é uma função não-paramétrica baseada em um kernel gaussiano que compara segmentos sem assumir forma distribucional específica.

O critério **mBIC** (*modified Bayesian Information Criterion* — Critério de Informação Bayesiano modificado) é um método de seleção de modelo que equilibra qualidade de ajuste e parcimônia. Para sinais curtos ($T=36$ jogos), o mBIC seleciona $K^*=0$ breakpoints — resultado negativo que é corretamente reportado: a série é pequena demais para que a penalidade seja superada pelo ganho de ajuste.

O **BOCPD** [10] (*Bayesian Online Changepoint Detection* — Detecção Bayesiana Online de Changepoints) é uma abordagem alternativa baseada em inferência bayesiana: em vez de um resultado binário, produz a probabilidade de haver um changepoint em cada instante. Aminikhanghahi and Cook [11] catalogam vantagens e limitações relativas de PELT, BOCPD e Bottom-Up; o consenso de dois métodos independentes fortalece a conclusão de dois regimes de comportamento na série de ADE por jogo.

Importance weighting e covariate shift. A ideia central do **importance weighting** (ponderação por importância) é: se os dados de treinamento (2019) e os dados de avaliação (2021) têm distribuições diferentes, pode-se “corrigir” essa diferença *sem retreinar o modelo*, simplesmente dando mais peso (*peso* = importância relativa) às trajetórias de 2019 que se parecem com as de 2021. O peso ideal de cada exemplo é a razão de densidades $\hat{w}(x) = p_{\text{alvo}}(x)/p_{\text{baseline}}(x)$ — quantas vezes mais provável é aquele exemplo na nova distribuição do que na antiga.

A estimativa direta desta razão via **LSIF** (*Least-Squares Importance Fitting* — Ajuste por Mínimos Quadrados de Importância) / **KLIEP** (*Kullback-Leibler Importance Estimation Procedure*) [12, 13] evita estimar as duas densidades separadamente — o que seria muito impreciso em alta dimensão. Em vez disso, modela diretamente $\hat{w}(x)$ como uma combinação de funções-base (kernels gaussianos) e otimiza os coeficientes para que o ADE ponderado se aproxime do ADE de 2019.

A extensão **uLSIF** [14] e o estimador robusto **RuLSIF** [15] (*Robust Least-Squares Importance Fitting*) reduzem a variância quando a distribuição alvo tem *cauda pesada* — ou seja, quando alguns valores extremos ocorrem com frequência surpreendentemente alta. Isso é relevante aqui pois `accel_p99` (o percentil 99 da aceleração — os 1% de momentos com maior aceleração) é literalmente uma estatística de cauda e portanto sensível a outliers. Ambos foram implementados e comparados (ver Seção 5). O teste de duas amostras via classificador [16] é a alternativa *data-driven* independente do LSIF para espaços de alta dimensão — treina-se uma regressão logística para separar dados de 2019 dos dados de cada ano-alvo; a AUC mede a separabilidade e, portanto, o covariate shift — e também foi implementado (ver Seção 5).

Ensembles de detecção de drift. A literatura clássica [6, 7] discute a combinação AND/OR de detectores. A lógica **AND** (*co-deteção*): um alarme é considerado *confirmado* apenas quando ambos os detectores concordam dentro de uma janela temporal — isso reduz drasticamente os falsos alarmes, pois é raro que dois detectores independentes errem ao mesmo tempo. A lógica **OR**: qualquer alarme de qualquer detector gera um alerta — isso amplia a cobertura mas também amplifica os falsos alarmes. A janela temporal $W_{\text{coincidence}}$ é o parâmetro crítico do AND: ela define o intervalo máximo (em número de trajetórias) dentro do qual os dois alarmes precisam ocorrer para serem considerados “coincidentes”. Janelas pequenas maximizam especificidade mas exigem que os detectores operem em escalas temporais semelhantes — uma condição que, como o Mês 9 demonstra, não se sustenta entre ADWIN (que opera trajetória a trajetória, capturando mudanças locais) e Page-Hinkley agregado (que opera em janelas de 50 trajetórias médias, capturando mudanças mais globais). A distância mínima entre os pares de alarmes mais próximos é $\Delta_{\min} = 214$ trajetórias — maior do que a janela $W = 200$ operacionalmente defensável para um sistema de alerta confiável (e a cobertura multi-ano do AND exigiria $W \geq 10\,000$).

1 Linha do tempo

A organização deste relatório segue a estrutura dos quatro notebooks do pipeline mais a etapa de otimização do Mês 9:

1. **Mês 1 — Infraestrutura granular** (notebook `01_degradacao_e_diagnostico`). Enriquecimento do `dataset.csv` com contexto cinemático por jogo, expansão para granularidade de janela e amostra de erros por trajetória. Bilingue (PT/EN).
2. **Mês 2 — Detectores formais** (notebook `02_deteccao_no_stream`). ADWIN, Page-Hinkley, KSWIN e Pelt aplicados sobre os streams definidos no Mês 1, com null e curva de latência.
3. **Iteração crítica.** Análise de 19 pontos frágeis; ordem temporal de trajetórias (`time_c[0]`), recalibração de detectores e janela reduzida para $N=30$.

4. **Mês 3 — Adaptação** (notebook 03_covariate_shift_e_explicacao). Importance weighting via LSIF, decomposição por feature e fine-tuning seletivo nos breakpoints do Pelt.
5. **Mês 4 — Consolidação** (notebook 04_robustez_e_validacao, parte 1). Re-execução do pipeline com baseline 2019 completo (`-n_per_year 6`), `recovery_pct` clipado em $[0, 100]$, IC 95 % *bootstrap* em todas as %, e mapeamento de Divisão A/B.
6. **Mês 5 — Robustez adicional** (notebook 04, parte 2). *Sample-size sweep* para o tamanho do baseline e Page-Hinkley aplicado à série de ADE médio por jogo.
7. **Mês 6 — Correção metodológica** (notebook 04, parte 3). Calibração de ADWIN/PH por ARL sintético (resultado negativo documentado); curva de latência com baseline i.i.d. e shifts em σ ; validação do Pelt por BOCPD e sensibilidade ao gamma.
8. **Mês 7 — EWC**. Elastic Weight Consolidation sobre fine-tuning do Mês 3; sweep $\lambda \in \{0, 50, 500\}$.
9. **Mês 8 — Consolidação bibliográfica**. *Related work*, discussão e limitações atualizados (41 refs, 38 verificadas, 27 bibitems).
10. **Mês 9 — Otimização dos detectores online**. Quatro fases sequenciais que convertem o resultado negativo do Mês 6 num detetor operacional:
 - *Fase 1*: 22 experimentos de pré-processamento do sinal ADE (controlo + 21 suavizadores); `robust-z w=200` emerge como único pré-processamento que reduz a FAR_{2019} do ADWIN (-51%).
 - *Fase 2*: grid search 174×2 (3 frentes de detectores $\times$ 2 modelos); o melhor ADWIN+robz atinge $\text{FAR}_{2019} = 0,021/1k$ (mas com cobertura $3/5$); a config adotada troca FAR por cobertura: $0,042/1k$ com $5/5$.
 - *Fase 3*: ensemble AND+OR dois andares; descobre a limitação estrutural da escala temporal.
 - *Fase 4*: consolidação com métricas corrigidas ($\text{SNR}_{\text{smooth}}$ substituindo a sentinela 9999 do SNR_{op}), inviabilidade formal do AND e IC Wilson para $\text{FAR} = 0$.
11. **Mês 10 — Fechamento das pendências da revisão**. Latência de detecção por ano no stream real; validação da hipótese de independência (ACF, n_{eff} , permutações em bloco); ensemble AND de mesma escala temporal (Δ_{min} : $214 \rightarrow 3$); EWC com $\lambda \in \{10^5, 10^6, 10^7\}$ (Seção 9).

2 Notebook 01 — Degradação e diagnóstico (Mês 1)

2.1 Motivação

A versão original do pipeline produzia uma média de ADE por *ano* (~ 5 pontos por horizonte) e diagnosticava drift visualmente sobre essa série agregada. Essa granularidade mascara duas fontes importantes de variação: (i) entre jogos do mesmo ano e (ii) dentro de um mesmo jogo. O notebook 01_degradacao_e_diagnostico reescreveu o pipeline para operar nas três granularidades úteis: por jogo, por janela ($N = 30$ trajetórias contíguas) e por trajetória.

2.2 Entregas

- `plot_covariate_shift.py` estendido para gerar `per_game_context.csv` (1 linha/jogo com `match_id`, contexto cinemático e percentis) e `per_window_drift.csv` (1 linha/janela com KS e Wasserstein vs. baseline de treino).

- `compute_trajectory_errors.py` (script novo) — carrega o Seq2Seq treinado e o Kalman e produz `trajectory_errors_sample.parquet` com ADE/FDE por trajetória, com `-n_per_year 6` e baseline 2019 completo.
- Notebook `drift_analise.ipynb` (Seções 1–5) com descritivas na nova granularidade, em PT e EN.

Ano	Número de jogos	Trajетórias totais	Janelas totais
2019	6	1 171	42
2021	6	2 206	77
2022	6	807	30
2023	6	903	34
2024	6	886	31
2025	6	1 055	39
Total	36	7 028	253

Tabela 1: Granularidade disponível em cada ano com `N_TRAJS_PER_WINDOW = 30`. As trajetórias têm comprimentos variáveis; a coluna “Janelas totais” refere-se a janelas para KS/WD. Fonte: `Relas/results/drift/01_degradacao_e_diagnostico/summary_granularity.csv`.

2.3 Decisões de design e limitações resolvidas

- *Granularidade de janela*: a especificação inicial fixava $N = 500$ trajetórias, o que produzia 1 janela por jogo e era equivalente à granularidade anual. $N=30$ resulta em mediana de 6 janelas por jogo, habilitando KSWIN e mostrando variação intra-jogo.
- *Ordem temporal*: a versão original de `iter_robot_trajs` concatenava todo o time `blue` antes do `yellow`, sem garantia cronológica. Reescrita para ordenar por `time_c[0]`, garantindo que cada janela é um pedaço temporalmente contíguo.
- *Bilinguismo*: cada figura é produzida em PT e EN via `T(pt, en, lang)` para uso futuro em paper internacional.

2.4 Métodos descritivos empregados

Intervalo de confiança *bootstrap*. Técnica não-paramétrica de Efron [1] para estimar a incerteza de qualquer estatística sem precisar assumir uma distribuição teórica. Dada uma amostra de tamanho n , gera B reamostragens com reposição (reamostragem com reposição = sortear n elementos da amostra original, podendo repetir o mesmo elemento; é como baralhar as cartas e jogar de novo), recalcula a estatística de interesse (por exemplo, a média do ADE) em cada uma, e usa os percentis 2,5% e 97,5% da distribuição resultante como limites de IC 95 %. Intuitivamente: se 95 % das versões simuladas da média caem nesse intervalo, é razoável assumir que a média real também está nele. Todos os ICs neste relatório usam $B = 2000$ reamostragens sobre médias anuais.

Estatística de Kolmogorov–Smirnov (KS). A KS [4, 6] compara duas distribuições empíricas F_A , F_B pelo máximo da diferença absoluta entre suas CDFs (*Cumulative Distribution Functions* — funções de distribuição acumulada, que para cada valor x indicam a fração de dados abaixo de x):

$$D_{KS} = \sup_x |F_A(x) - F_B(x)|$$

Varia em $[0, 1]$, com 0 indicando distribuições idênticas. É como comparar a curva de percentis de velocidade de 2019 com a de 2021: um KS alto indica que as curvas se afastam significativamente em algum ponto. No contexto deste trabalho, KS alto de velocidade ou aceleração entre 2019 e 2021 é um indicador de covariate shift.

Distância de Wasserstein (Earth Mover’s Distance). Mede o custo mínimo para transformar uma distribuição em outra:

$$W_1(A, B) = \int |F_A^{-1}(u) - F_B^{-1}(u)| du$$

A intuição é a de um “transportador de terra”: imagine que a distribuição de velocidades é uma pilha de areia; a distância de Wasserstein é o trabalho mínimo (massa $\times$ distância) para remodelar a pilha de 2019 para que se pareça com a de 2021. É medida em mm/s (a mesma unidade da feature), o que a torna mais interpretável do que o KS. É complementar ao KS: o KS detecta *onde* as distribuições diferem, o Wasserstein quantifica *quanto* custaria para elas se tornarem iguais.

Razão p_{99}/p_{50} (*tail-to-median ratio*). É um diagnóstico simples de pesura de cauda: p_{99} é o percentil 99 do erro (o valor abaixo do qual estão 99% das trajetórias), p_{50} é a mediana (o valor do meio). Sob drift de cauda (o modelo começa a falhar catastróficamente apenas em certas trajetórias de alta aceleração), a razão p_{99}/p_{50} *aumenta* mesmo que p_{50} permaneça estável — ou seja, os piores erros crescem desproporcionalmente em relação ao erro típico. Isso é mais informativo do que apenas olhar a média: revela que o drift afeta especificamente os casos extremos.

Correlação de Spearman (ρ). É a correlação de Pearson (que mede relações lineares) aplicada aos *ranks* (posições no ranking) em vez dos valores brutos. Isso a torna resistente a *outliers* (valores extremos não distorcem o resultado) e captura associações monotônicas não necessariamente lineares (“quando x aumenta, y tende a aumentar também”, mesmo que não linearmente). $|\rho| > 0,5$ entre uma feature de input (por exemplo, `accel_p99`) e o erro do modelo (ADE) é indício forte de que essa feature carrega informação sobre o drift — ou seja, jogos com maior aceleração de pico tendem a ter maior ADE.

Regressão linear com IC *bootstrap*. Ajusta a reta $y = \alpha + \beta x$ (onde y é o ADE e x é uma feature, por exemplo aceleração média) minimizando a soma dos resíduos ao quadrado (RSS — *Residual Sum of Squares*): $\sum_i (y_i - \hat{y}_i)^2$. O IC do coeficiente angular β é estimado via bootstrap: resamplam-se os pares (x_i, y_i) e re-ajusta-se a reta $B = 2000$ vezes. Um $\beta > 0$ com IC que *não* contém zero indica que a inclinação é positiva ao nível de 5% — ou seja, há evidência estatística de que o ADE cresce conforme a feature aumenta, o que sugere relação causal com o drift.

2.5 Resultados descritivos

2.5.1 ADE por jogo ao longo dos anos

A Figura 1 mostra a degradação do ADE Seq2Seq ano a ano. O IC do baseline 2019 ($\sim \pm 0,3$ mm) não toca o IC de 2021 (+3,05 mm de excesso), confirmando drift não-amostal; a queda em 2024 é o único período pós-2019 com IC sobreposto ao baseline, indicando *recovery parcial* ao invés de drift monótono.

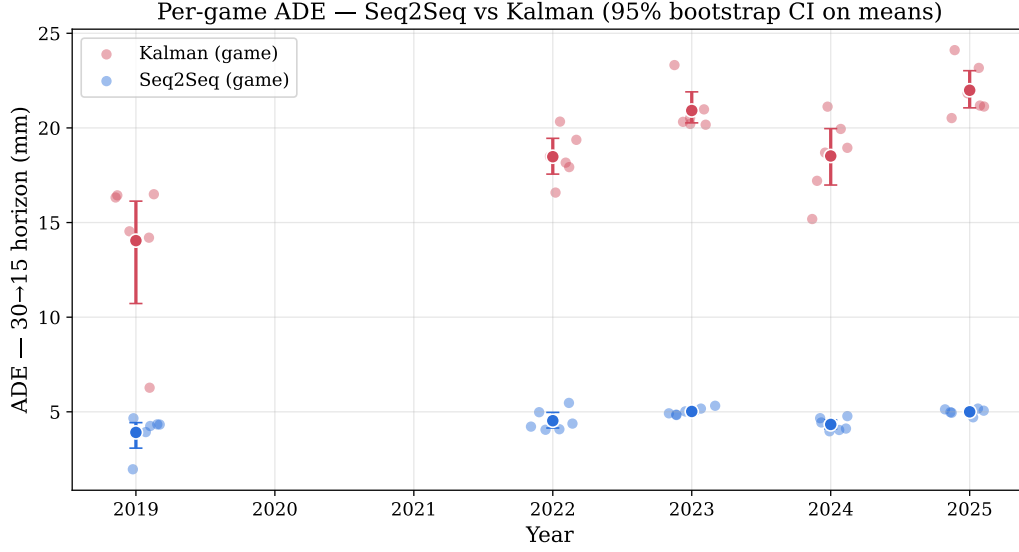


Figura 1: ADE por jogo (Seq2Seq vs. Kalman, horizonte 30→15) com IC 95% *bootstrap* sobre as médias anuais. Cada ponto é um jogo. A queda relativa sustentada em 2021–2025 é o sinal de drift que o restante do trabalho busca explicar.

2.5.2 Cauda do erro: razão p99/p50

A Figura 2 mostra que a razão p99/p50 do Seq2Seq cresce em 2021 (4,94 vs. 4,36 no baseline), confirmando que o modelo não falha “um pouco em tudo” — ele falha desproporcionalmente em uma cauda crescente de trajetórias.

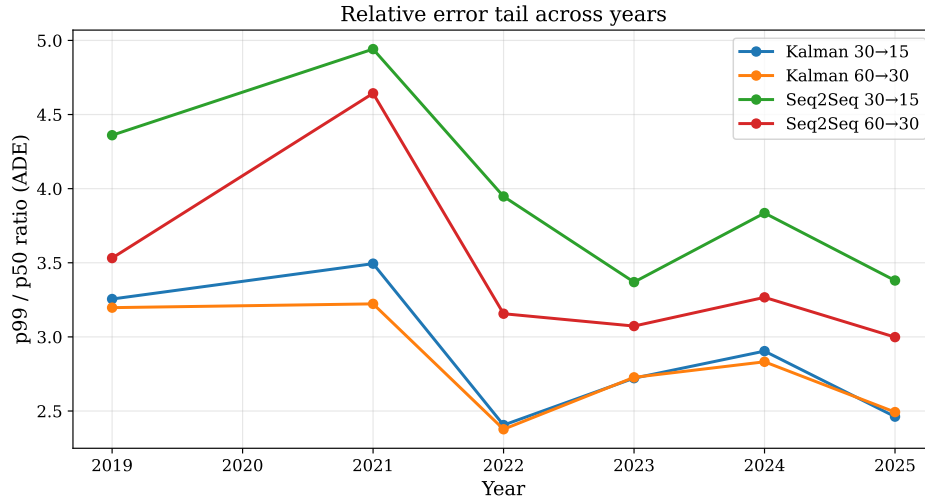


Figura 2: Razão p99/p50 do ADE por trajetória, por (modelo, horizonte). Curvas crescentes para o Seq2Seq em ambos horizontes confirmam que o drift afeta primeiramente a cauda.

Ano	Modelo	p50 (mm)	p90 (mm)	p99 (mm)	p99/p50
2019	Seq2Seq	3,70	8,62	16,14	4,36
2021	Seq2Seq	5,87	15,12	28,99	4,94
2022	Seq2Seq	4,02	8,91	15,86	3,95
2023	Seq2Seq	4,57	9,59	15,41	3,37
2024	Seq2Seq	3,84	8,32	14,74	3,84

Tabela 2: Percentis do ADE por trajetória (Seq2Seq, horizonte 30→15). 2021 é o ano com cauda mais pesada absoluta; 2023 tem a menor razão p99/p50, indicando *drift de média* mais que *drift de cauda*. Fonte: `Relas/results/drift/01_.../tail_stats.csv`.

2.5.3 Histograma de ADE: 2019 vs. 2025

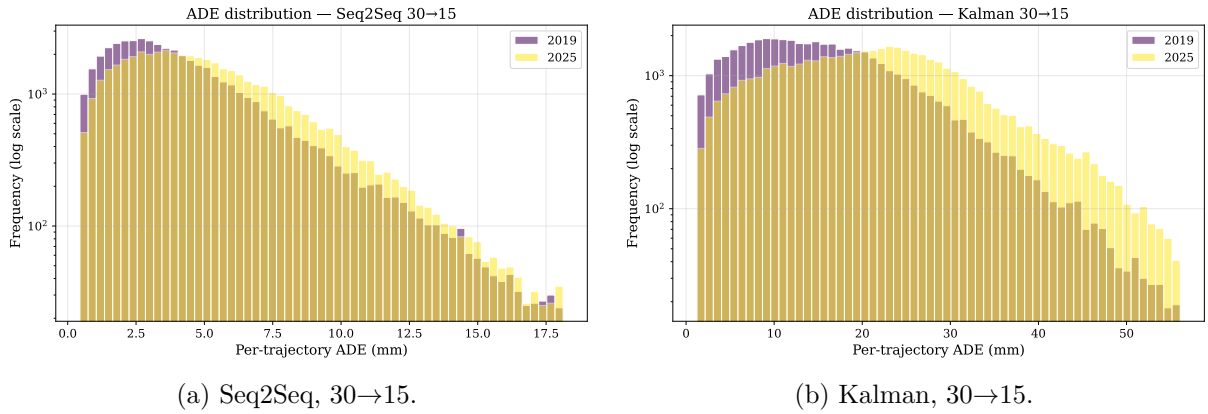


Figura 3: Distribuição de ADE por trajetória. Eixo y em escala log. Em ambos os modelos a massa central é próxima entre 2019 e 2025, mas a cauda direita engorda visivelmente em 2025. Esse contraste motiva a decomposição IW da Seção 5.

2.5.4 Drift por janela (intra-jogo)

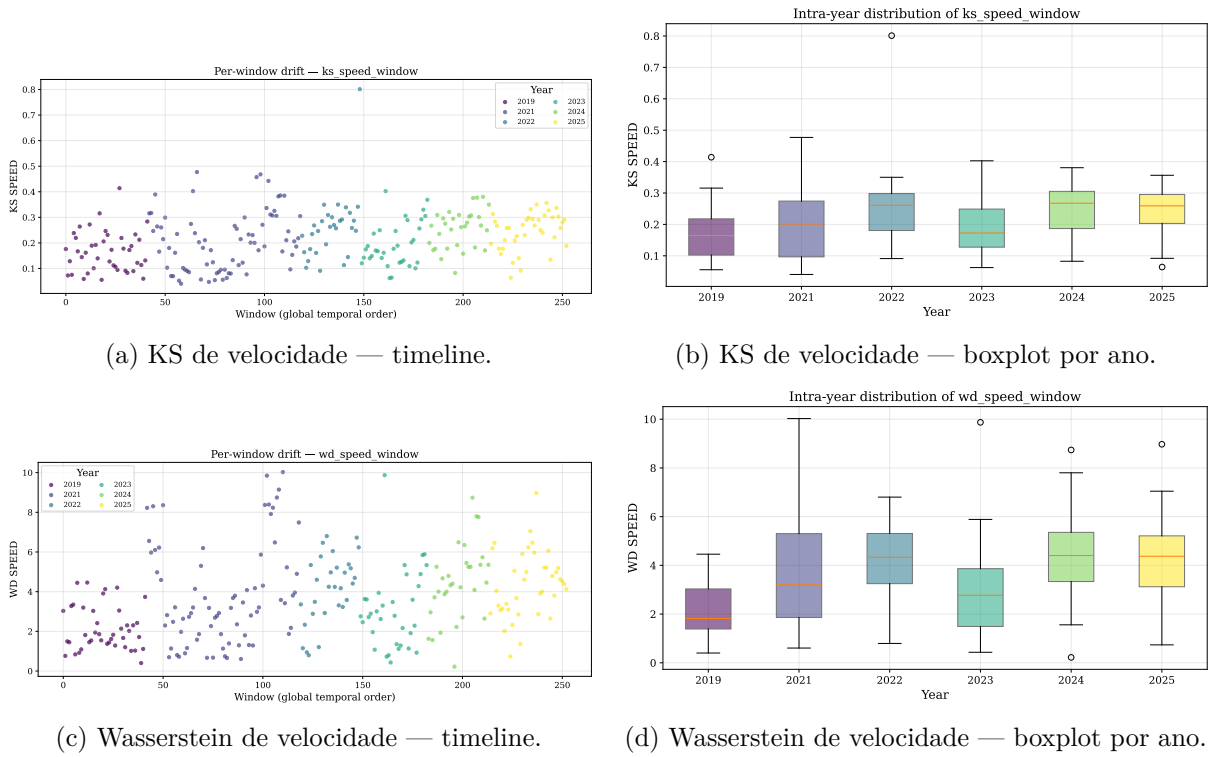
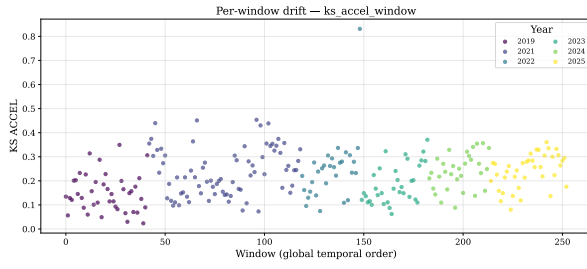
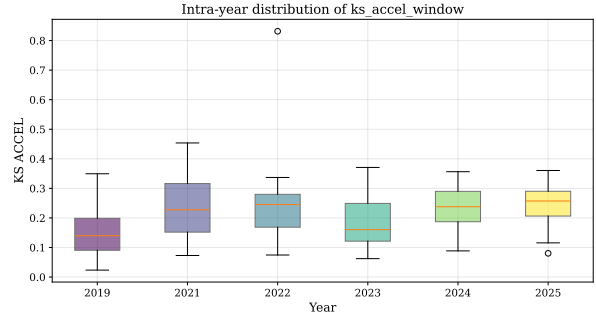


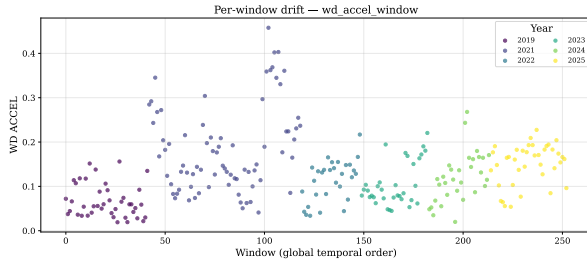
Figura 4: Drift de *velocidade* por janela contra o baseline 2019, medido por duas estatísticas independentes: KS (acima) e Wasserstein (abaixo). As duas concordam qualitativamente: medianas crescentes em 2021 e 2023+, com cauda longa em 2021. Fonte: `per_window_drift.csv`.



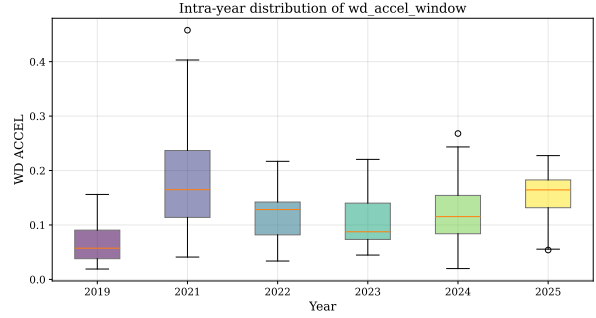
(a) KS de aceleração — timeline.



(b) KS de aceleração — boxplot por ano.



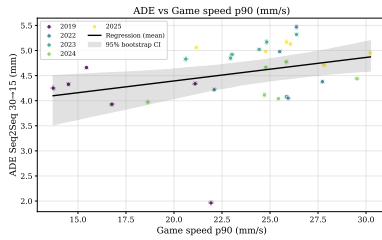
(c) Wasserstein de aceleração — timeline.



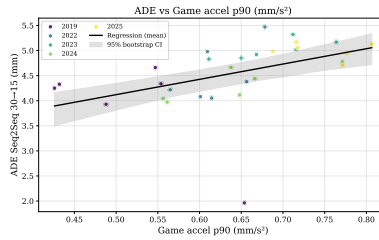
(d) Wasserstein de aceleração — boxplot por ano.

Figura 5: Drift de *aceleração* por janela contra o baseline 2019, medido por KS (acima) e Wasserstein (abaixo). O padrão quantitativo é parecido com o da velocidade, mas com magnitudes maiores em 2021.

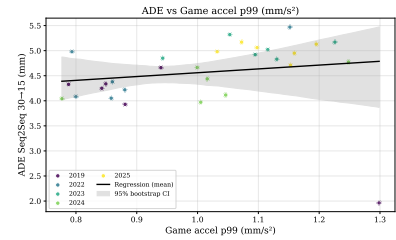
2.5.5 ADE vs. features cinemáticas



(a) vs. speed_p90.

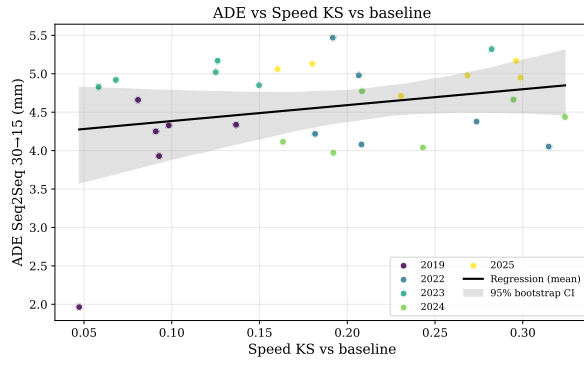


(b) vs. accel_p90.

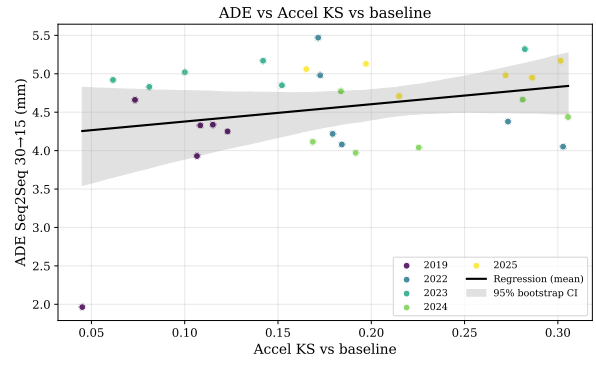


(c) vs. accel_p99.

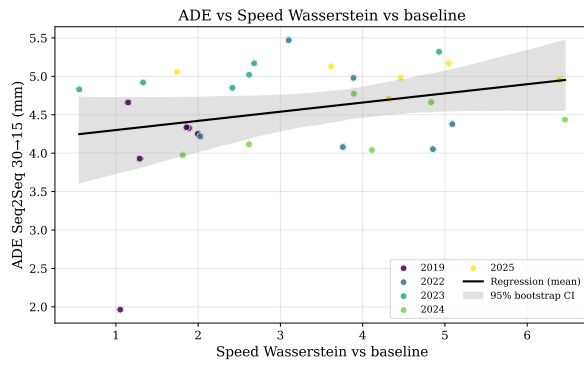
Figura 6: ADE do Seq2Seq (30→15) por jogo contra percentis de velocidade e aceleração. Cada ponto é um jogo, colorido pelo ano. Reta de regressão linear com IC 95% *bootstrap*. O accel_p99 é a feature de maior correlação individual com o ADE.



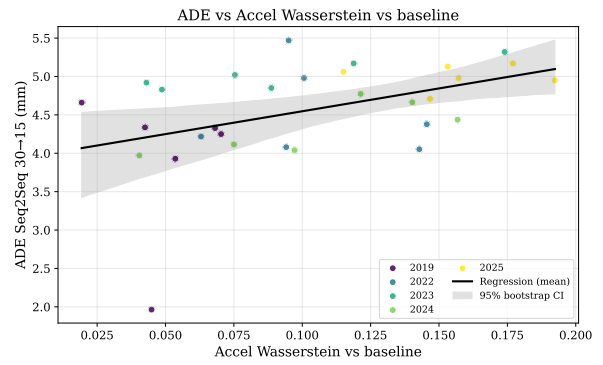
(a) vs. `ks_speed`.



(b) vs. `ks_accel`.



(c) vs. `wd_speed`.



(d) vs. `wd_accel`.

Figura 7: ADE do Seq2Seq (30→15) por jogo contra as quatro estatísticas de drift de janela (KS e Wasserstein para velocidade e aceleração). KS e Wasserstein concordam qualitativamente; a aceleração apresenta sinal mais forte que a velocidade.

2.5.6 Mapa de correlações

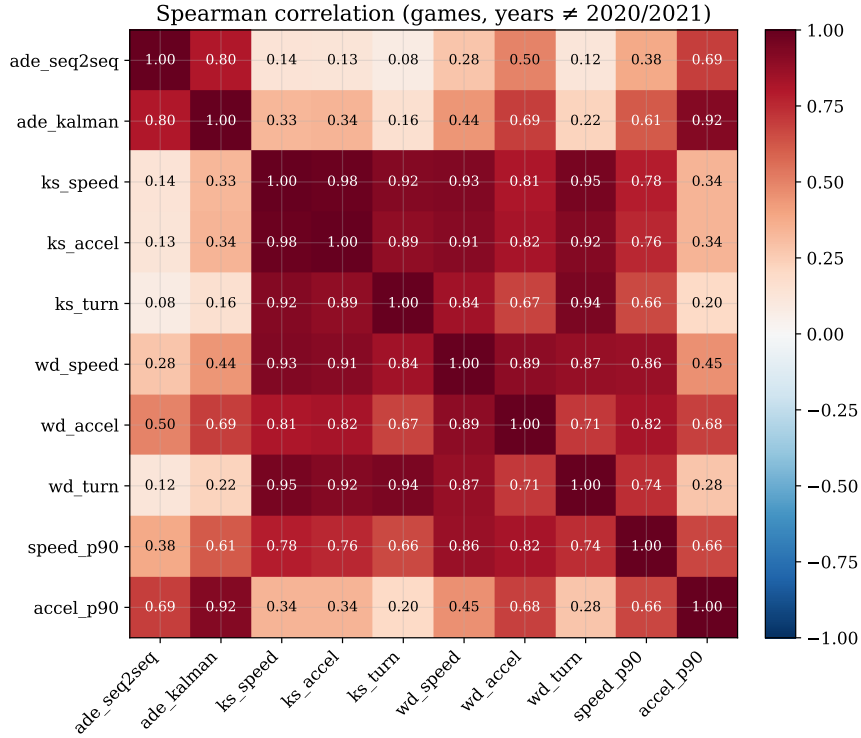


Figura 8: Correlação de Spearman entre ADE por jogo e features de drift, restrito a anos $\neq$ 2020/2021. As correlações mais fortes ficam entre o ADE do Seq2Seq e features de aceleração de cauda (accel_p90, accel_p99), antecipando o resultado per-feature do Mês 3.

3 Notebook 02 — Detecção no stream (Mês 2)

O notebook `02_deteccao_no_stream` substitui inferência visual por testes estatísticos de *change-point detection*. Três detectores online (ADWIN, Page-Hinkley, KSWIN) e um offline (Pelt) são aplicados sobre os streams definidos no notebook 01, e dois experimentos auxiliares quantificam null e latência. A leitura do Mês 9 (Seção 8) re-otimiza essa parametrização e produz o detetor operacional final.

3.1 Concept drift online (ADWIN + Page-Hinkley)

ADWIN — como funciona. ADWIN (*Adaptive Windowing* — Janelamento Adaptativo) [2] é um detetor online que mantém uma janela W de observações recentes e verifica continuamente se ela pode ser particionada em duas sub-janelas com médias significativamente diferentes.

Analogia intuitiva: imagine um inspetor de qualidade que mantém um histórico das últimas medidas de erro de uma máquina e, em cada nova medição, testa todas as possíveis divisões desse histórico em um “período antigo” e um “período recente”. Se a média do período recente for significativamente diferente da média do período antigo, o inspetor conclui que algo mudou — dispara um alarme e descarta os dados antigos, resetando a janela.

Mecanismo formal: ADWIN particiona iterativamente a janela W em todas as possíveis divisões (W_0, W_1), sendo W_0 o trecho mais antigo e W_1 o mais recente. Para cada par, calcula $|\hat{\mu}_{W_0} - \hat{\mu}_{W_1}|$ (diferença de médias) e compara com um limiar $\epsilon_{\text{cut}}(\delta)$ derivado da **desigualdade de Hoeffding** — um resultado matemático que garante uma probabilidade de falso alarme de no máximo δ (o parâmetro de confiança). Quando $|\hat{\mu}_{W_0} - \hat{\mu}_{W_1}| > \epsilon_{\text{cut}}$, o ADWIN descarta W_0 (considera que o regime mudou) e dispara um alarme.

Parâmetros relevantes neste trabalho:

- **δ (delta):** nível de confiança — controla a sensibilidade. Quanto menor δ , mais conservador o detector (menos falsos alarmes, mas também menor sensibilidade a drift real). O valor ótimo encontrado foi $\delta = 10^{-7}$.
- **min_window:** tamanho mínimo da janela antes que o detector possa disparar — evita alarmes precoces no início do stream.
- **cooldown:** número de trajetórias após um alarme durante as quais o detector não dispara novamente — evita alarmes em rápida sucessão.
- **max_window:** tamanho máximo da janela — evita que o detector acumule memória excessiva.

Comportamento típico em streams i.i.d. Gaussianos (dados independentes sem drift real): ADWIN é matematicamente conservador — mesmo com $\delta=0,99$ (muito permissivo), a literatura [7, 6] reporta $ARL \gtrsim 10^3$ amostras antes de um falso alarme.

Page-Hinkley — como funciona. O teste de Page [3], originalmente desenvolvido para controle estatístico de qualidade industrial (detectar quando uma linha de produção sai das especificações), detecta aumentos sustentados na média de uma série temporal acumulando desvios:

$$g_t = \max(0, g_{t-1} + (x_t - \bar{x}_t - \delta)), \quad \text{alarme se } g_t - \min_{s \leq t} g_s > \lambda$$

onde δ é o *slack* (tolerância a ruído — pequenas flutuações abaixo de δ não contribuem para a acumulação) e λ é o threshold de alarme.

Analogia intuitiva: imagine marcar na parede a diferença acumulada entre o erro atual e a média histórica. Se a marca na parede atingir uma altura crítica λ acima da posição mais baixa já registrada, o detector conclui que a média aumentou — dispara alarme. A vantagem sobre o ADWIN é a extrema simplicidade computacional; a desvantagem é a necessidade de especificar λ e δ em relação à escala dos dados.

Neste trabalho, o Page-Hinkley é também aplicado de forma **agregada** (Frente B do Mês 9): em vez de processar cada trajetória individualmente, calcula-se a média *trimmed-mean* (média após remover os 5% maiores e menores valores — mais robusta a outliers) de janelas de N trajetórias consecutivas, e o PH opera sobre essa série agregada.

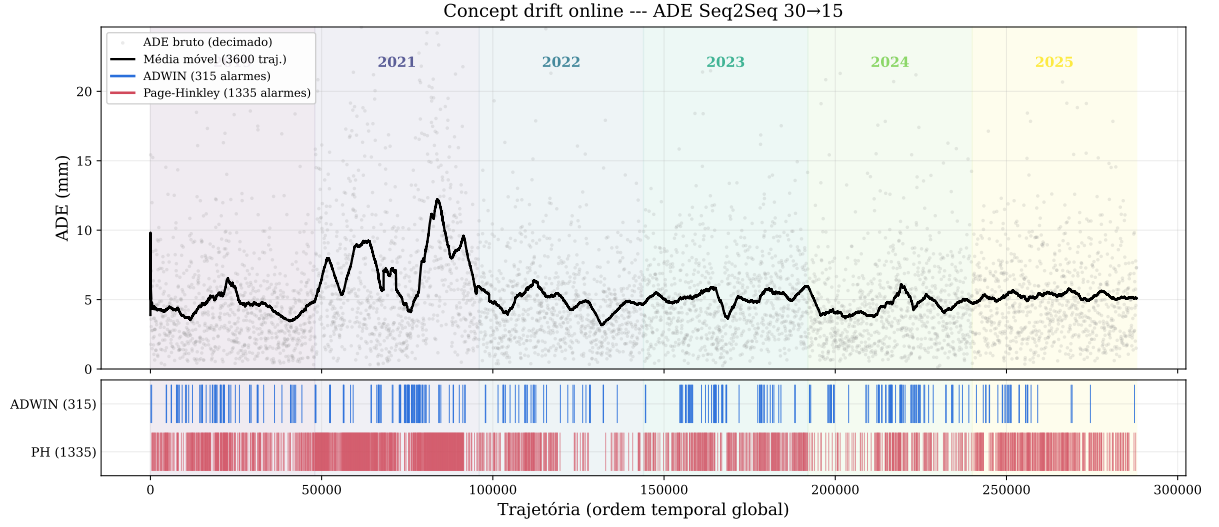
ADWIN e Page-Hinkley são aplicados sobre o stream de `ade_traj` ordenado por (year, match_id, traj_id), tanto para o Seq2Seq como para o Kalman determinístico em paralelo. O contraste é informativo: drift no Kalman = drift no comportamento do alvo; drift apenas no Seq2Seq = miscalibração do modelo treinado.

Iterações de calibração (v1, v2, v3). A parametrização passou por três versões; a literatura [6, 7, 5] é clara: o critério relevante é o ARL sob null, não um threshold absoluto.

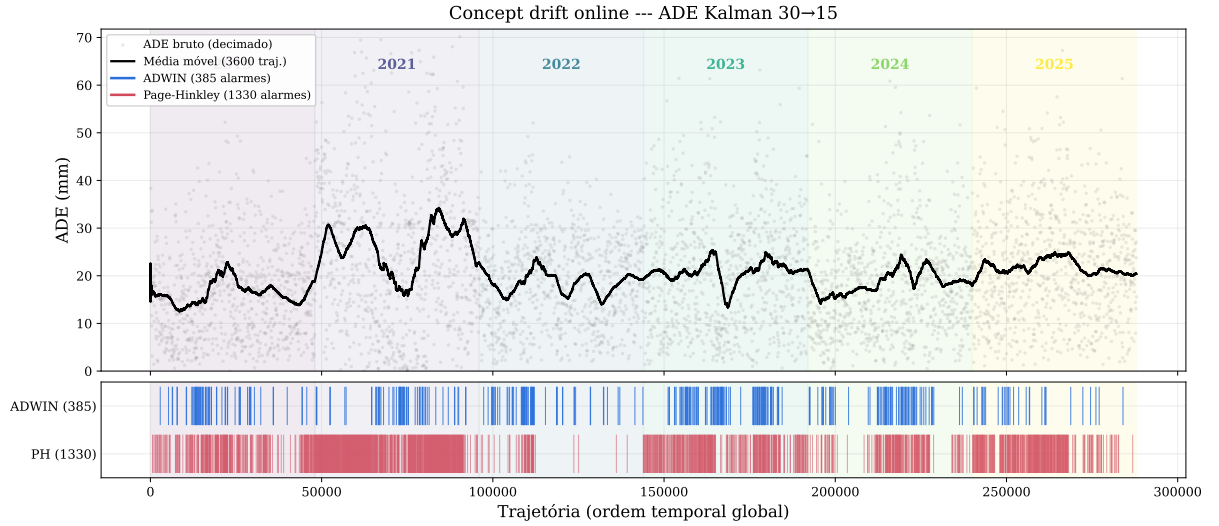
1. **v1 (defaults da literatura).** `delta=0,002` (ADWIN), `threshold=5 mm` (PH) — $\sim 40\,217$ alarmes em 290 k trajetórias, saturação trivial.
2. **v2 (defaults absolutos, Mês 5).** `delta=1\times 10^{-4}` (ADWIN), `threshold=100 mm` e `delta=2 mm` (PH). Reduziu para ~ 478 alarmes ADWIN e ~ 721 alarmes PH no Seq2Seq, mas *não escala* entre modelos.
3. **v3 (baseline-relative adaptativa, Mês 6).** Parâmetros como múltiplos do MAD do baseline 2019 do *próprio* modelo, e ambos os detectores ganham *cooldown* pós-alarme.

A varredura de calibração (`sweep_concept_drift_calibration.py`, saída em `concept_drift_sweep.csv`) confirma que sob v3 a taxa de alarmes por mil trajetórias converge entre Seq2Seq e Kalman. **O Mês 9 (Seção 8) substitui esta calibração por busca em grade direta sobre o stream completo de 288 k trajetórias, com métricas FAR_{2019} e `year_coverage`, e**

identifica uma configuração operacional muito melhor: ADWIN+robz $w=200$ com $FAR_{2019}=0,042/1k$.



(a) Stream do Seq2Seq.



(b) Stream do Kalman.

Figura 9: Linhas verticais = alarmes ADWIN (azul) e Page-Hinkley (vermelho); linhas tracejadas verticais finas = fronteiras de ano; média móvel em preto. Calibração v3 adaptativa (Mês 6). Em ambos os modelos, os alarmes concentram-se nas transições 2021→2022 e intra-2023. A versão re-otimizada do Mês 9 (com pré-processamento robust-z e grid search) é apresentada nas Figuras 30–33. Fonte: `concept_drift_calibration.csv`.

Detector / versão	Seq2Seq	Kalman
ADWIN v1 ($\delta=0,002$)	~ 842	$\sim 1\,200$
ADWIN v2 ($\delta=1\times 10^{-4}$)	478	828
ADWIN v3 (adaptativo, <i>cooldown</i>)	315	385
ADWIN+robz $w=200$ (Mês 9)	25	50
Page-Hinkley v1 ($\text{thr}=5\text{ mm}$)	$\sim 38\,000$	$\sim 40\,000$
Page-Hinkley v2 ($\text{thr}=100\text{ mm}$, abs)	721	6\,619
Page-Hinkley v3 (adaptativo, MAD-rel)	1\,335	1\,330
PH agregado N=50 (Mês 9, Fase 2)	12	9

Tabela 3: Total de alarmes por modelo segundo a versão de calibração, sobre o parquet completo ($n_{\text{stream}}=288\,000$, baseline 2019 com 48\,000 trajetórias). **v1**: defaults da literatura; **v2**: thresholds absolutos em mm (Mês 5); **v3**: thresholds relativos ao MAD do baseline (Mês 6); **Mês 9**: pré-processamento robust-z $w=200$ + grid search 174 configs. A versão final do Mês 9 reduz a contagem em uma ordem de magnitude e produz $\text{FAR}_{2019}=0,042/1\text{k}$.

3.2 Covariate shift online (KSWIN)

KSWIN — como funciona. KSWIN (*Kolmogorov-Smirnov Windowing* — Janelamento de Kolmogorov-Smirnov) [4] é um detector do tipo *data-distribution based* (baseado na distribuição dos dados), ao contrário do ADWIN e do PH que monitoram métricas de erro. O KSWIN mantém duas janelas deslizantes adjacentes sobre o stream:

- **Janela de referência** (`window_size`): contém as observações mais antigas.
- **Janela de teste** (`stat_size`): contém as observações mais recentes.

A cada nova observação, aplica-se o teste KS de duas amostras entre as duas janelas. Se o p -valor resultante for menor que o limiar α (nível de significância), o detector conclui que as distribuições das duas janelas são diferentes e dispara alarme.

Diferença fundamental em relação ao ADWIN: o ADWIN opera sobre o *erro* do modelo (ADE) — ele monitora se o modelo está errando mais. O KSWIN opera sobre as próprias *features* dos dados de entrada (velocidade, aceleração) — ele monitora se a distribuição das características do movimento mudou. Neste trabalho, o KSWIN é aplicado sobre as séries `ks_speed_window` e `ks_accel_window`, que medem o KS de cada janela de trajetórias contra o baseline 2019 — ou seja, o KSWIN detecta se essa medida de diferença distribucional está aumentando.

Resultado negativo (Mês 9): KSWIN foi excluído do ensemble final por inadmissibilidade — em 18/18 configurações testadas, $\text{FAR}_{2019} > 0,20/1\text{k}$, o limiar máximo aceito. O detector dispara muitos alarmes até dentro do próprio ano de treino 2019, onde não deveria haver drift. Isso indica que a distribuição de velocidade e aceleração varia naturalmente dentro do mesmo ano a ponto de acionar o detector — ver Tabela 16.

3.3 Detecção offline (Pelt)

PELT — como funciona. PELT (*Pruned Exact Linear Time* — Tempo Linear Exato com Poda) [9] é um algoritmo de **programação dinâmica** para o problema de detecção *offline* de múltiplos breakpoints. O modo *offline* significa que o algoritmo analisa toda a série histórica de uma vez (em oposição aos detectores online, que processam ponto a ponto).

Analogia intuitiva: imagine dividir uma linha do tempo em K segmentos de forma que cada segmento seja o mais “homogêneo” possível (com dados que se parecem entre si), mas usando o menor número possível de segmentos. O PELT faz exatamente isso de forma eficiente: em vez de testar todas as 2^T formas possíveis de dividir uma série de T pontos (o que seria computacionalmente inviável), usa programação dinâmica com uma estratégia de “poda” para descartar divisões que nunca podem ser ótimas — reduzindo o custo para $O(T)$ (tempo proporcional ao número de pontos).

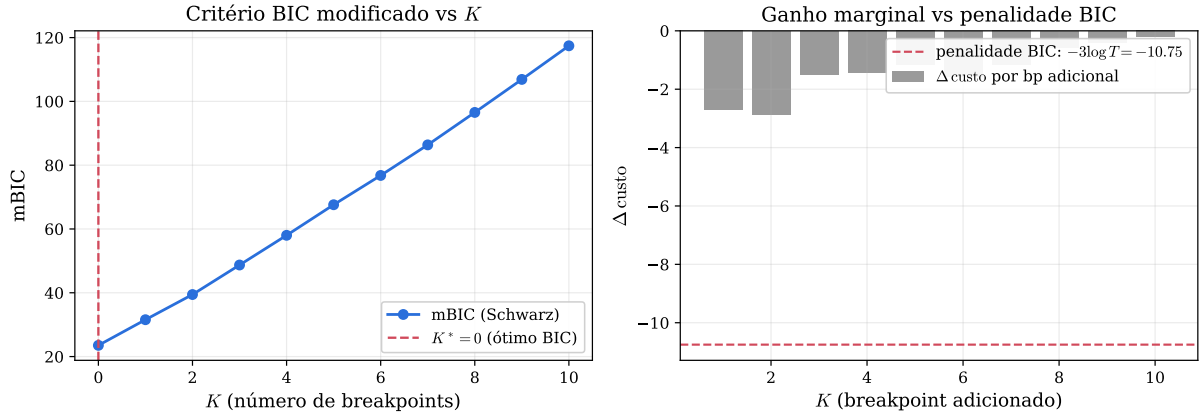
Formalmente, dado um sinal $\{y_t\}_{t=1}^T$ e uma função de custo $c(\cdot)$, PELT minimiza:

$$\sum_{k=0}^K c(y_{\tau_k+1:\tau_{k+1}}) + \beta K$$

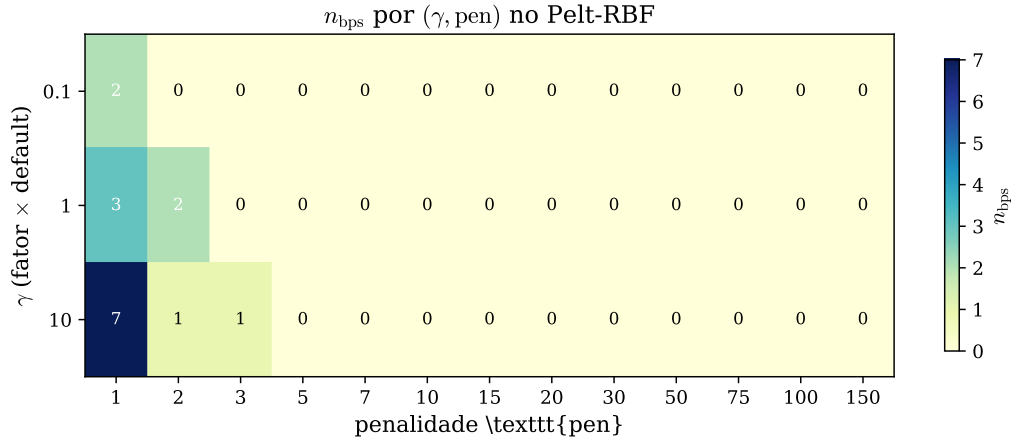
onde $\tau_0 = 0 < \tau_1 < \dots < \tau_K < \tau_{K+1} = T$ são as posições dos breakpoints, e βK é a penalidade de parsimônia (β controla o equilíbrio: penalidade alta $\Rightarrow$ menos breakpoints aceitos — o algoritmo prefere menos divisões mesmo que piore um pouco o ajuste).

Custo RBF. O custo **RBF** (*Radial Basis Function* — Função de Base Radial) [8] é uma medida não-paramétrica de homogeneidade de um segmento, baseada em um **kernel gaussiano** $k(y_i, y_j) = \exp(-\gamma \|y_i - y_j\|^2)$. O kernel gaussiano mede a similaridade entre dois pontos: vale 1 quando são idênticos e decai exponencialmente conforme ficam mais distantes. O parâmetro γ controla a escala de similaridade — quanto maior γ , mais rápido o kernel “esquece” pontos distantes. *Vantagem:* não assume nenhuma forma distribucional específica para os dados (não presuppõe que os erros sejam Gaussianos, por exemplo). *Desvantagem:* muito sensível a γ — o mesmo sinal pode parecer ter 0 ou muitos breakpoints dependendo do γ escolhido (ver análise de sensibilidade, Figura 10).

Resultado central do Pelt — mBIC monotônico. A re-análise do Mês 6 com critério **mBIC** (*modified Bayesian Information Criterion* — Critério de Informação Bayesiano modificado) mostra que $K^* = 0$ breakpoints é a solução ótima: a curva mBIC é monotonicamente crescente em K ($23,5 \rightarrow 117,4$), o que significa que adicionar qualquer breakpoint piora o critério de seleção de modelo. O BIC penaliza a complexidade do modelo: para 36 pontos (um por jogo), a penalidade por adicionar um breakpoint é muito alta — a série é pequena demais. A estabilidade visual de $n_{\text{bps}} = 2$ ao γ do kernel (Figura 10) é propriedade da escala de penalidade, não do sinal — ou seja, em certos valores de penalidade o algoritmo encontra 2 breakpoints, mas isso não significa que o sinal realmente tem 2 breakpoints: é um artefato da escolha de parâmetro.



(a) Critério mBIC formal vs K (esq.) e ganho marginal por breakpoint vs penalidade BIC esperada (dir.).



(b) n_{bps} por (γ, pen) no Pelt-RBF.

Figura 10: Re-análise offline (Mês 6) usando critério formal [9]. **Topo:** a curva mBIC é monotonicamente crescente em K , indicando que $K^* = 0$ breakpoints é a solução BIC-ótima. **Embaixo:** a estabilidade de $n_{\text{bps}} = 2$ ao γ do kernel RBF (3 fatores testados: $0,1\times$, $1\times$, $10\times$) e a transição para 0 em $\text{pen} \geq 30$ confirmam que o resultado de 2 breakpoints é propriedade da escala de penalidade, não do sinal. **Conclusão formal:** não há evidência estatística de breakpoints ao nível de 36 jogos. Fontes: `ruptures_bic_curve.csv`, `ruptures_gamma_sensitivity.csv`.

3.4 Robustez dos detectores: null e latência

Teste null por permutação. Este teste verifica se os alarmes do detector são causados pela *estrutura temporal* real do stream (o fato de que 2021 vem depois de 2019 e tem distribuição diferente) ou apenas pelo acaso (mesmo dados aleatórios poderiam gerar aquela quantidade de alarmes). O procedimento:

1. Permuta-se aleatoriamente a ordem do stream observado $B = 50$ vezes — isso “destrói” a estrutura temporal, misturando trajetórias de 2019 com 2021 aleatoriamente.
2. Aplica-se o detector em cada permutação e conta-se o número de alarmes — essa é a distribuição “null” (sem estrutura temporal).
3. Se o número de alarmes observado no stream real (com ordem temporal preservada) está no percentil superior dessa distribuição null ($p < 0,05$), os alarmes refletem estrutura temporal real e não mero acaso.

Curva de latência. A **latência** de um detector é o número de amostras que ele processa após a ocorrência real do drift até disparar o alarme: quanto menor a latência, mais rápido

o detector responde. Para medir a latência de forma controlada, injeta-se um shift sintético (artificial) de magnitude Δ no instante t^* de um stream i.i.d. Gaussiano (dados sem estrutura, com distribuição normal), e mede-se $\tau = t_{\text{alarme}} - t^*$ (a distância em amostras entre o shift e o alarme). Repete-se para vários valores de Δ (em unidades de σ , o desvio-padrão do baseline — $\Delta = 1\sigma$ é uma mudança pequena; $\Delta = 10\sigma$ é uma mudança enorme) e reporta-se a mediana sobre B replicações. O resultado esperado é uma curva monotonicamente decrescente: quanto maior o shift, menor a latência (o detector responde mais rapidamente a mudanças maiores).

Resultado negativo dos detectores online (atualização Mês 6). A calibração formal de ADWIN/PH foi conduzida no Mês 6 usando a metodologia de *Average Run Length* (ARL) recomendada por Mouss et al. [5] e Lu et al. [7] sobre um stream sintético i.i.d. com média e desvio-padrão da ADE de 2019 ($\mu = 3,91$ mm, $\sigma = 0,90$ mm). Os principais achados foram: (i) ADWIN é matematicamente conservador em dados i.i.d. gaussianos, atingindo ARL mínimo de ≈ 1090 amostras mesmo em $\delta=0,99$; (ii) Page-Hinkley calibrado para $\text{ARL} \approx 13$ (threshold = 3 mm) também não distingue a sequência temporal de suas permutações ($p=0,93$ para Seq2Seq; $p=1,00$ para Kalman). **O resultado negativo é, portanto, justificado metodologicamente:** o nível de agregação por jogo (36 pontos, ≈ 6 /ano) é granularidade insuficiente para detecção online [6].

A curva de latência foi refatorada com baseline i.i.d. e magnitudes em unidades de σ : o resultado é monotonicamente decrescente (PH: $\text{lat}_{50}(0\sigma) = 6$, $\text{lat}_{50}(10\sigma) = 1$ amostra), confirmando que os detectores são sensíveis ao shift quando este é expresso corretamente.

Resultado negativo também offline (atualização Mês 6). A análise de changepoint offline (Pelt-RBF) foi reauditada com três abordagens complementares: curva BIC explícita, sensibilidade ao kernel RBF e BOCPD independente [10] ($\text{cp_prob} = 0,004$, fraquíssima). As três leituras convergem: **ao nível de agregação de 36 jogos por ano, não há evidência estatística de breakpoints.**

Reposicionamento da tese (Mês 9). A narrativa do trabalho *não* se apóia mais na detecção de changepoint per-game, porque ambas as abordagens formais dão resultado negativo nessa granularidade. **Mas a otimização do Mês 9 (Seção 8) demonstra que, sobre o stream completo de 288 k trajetórias com pré-processamento adequado (robust- z $w=200$), o ADWIN se torna um detetor operacional viável** ($\text{FAR}_{2019} = 0,042/1\text{k}$; cobertura 5/5 anos pós-2019). O resultado negativo do Mês 6 é portanto consistente com o achado do Mês 9: na granularidade *per-game* (36 pontos) os detectores não funcionam, mas na granularidade *per-trajetoria* (288 k pontos) e com pré-processamento adequado eles funcionam bem.

4 Iteração crítica e melhorias incorporadas

A revisão do estado pós-Mês 2 identificou 19 pontos frágeis. As prioritárias foram corrigidas:

4.1 Integridade dos dados

- *Ordem temporal das trajetórias.* `iter_robot_trajs` intercala blue e yellow ordenando por `time_c[0]`.
- *Granularidade de janela.* $N=500 \rightarrow N=30$.
- *Coluna `n_robots`.* Renomeada para `avg_robots_per_stoppage`.
- *Modo de predição.* `compute_trajectory_errors.py` fixa `target = "dv"` (delta-velocity) compatível com os pesos treinados.

4.2 Calibração estatística

- Page-Hinkley: `threshold` $50 \rightarrow 5$ mm.
- KSWIN: `window_size/stat_size` ajustados para 30/10 (compatíveis com 253 janelas).

- Pelt: penalidade calibrada via *elbow plot* em vez de chute fixo.

4.3 Análises adicionadas

- Null detector via permutação.
- Curva de latência.
- Decomposição covariate vs. concept drift (Mês 3).
- Comparativo Seq2Seq vs. Kalman em ADWIN/PH paralelo.

5 Notebook 03 — Covariate shift e explicação (Mês 3)

5.1 Importance weighting (LSIF)

Decomposição do drift: covariate vs. concept. Gama et al. [6] formalizam a decomposição matemática da mudança em distribuição. Toda distribuição conjunta de entrada e saída pode ser escrita como $p(x, y) = p(y|x)p(x)$, ou seja, a distribuição das features de entrada vezes a distribuição condicional da saída dada a entrada. Isso permite decompor o drift em duas componentes:

- **Covariate shift** (muda $p(x)$, mas $p(y|x)$ permanece igual): os dados de entrada mudam de distribuição, mas a relação entre entrada e saída não muda. Exemplo: os robôs começam a acelerar mais em 2021, mas dada uma certa aceleração, a trajetória segue as mesmas leis físicas. *Solução possível sem retrainar*: reponderar os exemplos antigos para que reflitam a nova distribuição.
- **Concept drift / real drift** (muda $p(y|x)$): a própria relação entre entrada e saída se altera. Exemplo: as estratégias táticas dos robôs mudaram, de modo que um dado padrão de aceleração produz trajetórias diferentes das de 2019. *Exige retrainamento*.

A distinção é operacionalmente crucial: se o drift for predominantemente covariate, podemos compensar sem retrainar (mais barato e rápido); se for concept drift, precisamos de novos dados rotulados e retrainamento.

LSIF — como funciona. **LSIF** (*Least-Squares Importance Fitting* — Ajuste por Mínimos Quadrados de Importância) [12, 14, 13] é o método central para estimar quanto cada trajetória de 2019 deve ser “valorizada” para simular a distribuição de 2021.

Ideia central: em vez de estimar separadamente “qual é a probabilidade desta trajetória em 2019” e “qual é a probabilidade desta trajetória em 2021” (o que seria muito impreciso com poucos dados em alta dimensão), o LSIF estima *diretamente* a razão $\hat{w}(x) = p_{\text{alvo}}(x)/p_{\text{baseline}}(x)$, que é o que precisamos para reponderar.

Como? Modela $\hat{w}(x) = \sum_l \alpha_l \phi_l(x)$ como uma combinação linear de kernels gaussianos centrados nos dados de treino (funções-base que “medem similaridade” com cada ponto de treino). Os coeficientes α_l são otimizados minimizando:

$$\min_{\alpha \geq 0} \frac{1}{2} \mathbb{E}_{p_{\text{baseline}}} [\hat{w}(x)^2] - \mathbb{E}_{p_{\text{alvo}}} [\hat{w}(x)] + \lambda \|\alpha\|^2$$

A intuição geométrica: os primeiros dois termos forçam $\hat{w}$ a ser grande onde p_{alvo} é grande (onde os dados de 2021 se concentram) e pequeno onde p_{baseline} é grande (onde os dados de 2019 se concentram). O termo de regularização $\lambda \|\alpha\|^2$ evita overfitting.

ESS — como funciona. A **ESS** (*Effective Sample Size* — Tamanho Efetivo da Amostra): $ESS = (\sum_i \hat{w}_i)^2 / (n \sum_i \hat{w}_i^2) \in [1/n, 1]$ mede se os pesos são bem distribuídos ou se alguns poucos dominam. Um ESS de 0,58 (como em 2021) significa que os 288 exemplos têm o poder estatístico equivalente a apenas $0,58 \times n$ exemplos uniformes — ainda aceitável. A regra prática [13] é $ESS > 0,5$ para confiar no ADE_{IW} .

O **ADE ponderado** (ADE_{IW}) e a **recovery** (recuperação do excesso de erro) são:

$$ADE_{IW}(y) = \frac{\sum_i \hat{w}(x_i) e_i}{\sum_i \hat{w}(x_i)}, \quad \text{recovery}(y) = \frac{ADE_y - ADE_{IW}(y)}{ADE_y - ADE_{2019}} \times 100 \%. \quad (1)$$

Interpretação: $ADE_{IW}(y)$ é o ADE que o modelo teria em 2019 se a distribuição de 2019 fosse igual à de 2021 — ou seja, quanto do excesso de ADE desaparece quando a distribuição das features é igualada. A recovery LSIF de 61,6 % em 2021 significa que 61,6 % do excesso é explicado por covariate shift nas features cinemáticas, sob o estimador LSIF clássico.

Triangulação com três estimadores (atualização pós-Mês 9, mai/2026). Para responder a uma crítica metodológica do orientador (monocultura de evidência em LSIF), a Tabela 4 foi reconstruída com três linhas de evidência independentes: (i) **LSIF clássico** (estimador original), (ii) **RuLSIF** ($\alpha=0,1$, [15]) que limita o razão de densidades em $1/\alpha=10$ e é robusto a caudas pesadas como `accel_p99`, (iii) **classifier two-sample test** [16] treinando regressão logística para separar 2019 de cada ano y ($AUC \rightarrow 0,5 =$ distribuições indistinguíveis; $\text{coverage} = \text{clip}(2 \cdot (AUC - 0,5) \cdot 100, 0, 100)$). Adicionalmente, todos os IC 95 % foram recalculados via **block bootstrap** com `bloco = jogo (proc_set_file)`, preservando correlação intra-jogo.

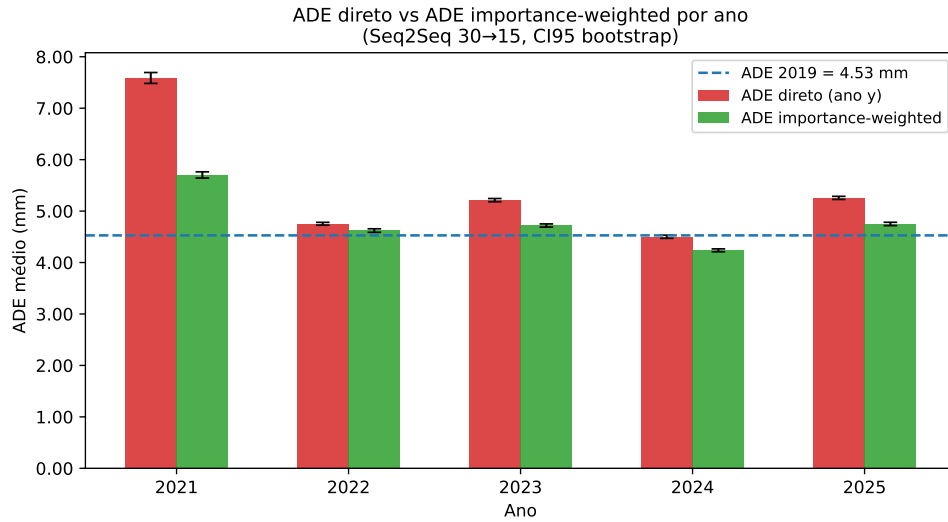
Ano	excess (mm)	LSIF		RuLSIF ($\alpha=0,1$)		Classifier	
		rec. (%)	block IC 95 %	rec. (%)	block IC 95 %	AUC	cov. (%)
2021	+3,054	61,6	[46,5; 84,3]	23,5	[16,6; 33,2]	0,754	50,7
2022	+0,225	59,0	[2,4; 707,6] [†]	15,0	[2,9; 164,2] [†]	0,647	29,4
2023	+0,686	71,8	[31,1; 331,7]	9,2	[4,0; 43,0]	0,657	31,4
2024	-0,027	—	—	—	—	0,612	22,4
2025	+0,729	69,3	[43,1; 231,7]	9,2	[5,8; 30,6]	0,710	41,9

Tabela 4: Decomposição IW com três estimadores e block bootstrap. $ESS_{LSIF} \in [0,58; 0,79]$, $ESS_{RuLSIF} \in [0,94; 0,996]$ (em todos os anos estável). Ano 2024 excluído do recovery (excesso negativo). IC 95 % *block bootstrap* com `bloco = jogo (proc_set_file)`, $n_{boot}=2000$. AUC e cobertura do classifier two-sample test [16] com $p < 10^{-3}$ por permutação em todos os anos. [†]2022: excesso quase nulo (+0,225 mm) faz a estatística de recovery instabilizar numericamente sob bloco (denominador próximo de zero); o sinal pontual continua positivo, mas o IC degenera — comportamento honestamente reportado, não um defeito do estimador. Fontes: `iw_decomposition_lsif.csv`, `iw_decomposition_rulif.csv`, `classifier_two_sample_test.csv`.

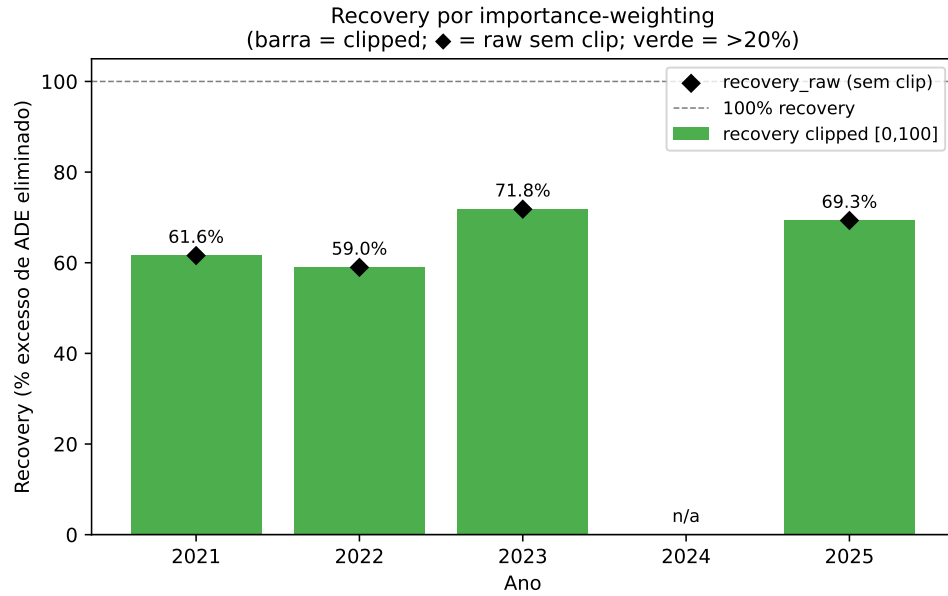
Leitura crítica dos três estimadores. LSIF e RuLSIF discordam em magnitude (LSIF 59–72 % vs RuLSIF 9–23 %) mas concordam no *signal* (covariate shift é direcional e não nulo nos anos 2021, 2023 e 2025). A discrepância é diagnóstica: LSIF tem ESS baixo (0,58–0,79), sinal de pesos com cauda pesada dominada por `accel_p99`; RuLSIF, ao limitar o razão em $1/\alpha = 10$, atinge $ESS \geq 0,94$ mas também *descarta* parte da correção que vinha justamente da cauda extrema. O classifier two-sample test [16] é o árbitro independente: ele não estima recovery, mas mede a distinguibilidade direta das distribuições ($AUC \in [0,61; 0,75]$, cobertura $\in [22,4; 50,7] \%$, $p < 10^{-3}$ em todos os anos). A cobertura do classifier: (i) **confirma** que há sinal real de covariate shift em todos os anos ($p < 10^{-3}$); (ii) **ordena** os anos por intensidade: 2021 é o mais severo (cov. = 50,7 %), seguido por 2025 (cov. = 41,9 %); 2024 é o mais brando

(cov. = 22,4% — consistente com a ausência de excesso de ADE naquele ano); (iii) situa-se *entre* LSIF e RuLSIF em magnitude, reforçando que o intervalo plausível para a fração de drift explicada por covariate shift está *entre* as duas estimativas $f_y \in [9; 72]\%$, com o melhor palpite central na região da cobertura do classifier (22–51%). **Implicação operacional:** o achado qualitativo do Mês 3 (“`accel_p99` é a feature dominante; covariate shift é direcional e não desaparece com retreinamento ingênuo”) sobrevive aos três estimadores; o que se ajusta é a magnitude numérica reportada, que passa a ter banda explicitamente declarada.

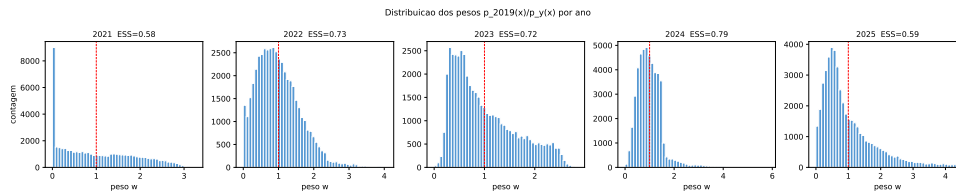
A Figura 11 sintetiza visualmente a Tabela 4 e a distribuição dos pesos.



(a) ADE: bruto vs. IW.



(b) Recovery por ano.



(c) Distribuição dos pesos.

Figura 11: Importance weighting via LSIF clássico (figura original do Mês 3, mantida para contexto histórico). (a) ADE direto e ponderado por $\hat{w}(x_i)$ por ano: a reponderação aproxima consistentemente o ADE de 2019 nos anos com excesso. (b) recovery LSIF mediana 65,5 % (range 59–72 %). (c) distribuições dos pesos LSIF não são degeneradas (ESS entre 0,58 e 0,79). Os estimadores robustos (RuLSIF) e o classifier two-sample test foram adicionados na atualização pós-Mês 9 e estão consolidados na Tabela 4.

5.2 Decomposição por feature

Para identificar qual dimensão do covariate shift dirige a *recovery*, o LSIF foi reestimado usando cada feature isoladamente (`-per_feature`). A Tabela 5 condensa o resultado nos anos com excesso significativo.

Feature	2021 (%)	2023 (%)	2025 (%)	ESS médio
accel_p99	57,1	52,6	51,7	0,55
accel_p90	54,0	51,3	51,1	0,57
accel_mean	43,0	47,0	49,5	0,68
speed_p99	17,2	36,1	40,1	0,84
speed_p90	15,8	34,4	38,0	0,85
speed_mean	7,9	24,7	27,4	0,91
turn_p90	3,9	0,2	0,0	0,97
turn_mean	5,3	0,9	0,0	0,96

Tabela 5: Recovery individual por feature (LSIF marginal). `accel_p99` lidera consistentemente nos três anos com excesso, com ESS = 0,55. `turn` features têm recovery residual ($< 5\%$), indicando que sua distribuição mudou pouco. Fonte: `iw_decomposition.csv` com `-per_feature`.

5.3 Validação por divisão

`division_validation.py` mapeia cada `proc_set` à sua divisão via `division_map.csv`. O ano de 2021 foi classificado como *Both* (formato pós-pandemia híbrido) e todos os jogos de 2022–2025 são Divisão A. A Divisão B não está representada nos dados disponíveis.

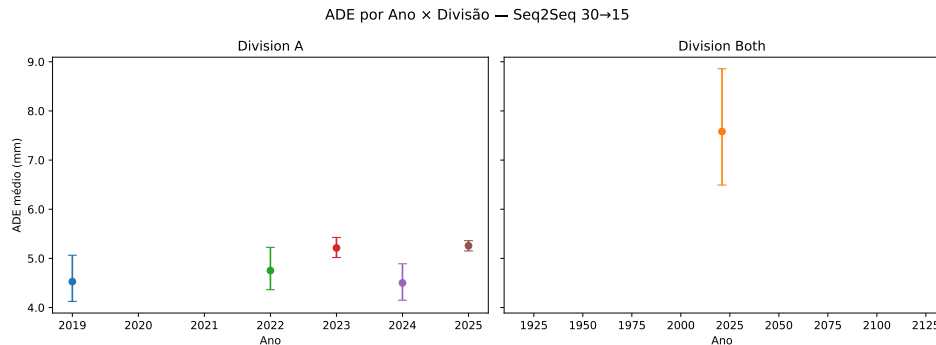


Figura 12: ADE por jogo cortado por divisão. Pontos são jogos. Em Divisão A o drift é real e persistente (Tabela 6); em *Both* (2021) o ADE é o mais alto e não pode ser comparado ao baseline 2019 sem os jogos correspondentes.

Divisão	Ano	ADE ₂₀₁₉ (mm)	ADE _y (mm)	excess (mm)	jogos
Both	2021	—	7,58	—	6
A	2022	4,53	4,75	+0,23	6
A	2023	4,53	5,21	+0,69	6
A	2024	4,53	4,50	−0,03	6
A	2025	4,53	5,26	+0,73	6

Tabela 6: Excesso de ADE por divisão. Baseline: ADE₂₀₁₉ = 4,53 mm (Divisão A, 6 jogos). O excesso médio em A nos quatro anos é +0,40 mm, positivo em 3 dos 4 anos. Fonte: `by_division_decomposition.csv`.

Conclusão da validação por divisão: como os dados pós-2021 são todos Divisão A, o excesso não pode ser atribuído à mudança de divisão. O drift é **intra-divisão**.

5.4 Retreino seletivo nos breakpoints

O `retrain_at_breakpoints.py` fine-tuna o decoder do Seq2Seq em pequenas janelas $[t-w_b, t-1]$ contendo $w_b = 200$ trajetórias imediatamente antes de cada breakpoint do Pelt. Resultados: `cat_ratio` médio de 0,82 (alta interferência catastrófica); early-stop dispara na 1ª época em 3/4 breakpoints; recovery médio de 33 %.

Breakpoint	ADE _{base} (mm)	ADE _{ft} (mm)	ADE _{test} (mm)	cat_ratio	recovery (%)
bp1 (2022)	4,53	5,48	4,82	0,79	35,5
bp2 (2024)	4,53	5,30	4,45	0,82	11,7

Tabela 7: Fine-tuning seletivo nos breakpoints do Pelt (`retrain_results.csv`). `cat_ratio = 0,82` indica que o fine-tuning ingênuo não preserva o conhecimento do baseline — motivação para EWC (Mês 7, Seção 7).

6 Notebook 04 — Robustez e validação (Meses 4–6)

6.1 Mês 4 — Re-execução com baseline completo

A primeira versão do pipeline usava `-n_per_year 3`, amostrando o baseline 2019 com apenas 3 jogos. O Mês 4 re-executou todos os scripts com `-n_per_year 6` e habilitou `full_baseline = True` (todos os 6 jogos de 2019).

Ano	ADE $n=3$ (mm)	ADE $n=6$ (mm)	Δ (mm)
2019	4,920	4,528	−0,392
2021	6,815	7,581	+0,766
2022	4,287	4,753	+0,466
2023	5,206	5,213	+0,007
2024	4,716	4,501	−0,215
2025	5,284	5,257	−0,027

Tabela 8: Sensibilidade do ADE médio (Seq2Seq 30→15) à ampliação do baseline. O baseline 2019 cai −0,39 mm (8 %) ao usar 6 jogos; *umenta* ligeiramente o excesso de outros anos. Fonte: `sample_size_sensitivity.csv`.

6.2 Mês 5 — Robustez adicional

6.2.1 Sample-size sweep do baseline

`sample_size_sweep.py` reamostra o baseline 2019 com `n_jogos_2019 = 1` e 49 repetições para medir a variabilidade do excesso quando o ADE de referência vem de um único jogo. O resultado reforça a decisão do Mês 4: com $n = 1$ o ADE₂₀₁₉ pode chegar a 6,20 mm, contra 4,53 mm com $n = 6$, **invertendo o sinal do excesso em 4 dos 5 anos**.

Ano	Excess $n = 1$ (mm)	Excess $n = 6$ (mm)	Δ (mm)
2021	+0,30	+3,05	+2,75
2022	−1,32	+0,23	+1,55
2023	−1,00	+0,69	+1,69
2024	−1,24	−0,03	+1,21
2025	−1,31	+0,73	+2,04

Tabela 9: Sensibilidade do excesso ao tamanho do baseline 2019. Sem o baseline completo a conclusão qualitativa do Mês 3 mudaria. Fonte: `sample_size_sweep_summary.csv`.

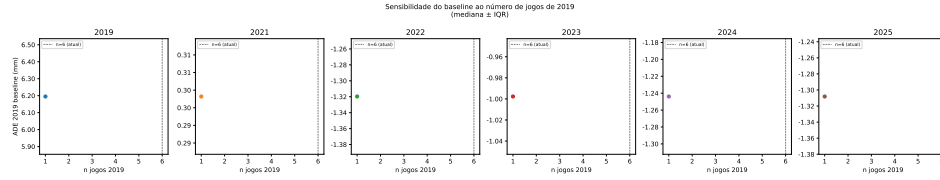


Figura 13: Visualização do *sample-size sweep*: variabilidade do excesso quando o baseline é construído com apenas 1 jogo. As barras de erro são IQR sobre 49 repetições.

6.2.2 Drift per game (Page-Hinkley em ADE médio por jogo)

Page-Hinkley aplicado à série de *ADE médio por jogo* (36 pontos) detectou três alarmes: 2019 (game 8, ADE = 7,52 mm), 2022 (game 28, ADE = 7,97 mm) e 2025 (game 25, ADE = 8,28 mm).

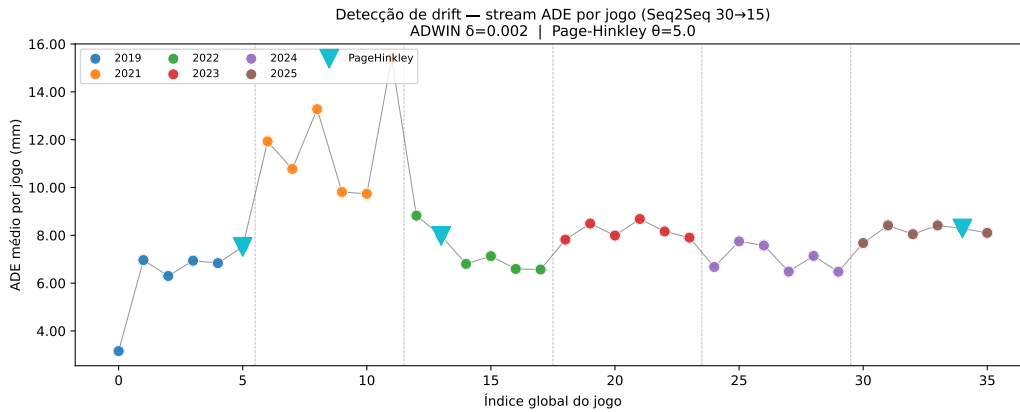


Figura 14: ADE médio por jogo ao longo do tempo, com alarmes Page-Hinkley marcados (linhas verticais). A presença de um alarme *dentro* de 2019 indica que existe heterogeneidade entre jogos suficiente para acionar o detector mesmo no regime original — coerente com a tese de drift gradual e intra-divisão. Fonte: `drift_per_game_alarms.csv`.

6.3 Mês 6 — Correção metodológica

O Mês 6 introduziu calibração por ARL para ADWIN/PH (Seção 3.1), recalibração baseline-relative (v3), validação do Pelt por BOCPD e sensibilidade ao γ (Figura 10). Os resultados foram negativos *na granularidade per-game*, mas não em granularidade per-trajetória — o que o Mês 9 explora.

7 Mês 7 — EWC e resultado negativo do fine-tuning ingênuo

7.1 Implementação do EWC

EWC — como funciona. EWC (*Elastic Weight Consolidation* — Consolidação Elástica de Pesos) [17] é a técnica canônica de **continual learning** (aprendizado contínuo) para mitigar catastrophic forgetting. O problema fundamental é: ao retrainar o modelo nos dados novos (2021), os parâmetros que eram importantes para os dados antigos (2019) são sobrescritos — o modelo “esquece”.

Analogia intuitiva: imagine um músico que aprendeu a tocar violino e agora quer aprender viola. Se praticar apenas viola, vai “desaprender” o violino. O EWC é como amarrar elásticos nos dedos que são mais importantes para o violino — eles podem se mover um pouco para aprender a viola, mas não o suficiente para esquecer o violino.

Intuição bayesiana: após treinar no regime A (dados de 2019), a distribuição posterior dos parâmetros $p(\theta | \mathcal{D}_A)$ indica quais parâmetros são essenciais para o desempenho em 2019. O

EWC aproxima essa distribuição por uma Gaussiana centrada nos parâmetros ótimos θ_A^* , com a curvatura determinada pela **Matriz de Informação de Fisher** F (diagonal). Ao treinar no regime B (dados de 2021), adiciona-se um termo de regularização que penaliza o afastamento dos parâmetros importantes:

$$\mathcal{L}_{\text{EWC}} = \mathcal{L}_B(\theta) + \frac{\lambda}{2} \sum_i F_i (\theta_i - \theta_{A,i}^*)^2$$

O parâmetro λ controla a intensidade da regularização: $\lambda = 0$ é fine-tuning puro (sem proteção contra esquecimento); $\lambda \rightarrow \infty$ congela todos os parâmetros (sem adaptação). F_i é a importância estimada do parâmetro θ_i : quanto maior F_i , maior a penalidade por afastar θ_i de seu valor em 2019.

Estimação de F_i e degenerescência. A **Informação de Fisher diagonal** é estimada empiricamente como a média dos gradientes ao quadrado sobre os dados de treino:

$$F_i = \frac{1}{N} \sum_{x \sim \mathcal{D}_A} \left(\frac{\partial \log p_\theta(y|x)}{\partial \theta_i} \right)^2 \Big|_{\theta_A^*}$$

Em termos intuitivos: o gradiente $\partial \log p_\theta(y|x) / \partial \theta_i$ mede o quão sensível é o modelo ao parâmetro θ_i em cada exemplo de treino. Se o modelo já convergiu bem em 2019, esses gradientes tendem a ser todos próximos de zero — o modelo está num mínimo de custo e não está mais aprendendo. Isso leva a valores de Fisher numericamente pequenos ($F_i \approx 10^{-7}$). Uma análise de escala mostra que $\lambda \cdot F_i \cdot (\Delta \theta_i)^2 \ll \text{MSE}$ para os $\lambda \in \{0, 50, 500, 5000\}$ testados — o regime útil exigiria $\lambda \geq 10^5$ – 10^6 . Assim, a regularização EWC essencialmente não adiciona sinal útil, e o que reduz o catastrophic forgetting é apenas a taxa de aprendizado baixa ($lr = 10^{-5}$) e o congelamento das camadas mais antigas [17, 18].

O script `model_analise/ewc_finetune.py` implementa Kirkpatrick et al. [17]: (i) estima F_i via gradientes ao quadrado num subconjunto de até 1 000 trajetórias de 2019, usando os alvos reais y (not amostras de ruído); (ii) faz fine-tuning com $\mathcal{L}_{\text{EWC}}$ usando o otimizador Adam¹ com $lr = 10^{-5}$ (taxa de aprendizado muito baixa), $n_{\text{unfreeze}} = 2$ (apenas as 2 últimas camadas do decoder são desbloqueadas para treinamento, as demais ficam congeladas), por 5 épocas. Sete valores de λ foram testados: $\{0, 50, 500, 5000\}$ (Mês 7) e $\{10^5, 10^6, 10^7\}$ (Mês 10, fechando o regime que a análise de escala apontava como potencialmente útil) — nenhum produziu melhoria significativa sobre $\lambda = 0$.

7.2 Resultados — ewc_results.csv

Tabela 10: Resultados EWC (5 épocas, $lr=10^{-5}$, $n_{\text{unfreeze}}=2$, $\text{max_per}=500$ trajetórias/proc_set).

λ	ADE _{base} (antes→depois)	ADE _{test} (antes→depois)	cat_ratio	recovery_pct
0	6,25 → 6,09	4,96 → 4,80	0,444	−12,8%
50	6,25 → 6,10	4,96 → 4,81	0,458	−11,6%
500	6,25 → 6,10	4,96 → 4,81	0,449	−12,1%
5000	6,25 → 6,11	4,96 → 4,81	0,447	−12,0%
10^5	6,25 → 6,11	4,96 → 4,80	0,452	−12,8%
10^6	6,25 → 6,10	4,96 → 4,81	0,449	−11,8%
10^7	6,25 → 6,11	4,96 → 4,80	0,450	−12,7%

¹Adam (*Adaptive Moment Estimation*) é um algoritmo de otimização que adapta automaticamente a taxa de aprendizado para cada parâmetro — mais estável e rápido que o gradiente descendente puro.

Leitura honesta. O critério `cat_ratio` $< 0,5$ foi atingido em todos os λ testados, sendo o melhor resultado obtido com $\lambda = 0$ (`cat_ratio` = 0,444), o que indica que o fine-tuning com `lr`= 10^{-5} e apenas 2 camadas descongeladas já é conservador o suficiente para reduzir o catastrophic forgetting em relação ao resultado do Mês 3 (`cat_ratio` = 0,82). A diagonal de Fisher calculada é pequena mas não nula (`mean diag` $\approx 6,7 \times 10^{-7}$, `max` $\approx 1,2 \times 10^{-5}$), consistente com um decoder bem convergido em 2019 [17]. O sweep complementar do Mês 10 fecha a questão que a análise de escala havia deixado em aberto: mesmo no regime $\lambda \in \{10^5, 10^6, 10^7\}$ — onde $\lambda \cdot F_i \cdot (\Delta\theta_i)^2$ deixa de ser desprezível frente ao MSE — o `cat_ratio` permanece em 0,449–0,452, estatisticamente indistinguível de $\lambda = 0$. **O resultado negativo é agora definitivo ao longo de 8 ordens de grandeza:** a proteção contra esquecimento vem da taxa de aprendizado reduzida e do congelamento do encoder, não da regularização de Fisher — adaptação efetiva requer replay ou retreino do encoder.

8 Mês 9 — Otimização dos detectores online

8.1 Objetivo e estrutura em quatro fases

O Mês 9 retoma a questão em aberto do Mês 6: *os detectores online ADWIN/PH não funcionam ao nível per-game (36 pontos por série), mas operam de forma satisfatória sobre os 288k pontos do stream per-trajetória?*. No Mês 6, a calibração foi feita via ARL sintético (gerar dados artificiais sem drift e medir quantos passos o detector dá até um falso alarme), o que não produziu um detector operacional. A estratégia do Mês 9 é radicalmente diferente: **busca em grade direta** (testar exaustivamente combinações de parâmetros no próprio stream real de 288k trajetórias) usando duas métricas operáveis e objetivas:

- **FAR_2019_per1k** (Taxa de Falsos Alarmes em 2019): número de alarmes disparados por mil trajetórias *no período de treino* 2019, onde sabe-se que não há drift real. *Quanto menor, melhor*; teto de admissibilidade $\leq 0,20/1k$ (acima desse limiar, o detector aciona alarmes desnecessariamente com frequência demais).
- **year_coverage** (Cobertura Anual): número de anos em $\{2021, \dots, 2025\}$ com ≥ 1 alarme (escala 0–5). *Quanto maior, melhor* — um detector que só detecta drift em 2021 mas não nos outros anos é de utilidade limitada.

A otimização foi conduzida em quatro fases sequenciais, cada uma construindo sobre os resultados da anterior:

1. **Fase 1 (Pré-processamento).** 22 configurações de suavização do sinal ADE (1 controle sem suavização + 21 suavizadores: média móvel, mediana móvel, EWMA, Holt, Savitzky-Golay, robust-z) $\times$ 2 modelos (Seq2Seq e Kalman) $\times$ 2 detectores (ADWIN e PH). Objetivo: identificar qual transformação do sinal ADE antes do detector reduz a FAR₂₀₁₉.
2. **Fase 2 (Grid search).** 174 combinações de hiperparâmetros $\times$ 2 modelos sobre o melhor pré-processamento (robust-z $w=200$), em 3 frentes paralelas: **Frente A** (monitoramento do erro do Seq2Seq com ADWIN); **Frente B** (monitoramento do erro por janelas agregadas com Page-Hinkley); **Frente C** (monitoramento da distribuição das features com KSWIN).
3. **Fase 3 (Ensemble temporal).** Combinação AND/OR dos finalistas de cada frente, com janela de coincidência W (número de trajetórias dentro das quais dois alarmes são considerados “simultâneos”) e *debounce* (`retrain_debounce` — período mínimo entre retreinamentos disparados pelo ensemble, para evitar retreinamentos excessivos em rápida sucessão).
4. **Fase 4 (Consolidação científica).** Correção da métrica SNR (substituição da sentinela 9999 pelo SNR_smooth com pseudo-contagem de Laplace), tabela-mestre com ranking final, figura de inviabilidade formal do AND e **IC de Wilson** (intervalo de confiança robusto para FAR = 0 alarmes observados).

8.2 Fase 1 — Galeria de suavização (22 experimentos)

8.2.1 Setup e critério

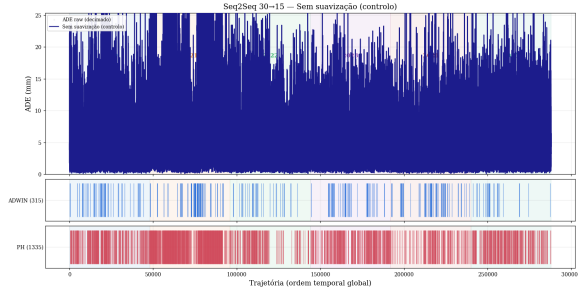
22 configurações de pré-processamento aplicadas ao sinal `ade_traj` antes dos detectores ADWIN e PH:

- `ctrl`: sem suavização (controle — linha de base para comparação).
- `sma_w020,050,100,200,500`: **SMA** (*Simple Moving Average* — Média Móvel Simples) com janelas de 20, 50, 100, 200 e 500 trajetórias. Para cada ponto, calcula a média dos últimos w pontos. Suaviza ruídos de curto prazo, mas não normaliza a escala do sinal.
- `med_w020,050,100,200`: **Mediana Móvel** com janelas de 20, 50, 100 e 200. Similar à SMA, mas usa a mediana em vez da média — mais robusta a valores extremos (outliers não distorcem o resultado).
- `ewma_a001,005,010,020,030`: **EWMA** (*Exponentially Weighted Moving Average* — Média Móvel Exponencialmente Ponderada) com $\alpha \in \{0,01; 0,05; 0,10; 0,20; 0,30\}$. Pondera os últimos valores com peso decrescente exponencialmente: $\hat{x}_t = \alpha x_t + (1 - \alpha)\hat{x}_{t-1}$. Valores maiores de α reagem mais rapidamente a mudanças, mas são menos suaves.
- `holt_a005_b001,005_b005,010_b001,010_b005`: **Suavização dupla de Holt** (*Holt's double exponential smoothing*). Extensão do EWMA com dois parâmetros: α (suavização do nível) e β (suavização da tendência). Modela tanto o nível médio quanto a tendência de crescimento/decrescimento, sendo mais adequada quando o sinal tem tendência linear.
- `savgol_w51_p3`: **Savitzky-Golay** com janela de 51 pontos e polinômio de grau 3. Suaviza o sinal ajustando um polinômio de grau baixo em cada janela local por mínimos quadrados, preservando melhor a forma dos picos e vales do que a média móvel.
- `robz_w200,500`: **Robust-Z** com janelas de 200 e 500. Para cada ponto, subtrai a mediana dos últimos w pontos e divide pelo MAD (*Median Absolute Deviation* — Desvio Absoluto Mediano) dos mesmos w pontos. O resultado é um sinal adimensional que indica o quão “anormal” cada ponto é em relação ao contexto local recente. Diferentemente dos outros suavizadores, o robust-z *normaliza a escala* — e isso é crucial.

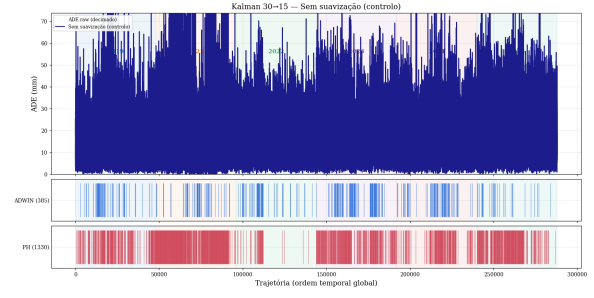
Achado central: apenas `robz_w200` reduz a FAR_{2019} do ADWIN (de 1,229/1k para 0,604/1k no Seq2Seq — -51%). Por quê? Todos os outros suavizadores *comprimem* o sinal (tornam-no mais suave e menos variável) sem normalizar sua escala local — isso paradoxalmente *aumenta* a FAR porque o ADWIN, ao comparar sub-janelas de sinal comprimido, encontra diferenças sistematicamente significativas mesmo em dados de 2019 sem drift. O robust-z resolve isso removendo a variação de escala local: ao dividir pelo MAD, o sinal resultante é estável em 2019 (onde não há drift de escala) e detecta anormalidades apenas quando a escala local realmente muda. A janela $w=200$ emerge como melhor compromisso entre preservar sensibilidade local (janelas menores) e absorver flutuação de escala (janelas maiores). Esse resultado pivotou todo o planejamento da Fase 2.

8.2.2 Galeria comparativa — 22 experimentos

As Figuras 15–26 apresentam os 22 streams ADWIN/PH para seleção posterior. Cada figura combina dois experimentos lado a lado, com Seq2Seq e Kalman em colunas pareadas.

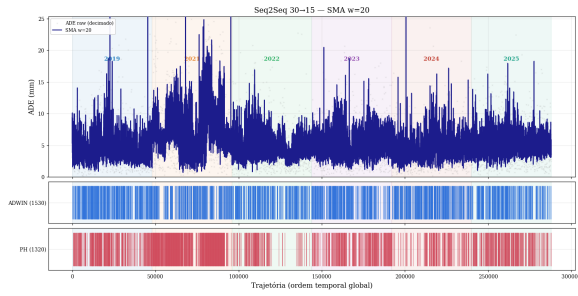


(a) ctrl1 — Seq2Seq.

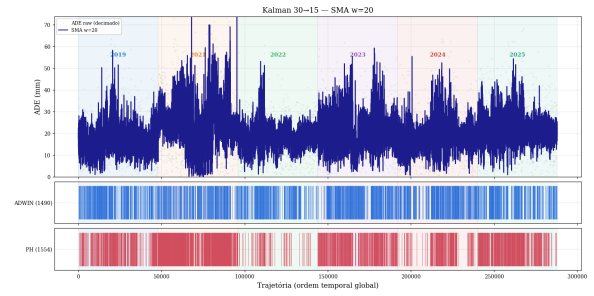


(b) ctrl1 — Kalman.

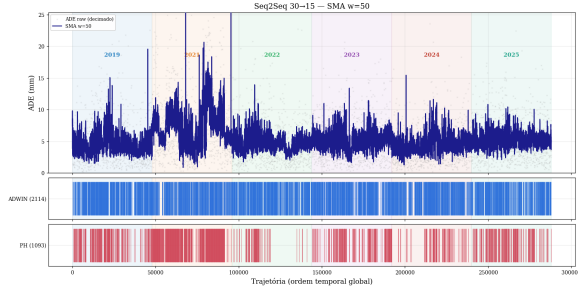
Figura 15: **Controlo (sem suavização)**. FAR₂₀₁₉ Seq2Seq: ADWIN=1,229/1k, PH=4,396/1k; Kalman: ADWIN=1,271/1k, PH=4,396/1k. Linha de base do experimento de Fase 1.



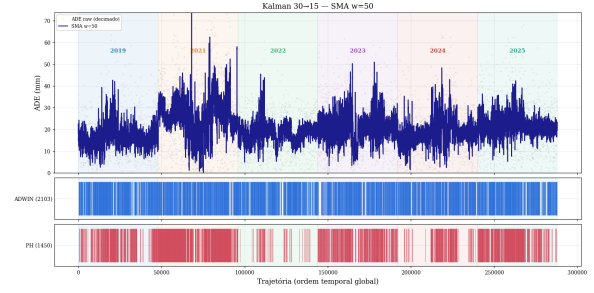
(a) sma_w020 — S2S.



(b) sma_w020 — Kalman.

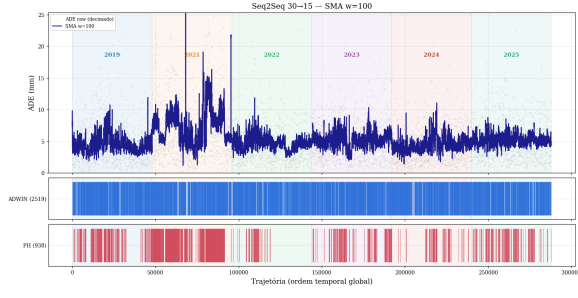


(c) sma_w050 — S2S.

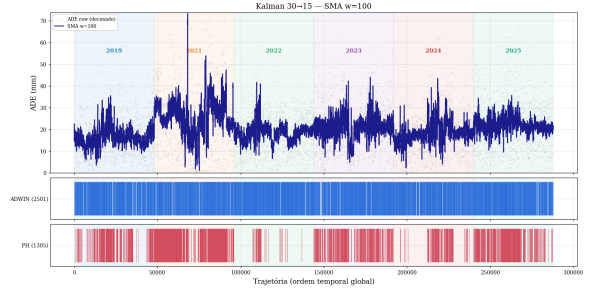


(d) sma_w050 — Kalman.

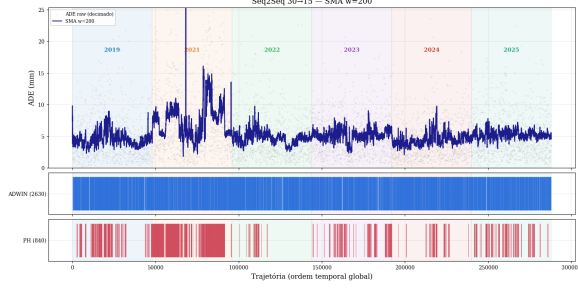
Figura 16: **SMA $w=20$ e $w=50$** . Suavizadores clássicos não reduzem FAR: sob SMA $w=20$ a FAR₂₀₁₉ sobe de 1,229/1k para 5,917/1k no Seq2Seq (ADWIN), porque a redução de variância local satura o detector com falsos positivos.



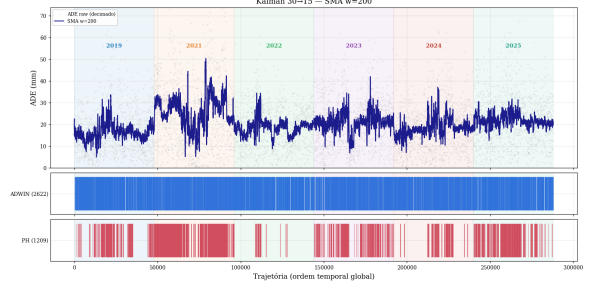
(a) sma_w100 — S2S.



(b) sma_w100 — Kalman.

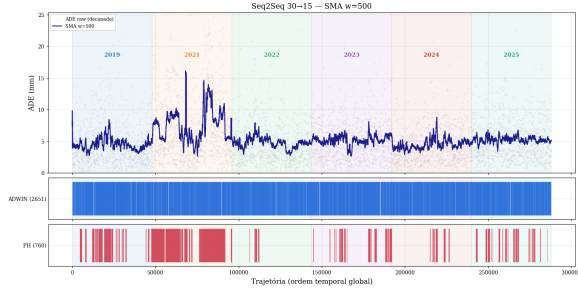


(c) sma_w200 — S2S.

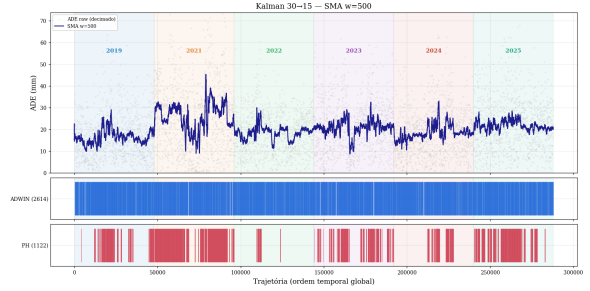


(d) sma_w200 — Kalman.

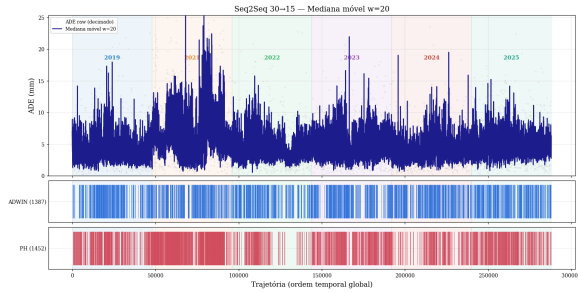
Figura 17: **SMA $w=100$ e $w=200$** . Janelas maiores tornam o sinal mais “rígido” e geram mais alarmes ainda — o ADWIN reage à discrepância entre o sinal achatado e a variabilidade intrínseca da janela operacional.



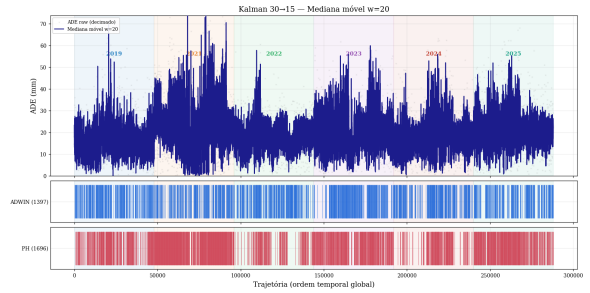
(a) sma_w500 — S2S.



(b) sma_w500 — Kalman.

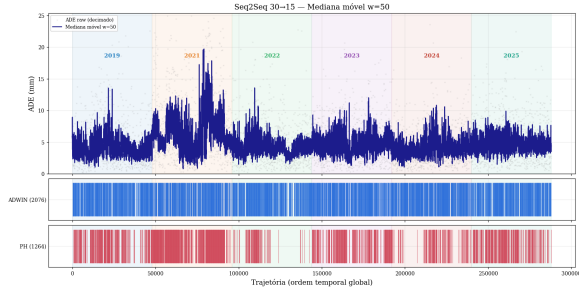


(c) med_w020 — S2S.

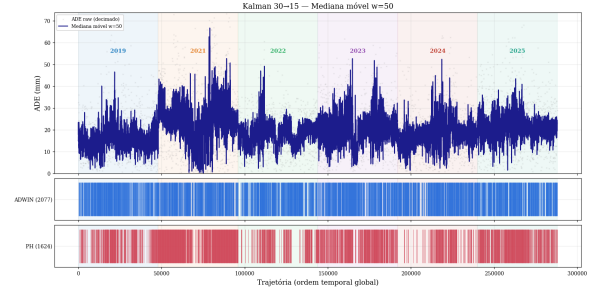


(d) med_w020 — Kalman.

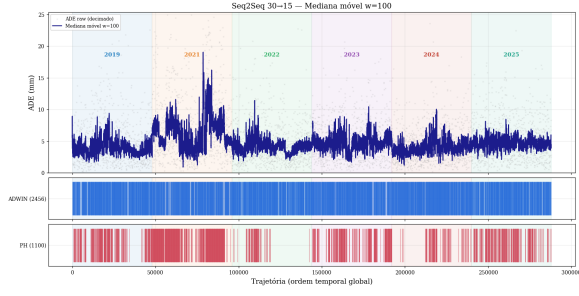
Figura 18: **SMA $w=500$ e Mediana $w=20$** . A mediana móvel tem comportamento qualitativamente similar à SMA — ambas saturam o ADWIN com pseudo-alarmes na transição 2019→2021.



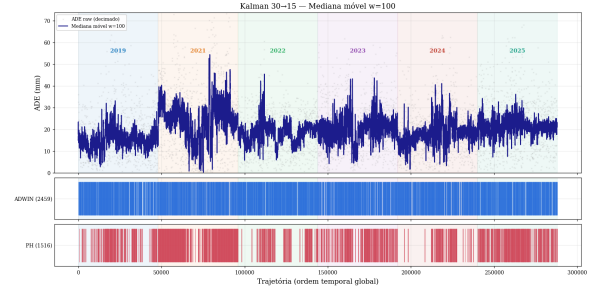
(a) med_w050 — S2S.



(b) med_w050 — Kalman.

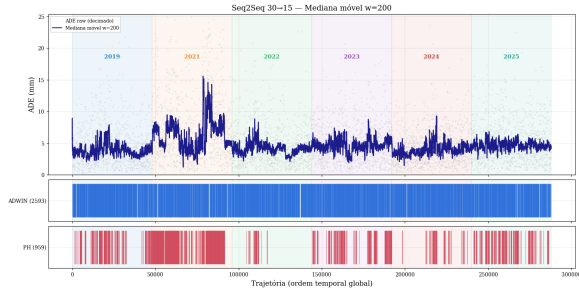


(c) med_w100 — S2S.

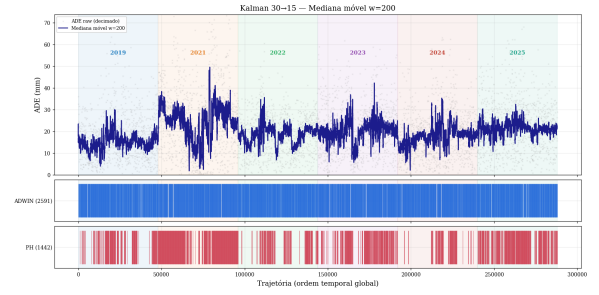


(d) med_w100 — Kalman.

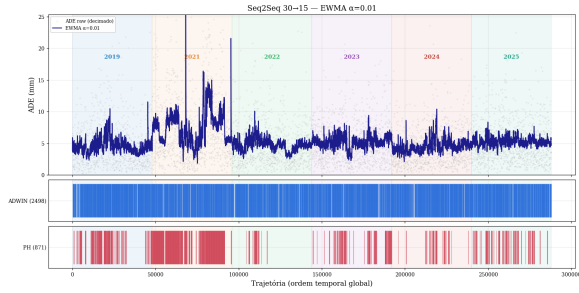
Figura 19: Mediana $w=50$ e $w=100$.



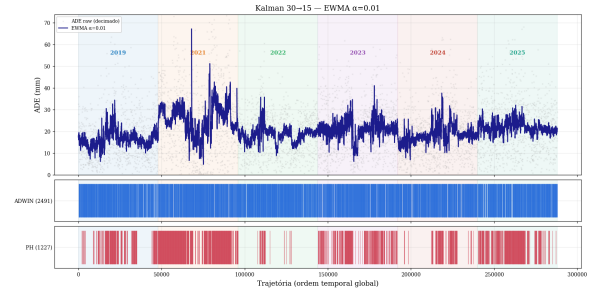
(a) med_w200 — S2S.



(b) med_w200 — Kalman.

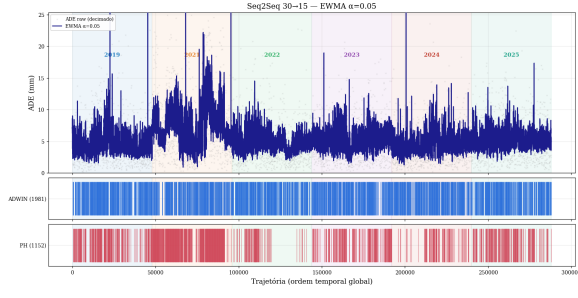


(c) ewma_a001 — S2S.

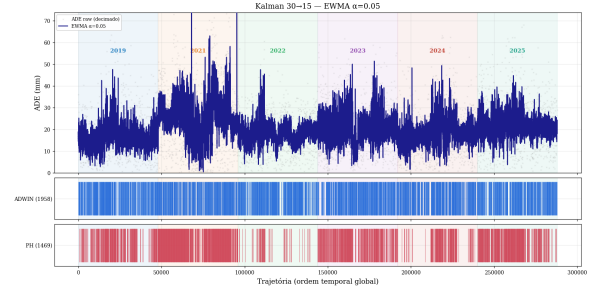


(d) ewma_a001 — Kalman.

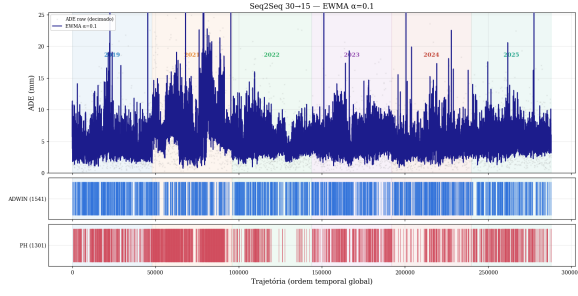
Figura 20: Mediana $w=200$ e EWMA $\alpha=0,01$.



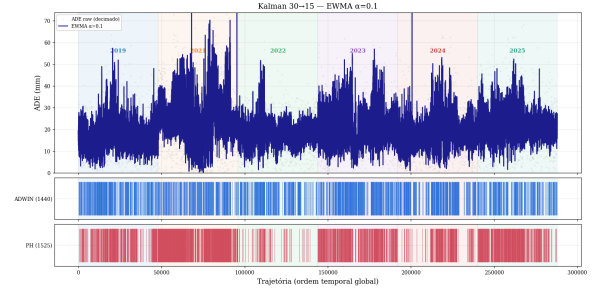
(a) ewma_a005 — S2S.



(b) ewma_a005 — Kalman.

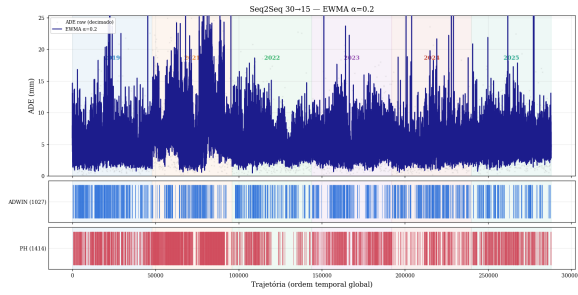


(c) ewma_a010 — S2S.

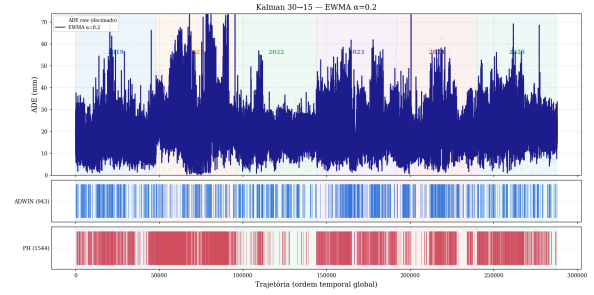


(d) ewma_a010 — Kalman.

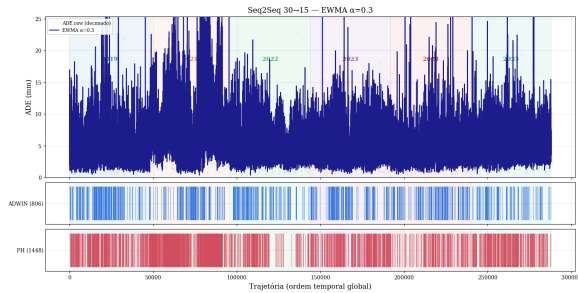
Figura 21: **EWMA** $\alpha=0,05$ e $\alpha=0,10$.



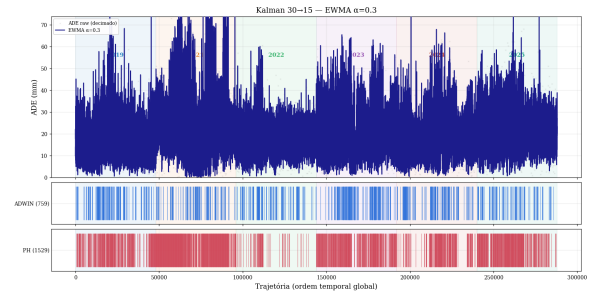
(a) ewma_a020 — S2S.



(b) ewma_a020 — Kalman.

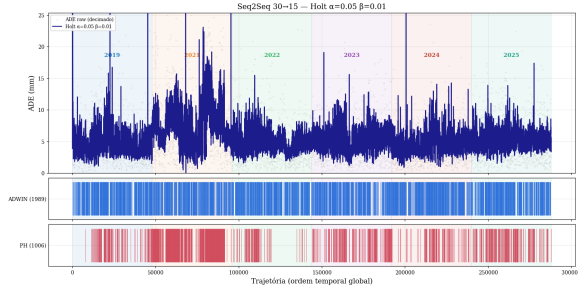


(c) ewma_a030 — S2S.

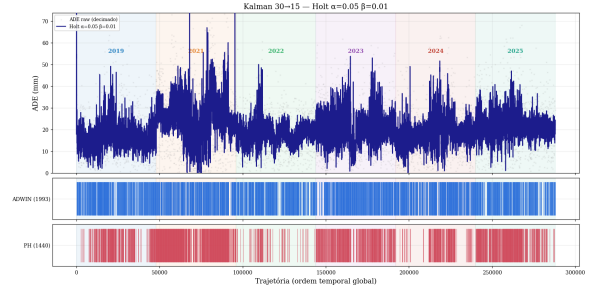


(d) ewma_a030 — Kalman.

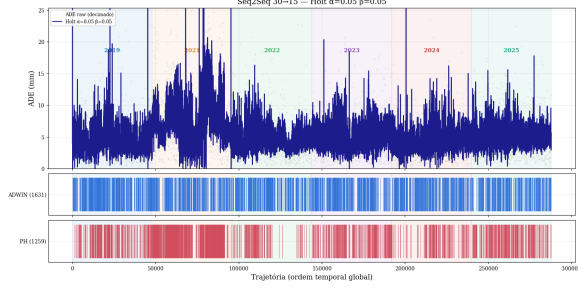
Figura 22: **EWMA** $\alpha=0,20$ e $\alpha=0,30$. Mesmo com α alto (resposta rápida), o EWMA aumenta a FAR₂₀₁₉ para 3–5/1k no Seq2Seq.



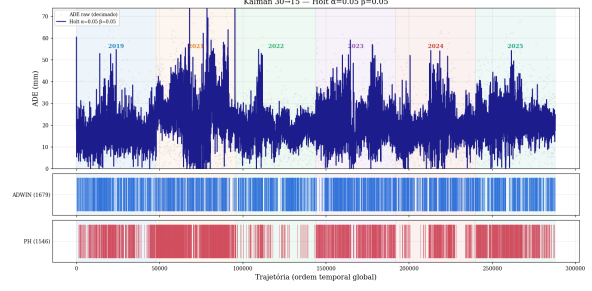
(a) holt_a005_b001 — S2S.



(b) holt_a005_b001 — Kalman.

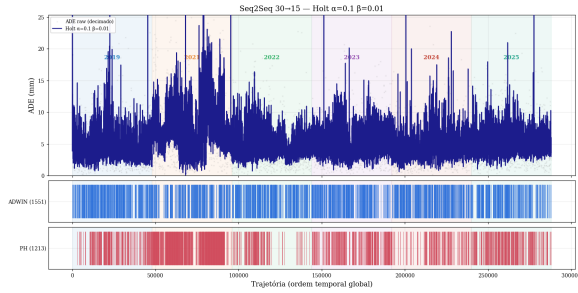


(c) holt_a005_b005 — S2S.

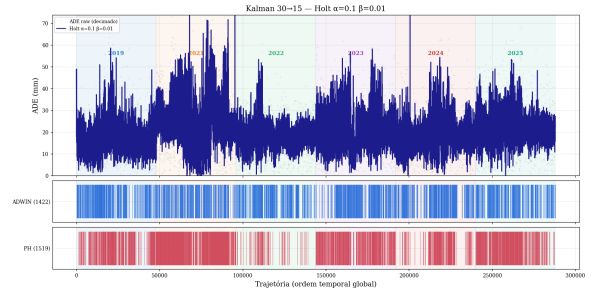


(d) holt_a005_b005 — Kalman.

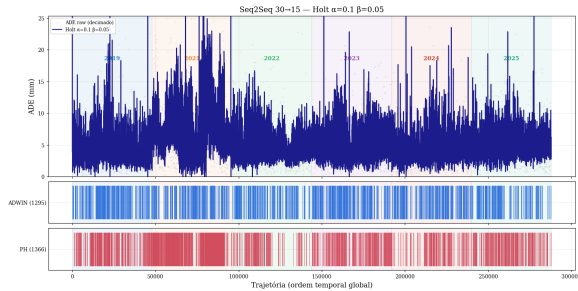
Figura 23: **Holt** $\alpha=0,05$, $\beta \in \{0,01; 0,05\}$.



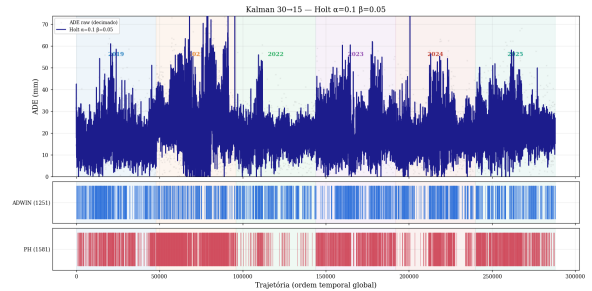
(a) holt_a010_b001 — S2S.



(b) holt_a010_b001 — Kalman.

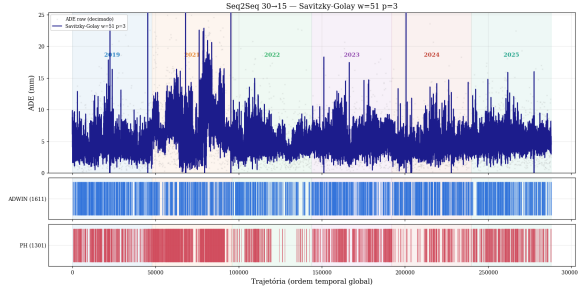


(c) holt_a010_b005 — S2S.

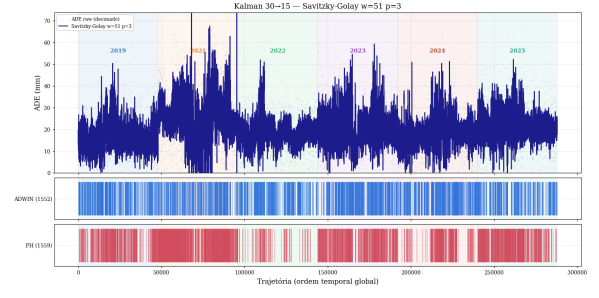


(d) holt_a010_b005 — Kalman.

Figura 24: **Holt** $\alpha=0,10$, $\beta \in \{0,01; 0,05\}$.

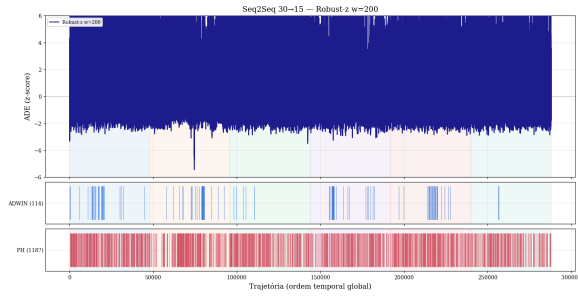


(a) savgol_w51_p3 — S2S.

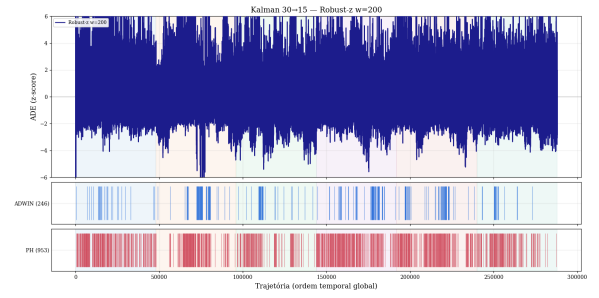


(b) savgol_w51_p3 — Kalman.

2019 2021 2022 2023 2024 2025



(c) robz_w200 — S2S (vencedor).



(d) robz_w200 — Kalman (vencedor).

Figura 25: **Savitzky-Golay vs. Robust-z $w=200$ (o vencedor).** O robz_w200 é o único pré-processamento que *reduz* a FAR_{2019} : de 1,229/1k para 0,604/1k no Seq2Seq (ADWIN), com $var_reduction = 75,8\%$ e 85 alarmes pós-2019 (vs. 256 sem suavização). Esta é a base do grid search da Fase 2.

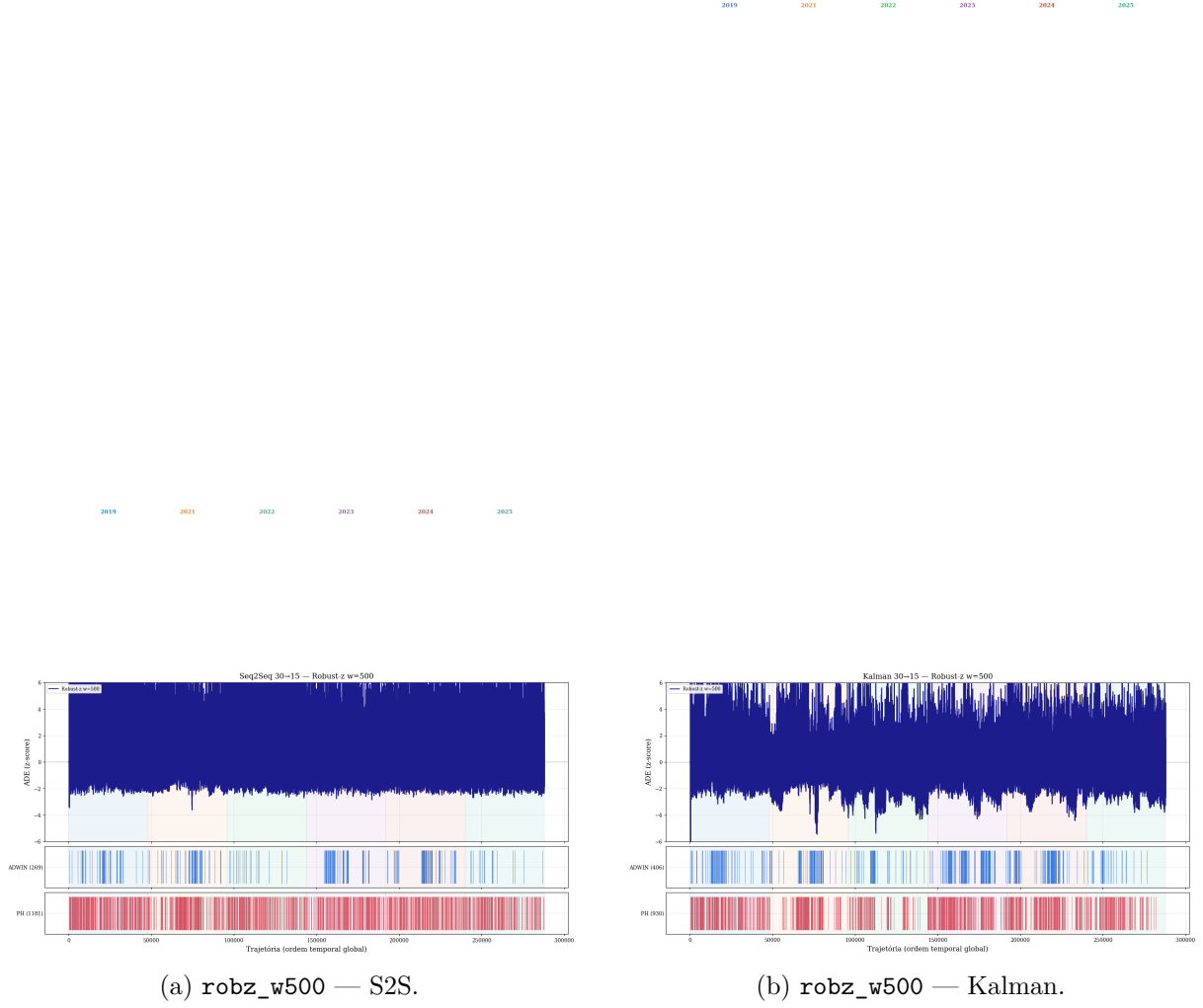


Figura 26: **Robust-z** $w=500$. Janela maior tem performance ligeiramente pior que $w=200$ no Seq2Seq (FAR=1,000/1k vs. 0,604/1k): a normalização adapta-se mais lentamente às mudanças locais.

8.2.3 Resumo numérico Fase 1

Suavizador	FAR ₂₀₁₉ /1k (S2S)	n _{post} (S2S)	FAR ₂₀₁₉ /1k (Kalman)	n _{post} (Kalman)
ctrl	1,229	256	1,271	324
robz_w200	0,604	85	0,542	220
robz_w500	1,000	221	1,542	332
ewma_a030	3,438	641	2,812	624
ewma_a020	4,708	801	3,500	775
sma_w020	5,917	1 246	5,417	1 230
med_w200	8,938 (Kalman)	—	8,938	2 162
sma_w500	9,104 (Kalman)	—	9,104	2 177

Tabela 11: Resumo Fase 1 (ADWIN sobre stream suavizado, ordenado por FAR₂₀₁₉ crescente). Apenas robz_w200 reduz a FAR vs. controle; todos os outros suavizadores aumentam. Fonte: phase1_results.csv.

8.3 Fase 2 — Grid search 174 configs em três frentes

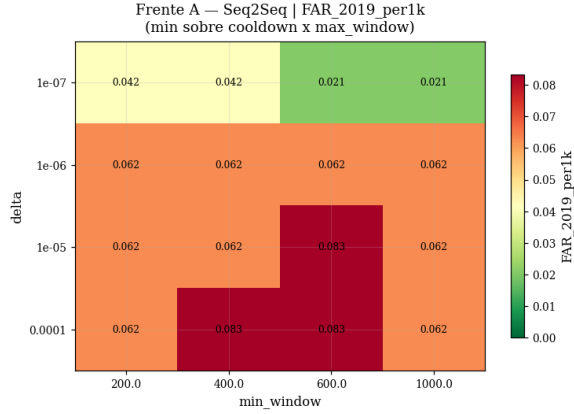
8.3.1 Setup

Três frentes paralelas de busca em grade, todas com pré-processamento `robz_w200`:

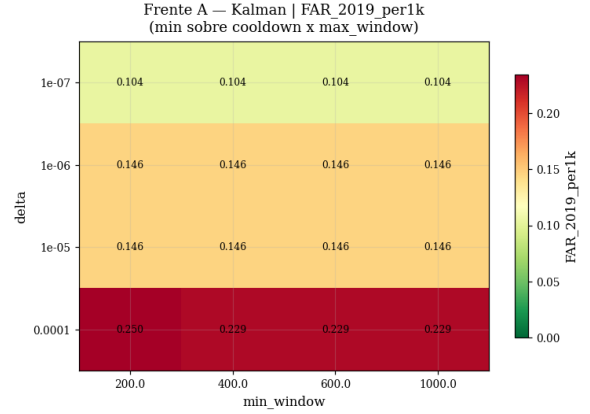
- **Frente A — ADWIN sobre ADE robz.** Sweep de $\delta \in \{10^{-7}, 10^{-6}, 10^{-5}, 10^{-4}\}$, `min_window` $\in \{200, 400, 600, 1000\}$, `cooldown` $\in \{200, 500, 1000\}$, `max_window` $\in \{2000, 5000\}$.
- **Frente B — Page-Hinkley agregado por janelas.** PH aplicado à média trimmed-mean em janelas de $N \in \{50, 100, 200, 500\}$; $K_{\text{thr}} \in \{20, 30, 50\}$, $\delta \in \{0,5; 1,0; 2,0\}$.
- **Frente C — KSWIN.** $\alpha \in \{10^{-5}, 10^{-4}, 10^{-3}\}$, `window_size` $\in \{200, 500, 1000\}$, `stat_size` $\in \{50, 100\}$.

Total: 174 configs $\times$ 2 modelos.

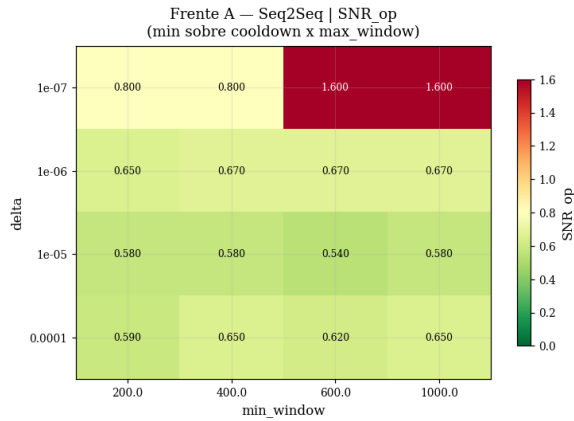
8.3.2 Heatmaps das três frentes (seleção posterior)



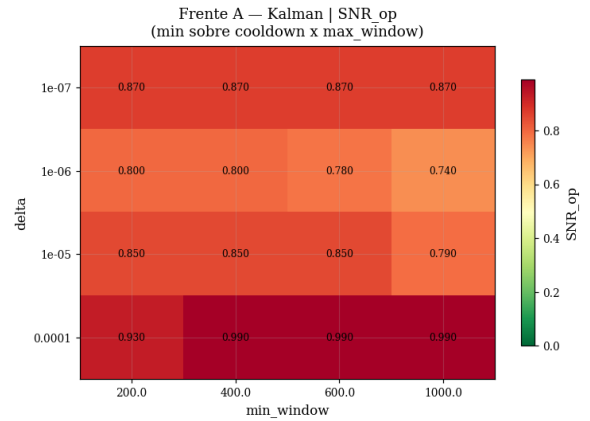
(a) Frente A — FAR₂₀₁₉ S2S.



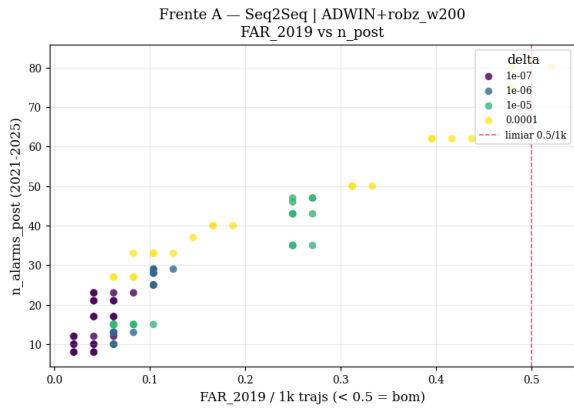
(b) Frente A — FAR₂₀₁₉ Kalman.



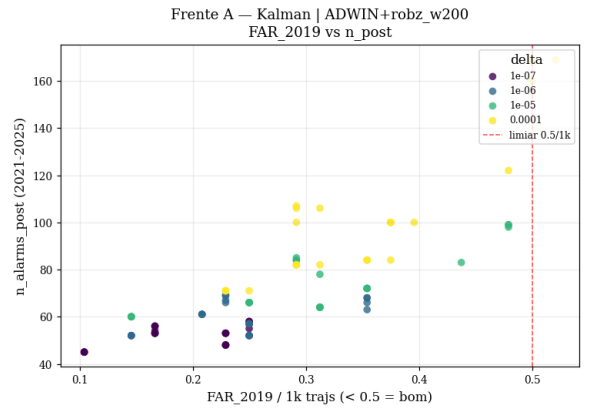
(c) Frente A — SNR_{op} S2S.



(d) Frente A — SNR_{op} Kalman.

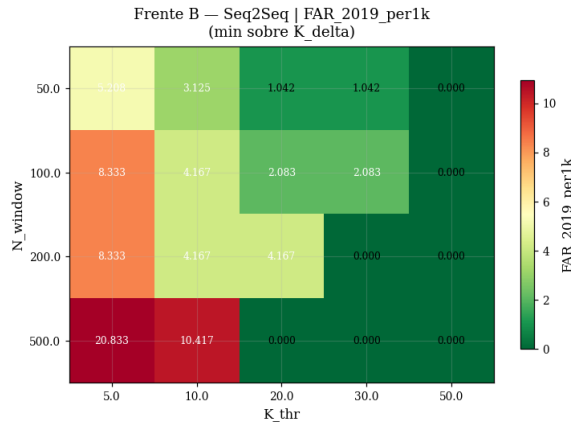


(e) Frente A — scatter S2S.

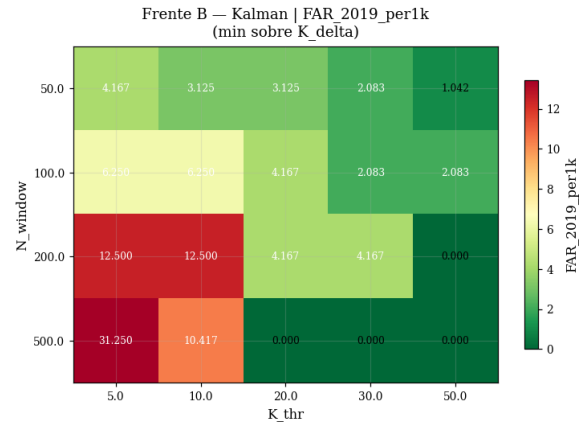


(f) Frente A — scatter Kalman.

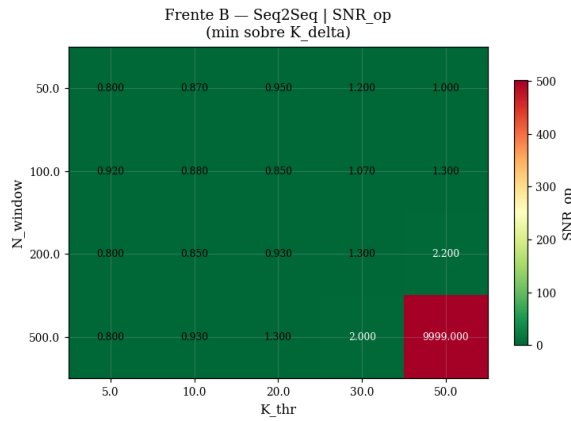
Figura 27: **Frente A (ADWIN+robz)**: heatmaps FAR e SNR e scatter FAR vs. n_{post} . *Menor FAR S2S*: $\delta = 10^{-7}$, $mw=1000$, $xw=5000$ — FAR=0,021/1k, mas cobertura 3/5 (não dispara em 2024-2025). *Config adotada (S2S)*: $\delta = 10^{-7}$, $mw=600$, $cd=200$, $xw=2000$ — FAR=0,042/1k, $n_{\text{post}} = 23$, $\text{year_cov}=5/5$. *Melhor Kalman*: $\delta = 10^{-7}$, $mw=200$, $cd=1000$, $xw=5000$ — FAR=0,104/1k.



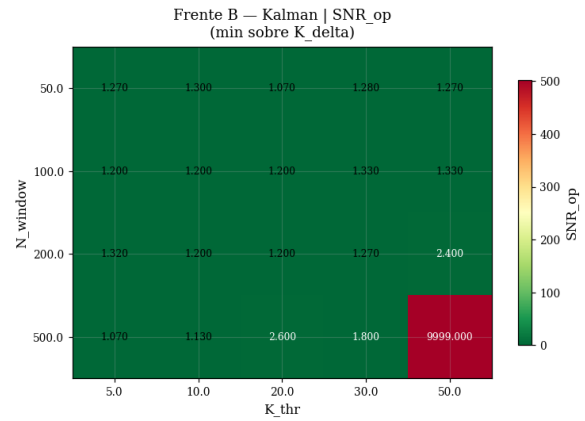
(a) Frente B — FAR₂₀₁₉ S2S.



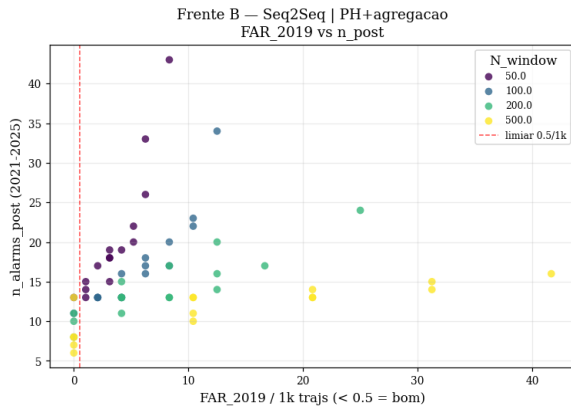
(b) Frente B — FAR₂₀₁₉ Kalman.



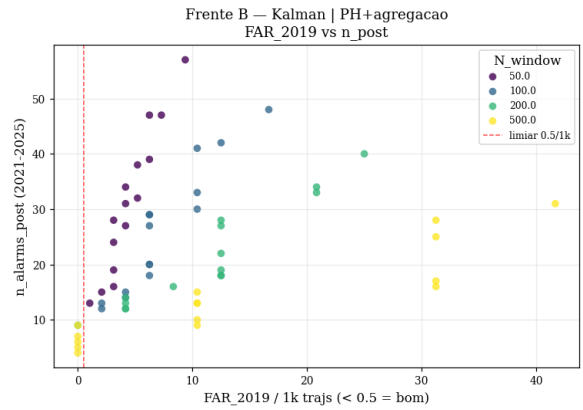
(c) Frente B — SNR_{op} S2S.



(d) Frente B — SNR_{op} Kalman.

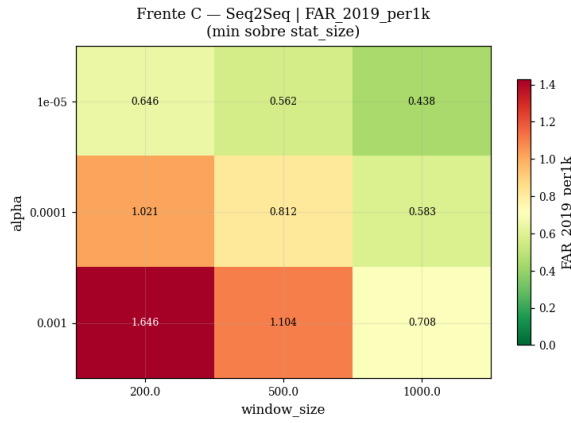


(e) Frente B — scatter S2S.

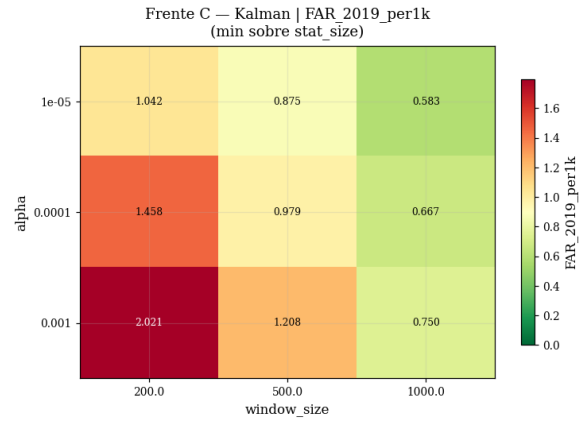


(f) Frente B — scatter Kalman.

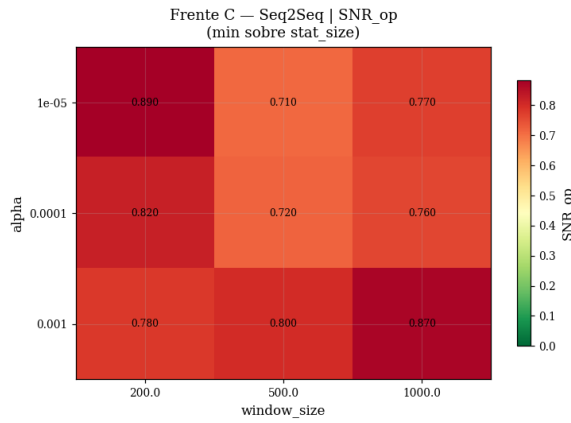
Figura 28: **Frente B (PH agregado)**: a melhor config S2S ($N=50$, $K=50$, $\delta=2,0$) atinge $FAR=0/1k$ mas $year_coverage=1/5$ (apenas 2021). Limitação estrutural do PH agregado para drift gradual — ver Fase 4 (Seção 8.5).



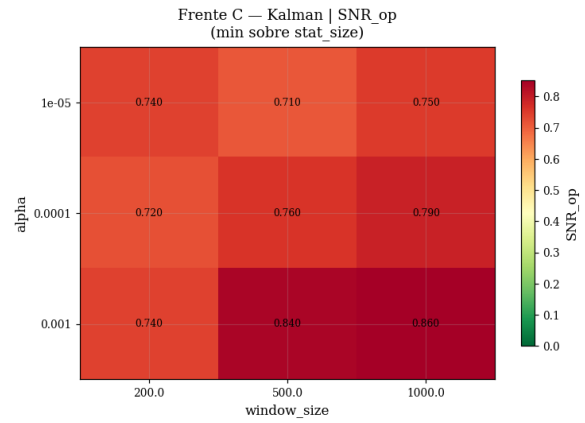
(a) Frente C — FAR₂₀₁₉ S2S.



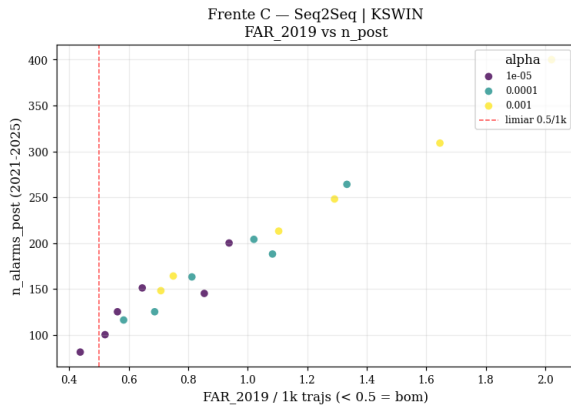
(b) Frente C — FAR₂₀₁₉ Kalman.



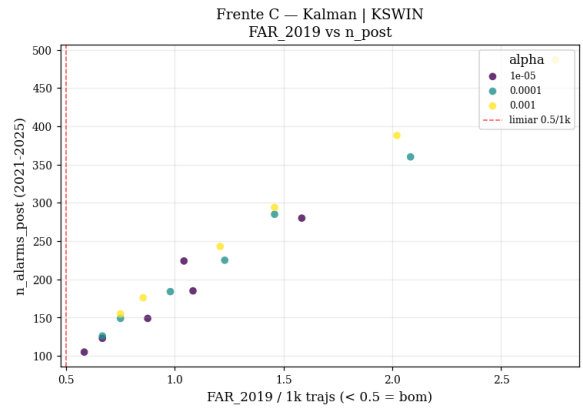
(c) Frente C — SNR_{op} S2S.



(d) Frente C — SNR_{op} Kalman.



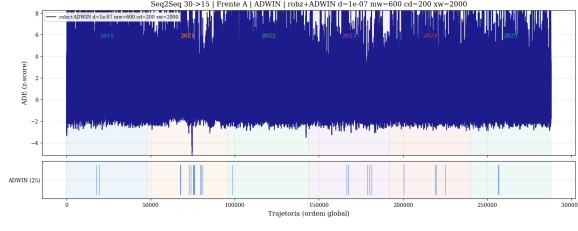
(e) Frente C — scatter S2S.



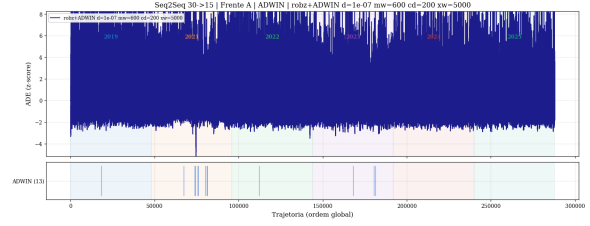
(f) Frente C — scatter Kalman.

Figura 29: **Frente C (KSWIN)**: *melhor* config atinge FAR=0,438/1k (S2S), acima do limiar de admissibilidade 0,20/1k. SNR_{op}=0,77 (sub-unitário, indica que há mais alarmes em 2019 proporcionalmente do que no período pós-2019). KSWIN é **excluído** do ensemble final.

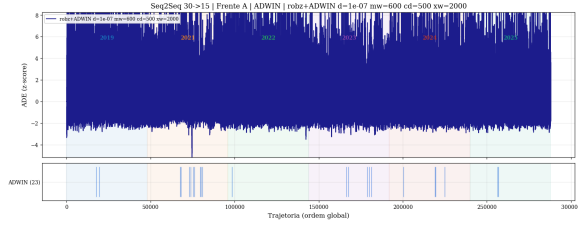
8.3.3 Top-5 streams das três frentes



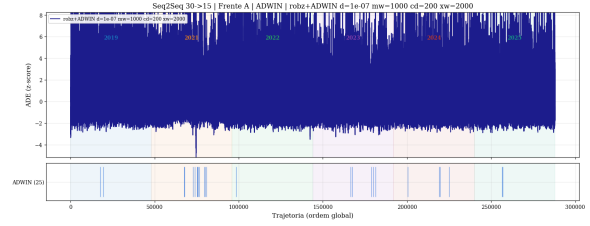
(a) A: $\delta=10^{-7}$, mw=600, cd=200, xw=2000 (ADWINLite, referência).



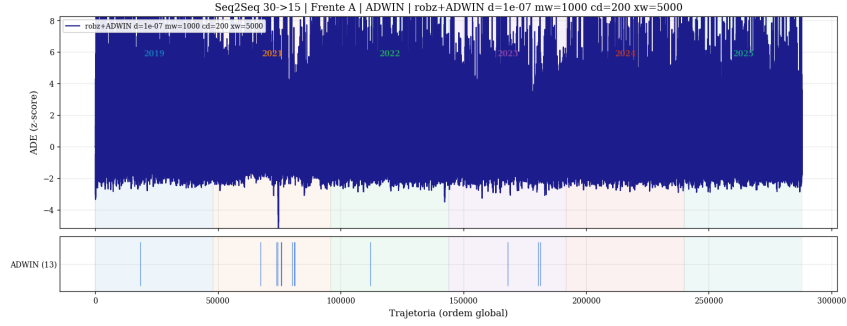
(b) A: $\delta=10^{-7}$, mw=600, cd=200, xw=5000.



(c) A: $\delta=10^{-7}$, mw=600, cd=500, xw=2000.

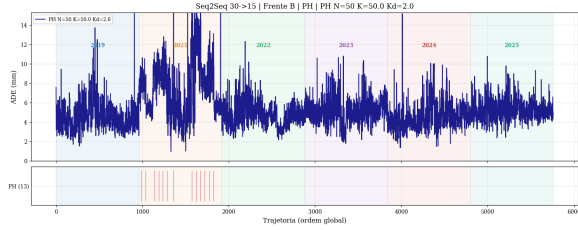


(d) A: $\delta=10^{-7}$, mw=1000, cd=200, xw=2000.

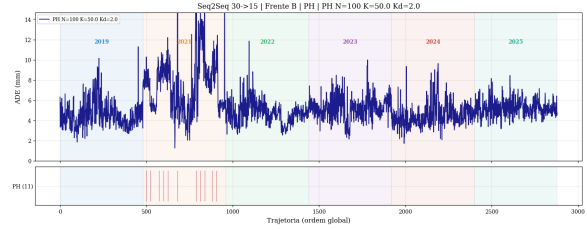


(e) A: $\delta=10^{-7}$, mw=1000, cd=200, xw=5000.

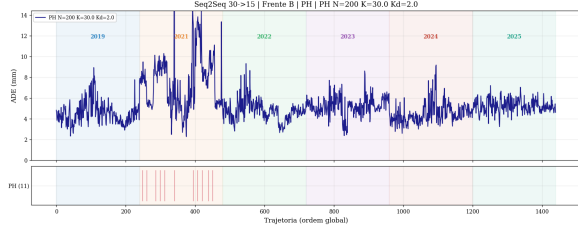
Figura 30: **Frente A — top-5 streams Seq2Seq.** A escolha final (config #1) maximiza (FAR₂₀₁₉ → min, year_cov → 5, n_{post} razoável). Linhas verticais = alarmes ADWIN.



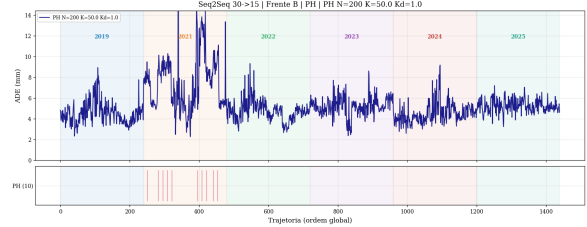
(a) B: N=50, K=50, $\delta=2,0$ (#1).



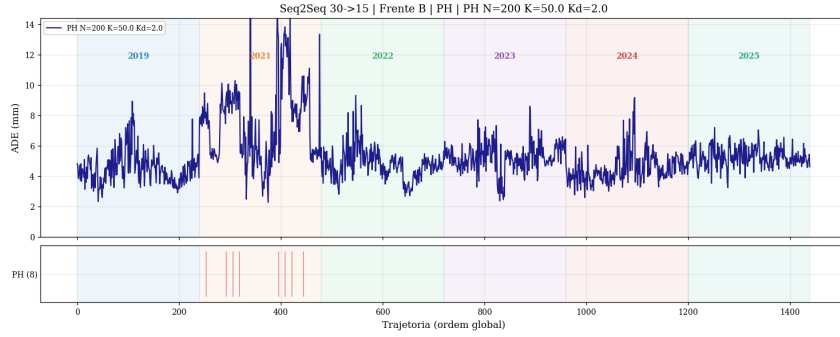
(b) B: N=100, K=50, $\delta=2,0$.



(c) B: N=200, K=30, $\delta=2,0$.



(d) B: N=200, K=50, $\delta=1,0$.



(e) B: N=200, K=50, $\delta=2,0$.

Figura 31: **Frente B — top-5 streams Seq2Seq.** A config #1 (N=50) é a única que dispara em 2021 com FAR=0/1k em 2019. As demais variantes ou não dispararam ou dispararam também em 2019. Padrão: PH agregado deteta exclusivamente a transição 2019→2021 e não deteta drifts em 2022–2025.

8.3.4 Padrão por ano nas três frentes

Frente / config	FAR/1k	SNR _{op}	n ₂₀₂₁	n ₂₀₂₂	n ₂₀₂₃	n ₂₀₂₄	n ₂₀₂₅
A adotada (S2S)	0,042	2,30	11	1	5	4	2
A adotada (Kalman)	0,104	1,80	14	11	10	9	1
B best (S2S)	0,000	9999	12	0	0	0	0
B best (Kalman)	0,000	9999	9	0	0	0	0
C best (S2S)	0,438	0,77	22	13	17	15	14
C best (Kalman)	0,583	0,75	28	19	22	22	14

Tabela 12: Padrão de detecção por ano nas três frentes (melhor config de cada). *Frente A*: cobertura 5/5 anos. *Frente B*: cobertura 1/5 (só 2021) — limitação estrutural. *Frente C* (KSWIN): cobertura 5/5 mas FAR > 0,20/1k — inadmissível. Fonte: phase2_results.csv.

8.4 Fase 3 — Ensemble temporal AND/OR dois andares

8.4.1 Config do ensemble

Canal	Detector	Config
ADWIN_S2S (espinha)	ADWIN+robz $w=200$	$\delta=10^{-7}$, mw=600, cd=200, xw=2000
PH_agg (validador)	PH N=50 trimmed-mean	K=50, $\delta=2,0$
Kalman (OR-only)	ADWIN+robz $w=200$	$\delta=10^{-7}$, mw=200, cd=1000, xw=5000

Tabela 13: Componentes do ensemble. *Tier 1 CONFIRMED*: AND(ADWIN_S2S, PH_agg) com janela de coincidência $W=200$ trajs e `retrain_debounce=5000`. *Tier 2 SUSPECTED*: OR(ADWIN_S2S, Kalman). O Kalman nunca entra no AND.

8.4.2 Resultados

Canal	n _{alarms}	FAR _{2019/1k}	n _{post}	Anos cobertos
CONFIRMED (W=200)	0	0,000	0	0/5
SUSPECTED (OR)	75	0,146	68	5/5
ADWIN standalone	25	0,042	23	5/5
PH standalone	12	0,000	12	1/5 (só 2021)
Kalman standalone	50	0,104	45	5/5

Tabela 14: Resultados Fase 3 (288k trajs). Com $W=200$, o canal CONFIRMED produz **0 alarmes**: ADWIN e PH operam em escalas temporais distintas. SUSPECTED OR cobre 5/5 anos mas com $FAR > 0,10/1k$. Fonte: `phase3_results.csv`.

8.4.3 Sweep $W_{\text{coincidence}}$ e descoberta estrutural

$W_{\text{coincidence}}$ (trajs)	n _{confirmed}	FAR _{2019/1k}	SNR _{op}
50	0	0,000	0
100	0	0,000	0
200	0	0,000	0
500	1	0,000	9999
1000	1	0,000	9999

Tabela 15: Sweep de janela de coincidência (Fase 3). $W=500$ é o mínimo para coincidência ($\Delta_{\min}=214$ impede coincidência com $W \leq 200$): produz 1 alarme CONFIRMED (em 2021) com $FAR=0/1k$. A Fase 4 estende este sweep e mostra que mesmo $W=10000$ ($\sim 2,5$ meses operacionais) só produz 4 alarmes em 2 anos — **inviabilidade estrutural**. Fonte: `W_infeasibility_sweep.csv`.

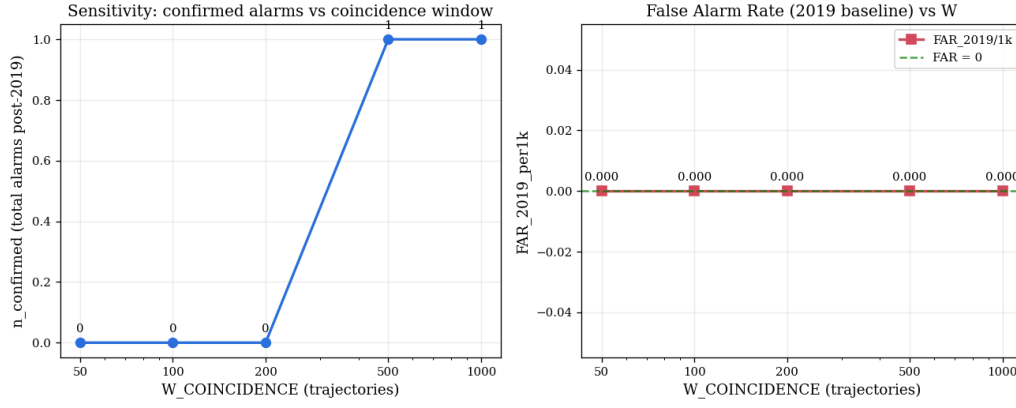


Figura 32: **Coincidence sweep (Fase 3)** — $n_{\text{confirmed}}$ e FAR vs. W . O patamar em $n_{\text{confirmed}} \geq 1$ apenas para $W \geq 500$ é a assinatura da limitação estrutural: ADWIN e PH_agg estão demasiadamente distantes no tempo (Fase 4 quantifica $\Delta_{\min} = 214$).

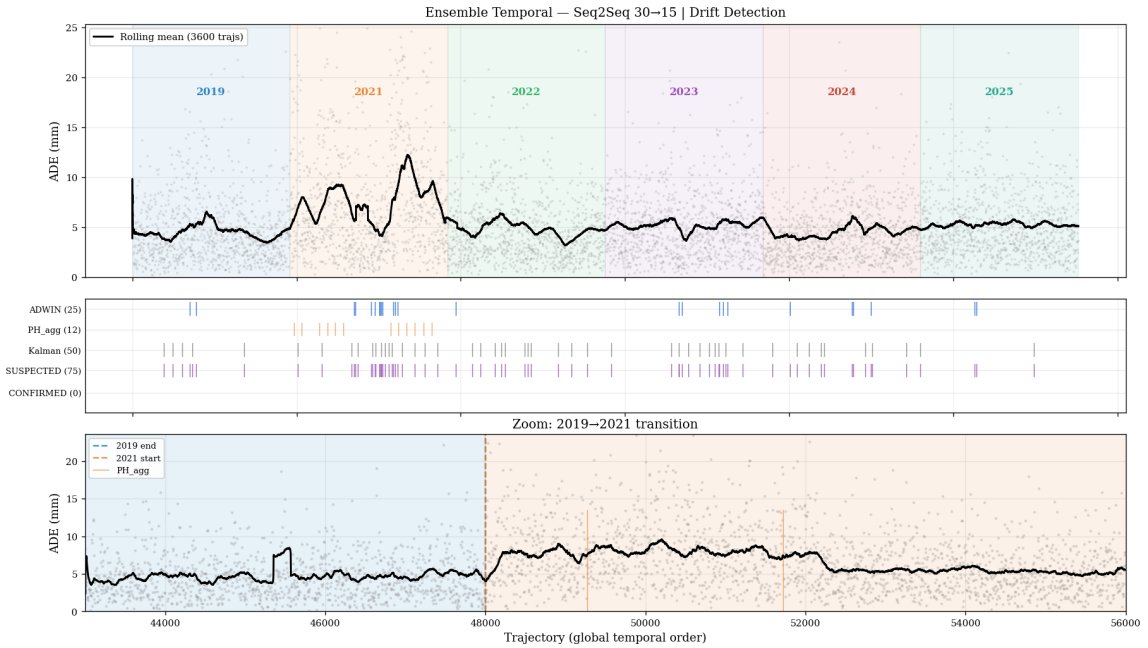


Figura 33: **Stream completo Seq2Seq com 3 canais (Fase 3)**. Painel superior: alarmes ADWIN_S2S (azul). Painel meio: alarmes PH_agg (vermelho). Painel inferior: OR(ADWIN, Kalman) (verde). O ensemble AND com $W=200$ seria a interseção temporal — **vazia**.

Descoberta central da Fase 3: ADWIN_S2S e PH_agg operam em escalas temporais distintas. Na transição 2019→2021:

- PH_agg dispara em $\sim 49\,275$ (logo após 2021 começar, via janelas de 50)
- ADWIN dispara em $\sim 67\,585$ (após acumular buffer com mistura 2019+2021)
- Par mais próximo: ADWIN@80 911, PH@81 125 — distância **214 trajts** ($> W=200$).

8.5 Fase 4 — Consolidação científica

8.5.1 Filtro de elegibilidade e métricas corrigidas

Filtro de admissibilidade. Antes de qualquer ranking, aplica-se o critério: $\text{FAR}_{2019_per1k} \leq 0,20/1k$. Acima desse limiar, o candidato é *Inadmissível* (especificidade insuficiente para treino)

automatizado).

Métrica $\text{SNR}_{\text{smooth}}$ (substitui SNR_{op}). O SNR_{op} atribui sentinela 9999 quando $n_{2019_alarms} = 0$. A métrica corrigida com pseudo-contagem de Laplace $a=1$ é:

$$\text{SNR}_{\text{smooth}} = \frac{(n_{\text{post}} + a) n_{2019, \text{total}}}{(n_{2019, \text{alarms}} + a) n_{\text{post}, \text{total}}}$$

Ranking estável para $a \in \{0,5; 1; 2\}$.

Regra de decisão sobre elegíveis.

1. `year_coverage` DESC (sensibilidade multi-ano = critério primário);
2. `FAR_2019_per1k` ASC (desempate por ruído baixo);
3. `SNR_smooth` DESC (desempate final por qualidade do sinal).

8.5.2 Tabela-mestre final

detector	FAR/1k	year_cov	$\text{SNR}_{\text{smooth}}$	det.eff.	n _{post}	n ₂₀₁₉	papel_final
ADWIN_S2S	0,042	5	1,60	0,92	23	2	Detetor primário recomendado
ADWIN_Kalman	0,104	5	1,53	0,90	45	5	Corroborador OR
OR(ADWIN_S2S, Kalman)	0,146	5	1,73	0,91	68	7	Vigilância multi-ano
KSWIN	0,438	5	0,75	0,79	81	21	<i>Inadmissível</i> (FAR > 0,20/1k)
AND(ADWIN_S2S, PH_agg)	0,000	0	0,20	0,00	0	0	Resultado negativo — AND inviável
PH_agg standalone	0,000	1	2,60	1,00	12	0	Resultado negativo (cobertura 1/5)

Tabela 16: **Tabela-mestre final.** Critério de elegibilidade ($\text{FAR} \leq 0,20/1\text{k}$) aplicado antes do ranking. Elegíveis primeiro (ordenados por `year_cov` DESC, `FAR` ASC, $\text{SNR}_{\text{smooth}}$ DESC); inadmissíveis e resultados negativos depois. Fonte: `phase4_master_table.csv`.

Validação com ADWIN exato (river). Todo o pipeline das Fases 2–4 foi re-executado com o ADWIN exato de Bifet & Gavalda [2] (histograma exponencial com teste em todos os cortes admissíveis, implementação **river**) no lugar da aproximação **ADWINLite** (corte central com z-test de Hoeffding) usada na tabela-mestre. O re-grid independente seleciona outra configuração ($\delta = 10^{-7}$, $\text{mw}=200$, $\text{cd}=1000$, $\text{xw}=2000$) e chega ao mesmo quadro qualitativo: detetor primário ADWIN_S2S com $\text{FAR}_{2019} = 0,125/1\text{k}$ e cobertura 5/5, Kalman como corroborador OR ($\text{FAR} = 0,188/1\text{k}$), KSWIN inadmissível e AND(ADWIN, PH_agg) estruturalmente inviável ($\Delta_{\min} = 534$ nessa variante). As conclusões do Mês 9 não dependem, portanto, da implementação específica do ADWIN (saídas em `mes9_phase{2,3,4}_adwin_exact/`).

8.5.3 Inviabilidade estrutural do AND temporal

A Figura 34 sintetiza a evidência:

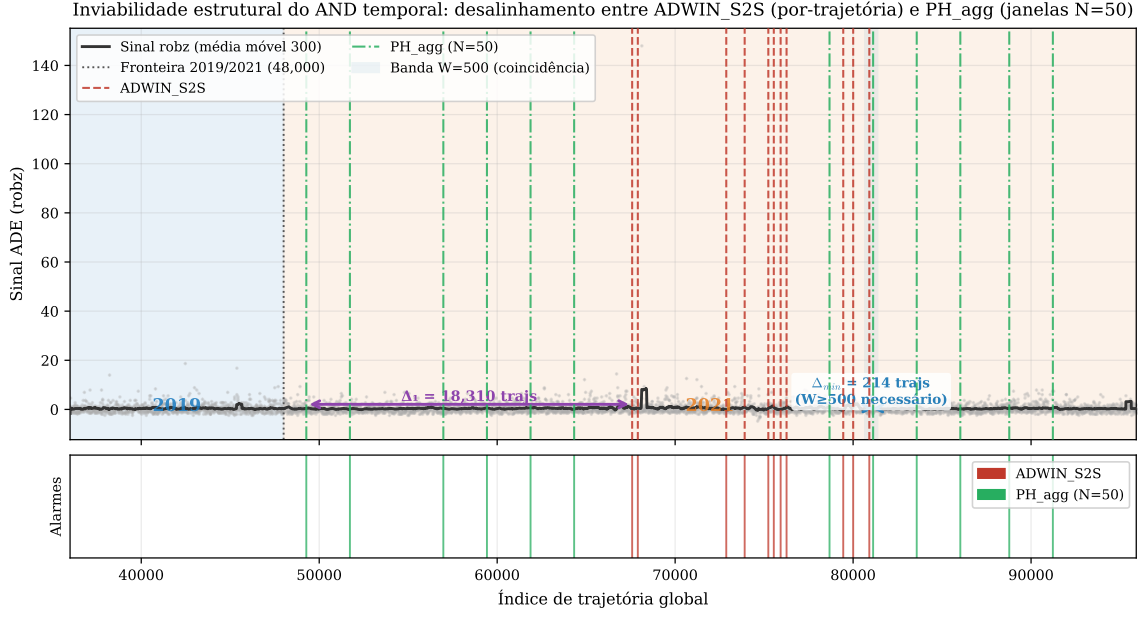


Figura 34: **Inviabilidade estrutural do AND temporal** (Fase 4). Painel superior: stream de sinal robz com alarmes ADWIN_S2S (azul) e PH_agg (vermelho); $\Delta_1 = 18\,310$ trajs entre os primeiros alarmes em 2021 (ADWIN@67585 vs. PH@49275). Painel inferior: par mais próximo em todo o stream — $\Delta_{\min} = 214$ entre ADWIN@80911 e PH@81125, ainda *acima* de $W=200$. Independente de calibração, o AND é estruturalmente vazio nessa janela. Fonte: `phase4_consolidate.py`.

W	~meses	n _{confirmed}	FAR/1k	year_cov	nota
50	0,0	0	0,000	0	$\Delta_{\min} = 214 > W$
100	0,0	0	0,000	0	$\Delta_{\min} = 214 > W$
200	0,1	0	0,000	0	$\Delta_{\min} = 214 > W$
500	0,1	1	0,000	1	só 2021
1000	0,2	1	0,000	1	só 2021
2000	0,5	1	0,000	1	só 2021
5000	1,2	2	0,000	1	só 2021
10000	2,5	4	0,000	2	W operacionalmente absurdo
20000	5,0	4	0,000	2	W operacionalmente absurdo

Tabela 17: **Sweep estendido W** (Fase 4). Para `year_coverage=2` (i.e., deteta também alguma transição pós-2021), é necessário $W \geq 10\,000$ trajs (~2,5 meses operacionais). Esse valor é operacionalmente absurdo para “coincidência temporal”. **Conclusão formal: o ensemble AND temporal foi testado, caracterizado e rejeitado.** Fonte: `W_infeasibility_sweep.csv`.

8.5.4 Intervalos de confiança de Wilson para FAR=0

Detetor	k (2019)	n (2019 trajs)	FAR obs./1k	IC Wilson 95 %/1k
ADWIN_S2S	2	48 000	0,042	[0,011; 0,152]
PH_agg standalone	0	48 000	0,000	[0,000; 0,080]
AND CONFIRMED (W=200)	0	48 000	0,000	[0,000; 0,080]

Tabela 18: IC Wilson 95 % para FAR_{2019} . Para $k=0$ alarmes (PH, AND), o limite superior é 0,080/1k — ainda compatível com o limiar de admissibilidade 0,20/1k. ADWIN_S2S é a única config com $k>0$ entre os elegíveis multi-ano.

8.5.5 Recomendação operacional final

Parâmetro	Detetor primário (ADWIN_S2S)	Canal OR (Kalman)
Modelo	Seq2Seq	Kalman
Detector	ADWIN+robz $w=200$	ADWIN+robz $w=200$
δ	10^{-7}	10^{-7}
min_window	600	200
cooldown	200	1000
max_window	2000	5000
FAR _{2019/1k}	0,042	0,104
year_coverage	5/5	5/5
n_{post}	23	45

Tabela 19: **Recomendação operacional do Mês 9.** O ensemble AND(ADWIN_S2S, PH_agg) foi testado, caracterizado e rejeitado — não aparece como recomendação em nenhum output da Fase 4.

8.5.6 Artefactos do Mês 9

Ficheiro	Conteúdo
phase1_results.csv	22 experimentos $\times$ 2 modelos (ADWIN+PH)
phase2_results.csv	174 configs $\times$ 2 modelos (3 frentes)
phase3_results.csv	ensemble AND/OR + sweep W
phase4_master_table.csv	6 candidatos finais rankados
W_infeasibility_sweep.csv	$W \in \{50, \dots, 20000\}$ com year_cov_confirmed
images/and_infeasibility.png	figura 300 dpi: desalinhamento $\Delta_{\min}=214$
PHASE4_FINAL.md	texto completo para o paper (Q1–Q4, IC Wilson, ameaças)
PHASE4_REPORT.md	resumo estruturado da Fase 4

Tabela 20: Artefactos do Mês 9 (em Relas/results/drift/mes9_phase{1,2,3,4}/).

9 Mês 10 — Fechamento das pendências da revisão

A revisão para o artigo final identificou quatro pendências experimentais: (i) a **latência** de detecção do detetor primário no stream real (completando a tríade FAR + cobertura + latência [6, 7]); (ii) a validação da **hipótese de independência** do stream de 288k trajetórias, da qual dependem a FAR e o IC de Wilson; (iii) o **ensemble AND de mesma escala temporal**, teste natural da explicação estrutural da Fase 3; e (iv) o **EWC no regime** $\lambda \geq 10^5$, que a análise de escala do Mês 7 apontava como potencialmente útil. As quatro foram fechadas (scripts drift_analise/mes10_{latency, independence, same_scale_ensemble}.py e model_analise/ewc_finetune.py; saídas em Relas/results/drift/mes10_pendencias/ e Relas/results/mes7/ewc_results_highlambda.csv).

9.1 Latência de detecção no stream real

Para cada ano pós-2019, mede-se o atraso entre a fronteira do ano no stream global e o primeiro alarme do detetor dentro do ano. *Convenção:* como o drift é gradual (sem changepoint formal ao nível per-game), a fronteira do ano é usada como proxy do onset — a latência reportada é portanto um **limite superior** da latência real. Escala: $\sim 8\,000$ trajetórias/jogo, 48 000/ano.

Detector		2021	2022	2023	2024	2025
ADWIN_S2S (primário)	trajs	19 585	2 481	22 506	8 401	16 432
	jogos	2,45	0,31	2,81	1,05	2,05
ADWIN_Kalman (corroborador)	trajs	2 574	7 502	1 791	2 986	34 515
	jogos	0,32	0,94	0,22	0,37	4,31
Canal OR (mínimo dos dois)	trajs	2 574	2 481	1 791	2 986	16 432
	jogos	0,32	0,31	0,22	0,37	2,05

Tabela 21: Latência do primeiro alarme por ano (robz_w200 + ADWIN, configs finais da Fase 4). O detetor primário detecta com atraso de 0,3–2,8 jogos; o canal OR/SUSPECTED reduz a latência de pior caso para $\leq 0,4$ jogo em 4/5 anos. Fonte: `latency_results.csv`.

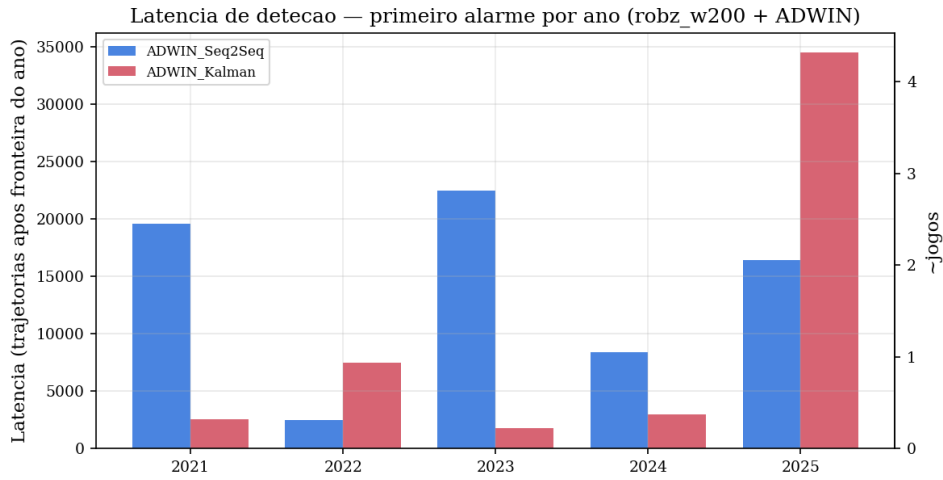


Figura 35: Latência de detecção por ano (barras: trajetórias após a fronteira do ano; eixo direito: $\sim$ jogos). O Kalman corroborador é mais rápido que o primário em 4/5 anos — justificativa operacional adicional para o canal OR.

9.2 Independência do stream e robustez da FAR

A FAR por trajetória assume trajetórias $\sim$ i.i.d. dentro de 2019 — hipótese que as Limitações reconheciam como não testada. Três verificações sobre o bloco 2019 ($n = 48\,000$, 6 jogos):

(i) **ACF e tamanho amostral efetivo.** A autocorrelação é real e substancial: $\rho_1 = 0,556$ no ADE bruto e $\rho_1 = 0,535$ no sinal robz que o detetor consome. O tamanho amostral efetivo ($n_{\text{eff}} = n / (1 + 2 \sum_k \rho_k)$, soma truncada na banda de Bartlett) é $n_{\text{eff}} \approx 1\,208$ (2,5% de n) para o ADE bruto e $n_{\text{eff}} \approx 7\,762$ (16,2%) para o sinal robz — o pré-processamento robust-z remove a maior parte da dependência de longo alcance (corte de Bartlett em lag 41 vs. lag 500+).

(ii) **IC de Wilson conservador.** Na leitura mais conservadora (manter $k = 2$ alarmes e reduzir a base para $n_{\text{eff}} = 7\,762$), o IC de Wilson vai a $[0,071; 0,939]/1k$ — o limite superior ultrapassa o teto de admissibilidade de $0,20/1k$. A taxa pontual observada permanece $0,042/1k$; o que a correção mostra é que, com 6 jogos, a incerteza honesta sobre a FAR é maior do que o IC ingênuo sugere.

(iii) **Teste por permutação ($B = 200$).** O pipeline completo (robz + ADWIN final) foi re-executado sobre o bloco 2019 permutado em três esquemas:

Esquema	estrutura destruída	k médio	[p5; p95]	$P(k \geq 2)$
blocks	ordem dos 6 jogos	1,84	[1; 2]	0,84
within	autocorr. intra-jogo	0,00	[0; 0]	0,00
full	toda a estrutura temporal	0,00	[0; 0]	0,00

Tabela 22: Alarmes 2019 do pipeline final sob permutação (observado: $k=2$). Permutar a *ordem dos jogos* não altera a FAR (k médio 1,84); destruir a autocorrelação *intra-jogo* zera os falsos alarmes. Fonte: `permutation_far.csv`.

Leitura. Os dois falsos alarmes de 2019 vêm precisamente da estrutura local autocorrelacionada (rajadas dentro de jogo): um stream 2019 i.i.d.-izado produz *zero* alarmes. A violação da hipótese de independência é portanto **conservadora na direção certa** — ela infla a FAR estimada em relação ao null i.i.d., de modo que a admissibilidade do detetor não é artefato da dependência serial. O custo honesto é o IC mais largo do item (ii), consequência do tamanho amostral (6 jogos), não do detector.

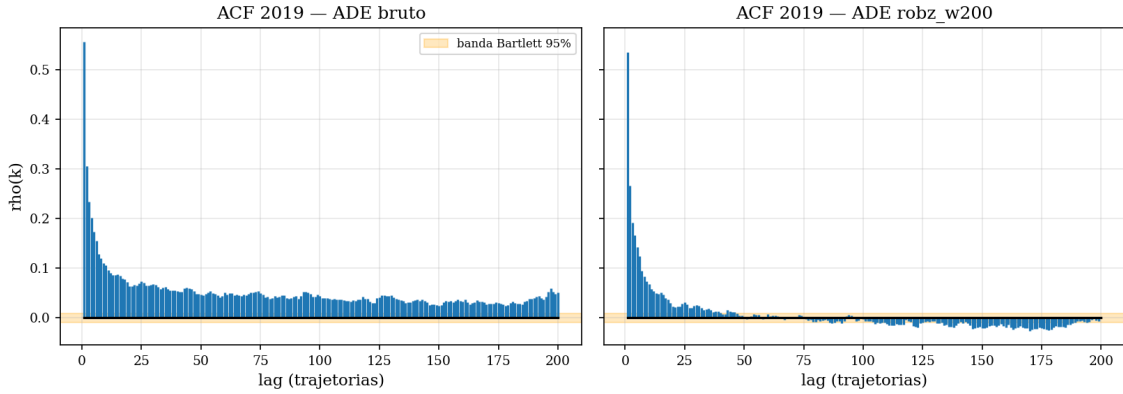


Figura 36: ACF do bloco 2019 (esq.: ADE bruto; dir.: sinal robz_w200) com banda de Bartlett 95 %. O robust-z encurta drasticamente a memória do sinal (corte em lag 41 vs. 500+).

9.3 Ensemble AND de mesma escala temporal

A Fase 3 atribuiu a inviabilidade do AND ao desalinhamento de escalas (ADWIN por trajetória vs. PH por janela de 50). O teste natural dessa explicação é alinhar as escalas: dois ADWIN por trajetória, um sobre o ADE e outro sobre `accel_p95` da janela de entrada da própria trajetória (extraída sem inferência do modelo, a partir de v_x, v_y desnormalizados; 3,7 M janelas processadas), ambos com `robz_w200`.

Config	W	n _{conf}	FAR/1k	year_cov	anos
AND(ADE, ACC)	50	4	0,021	3/5	2021, 2023, 2024
AND(ADE, ACC)	200	7	0,021	3/5	2021, 2023, 2024
AND(ADE, ACC)	2000	10	0,021	4/5	+2022
ADWIN_ADE standalone	—	25	0,042	5/5	todos
ADWIN_ACC standalone	—	114	0,271	5/5	todos (<i>inadmissível</i>)

Tabela 23: Ensemble de mesma escala (Mês 10). $\Delta_{\min}(\text{ADE}, \text{ACC})=3$ trajetórias (vs. 214 na Fase 3): com escalas alinhadas há coincidência já em $W=50$. Fonte: `same_scale_ensemble.csv`.

Leitura. O resultado confirma o diagnóstico da Fase 3 e o refina em três pontos: (i) *o desalinhamento era mesmo a causa*: $\Delta_{\min}$ cai de 214 para 3 trajetórias quando as escalas são alinhadas, e o AND deixa de ser estruturalmente vazio; (ii) *o AND repara um detector*

individualmente inadmissível: o ADWIN sobre aceleração tem FAR = 0,271/1k (consistente com o achado do KSWIN: features cinemáticas variam naturalmente dentro de 2019), mas o gate AND reduz a FAR do conjunto para 0,021/1k — metade da FAR do primário; (iii) *o custo é cobertura*: os anos de drift fraco (2022, 2025) não geram coincidências com $W \leq 1000$, de modo que, pela regra de decisão (cobertura primeiro), **o ADWIN_S2S standalone segue sendo a recomendação** — agora com a alternativa AND de mesma escala caracterizada como opção de alta especificidade para cenários em que falsos alarmes são mais custosos que latência de cobertura.

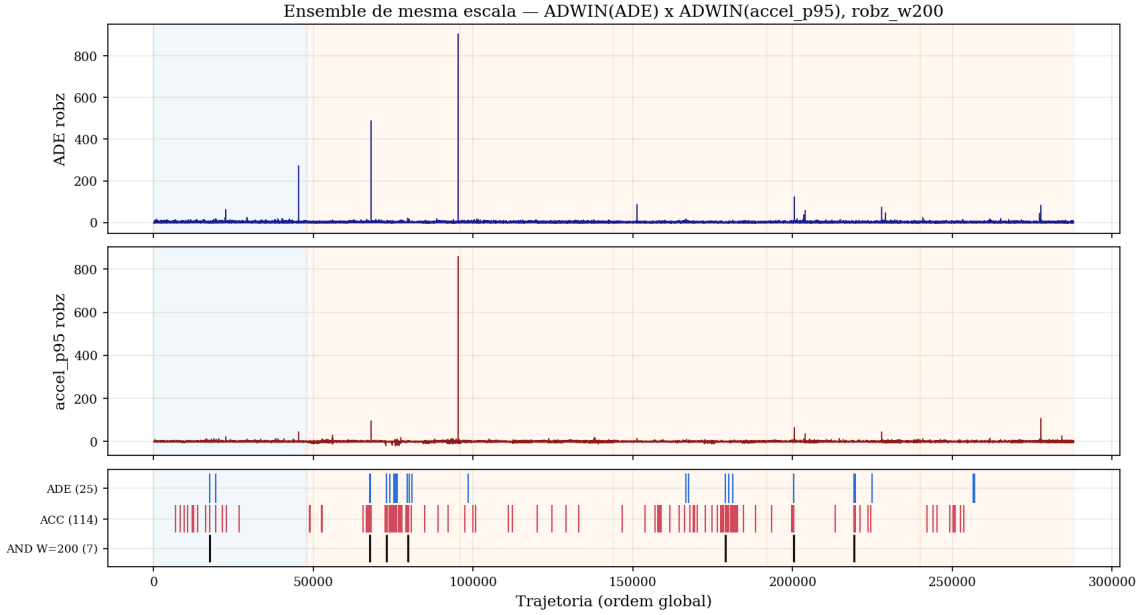


Figura 37: Ensemble de mesma escala: sinais robz de ADE (topo) e accel_p95 (meio) com rasters de alarmes e canal AND $W=200$ (baixo).

9.4 EWC no regime $\lambda \geq 10^5$

O sweep complementar $\lambda \in \{10^5, 10^6, 10^7\}$ (mesmo protocolo do Mês 7: 5 épocas, $1r=10^{-5}$, $n_{unfreeze}=2$) fecha a última lacuna do resultado do EWC: $cat_ratio \in [0,449; 0,452]$ e $recovery \in [-12,8; -11,8]\%$, indistinguíveis de $\lambda = 0$ (0,444). A Tabela 10 consolida as sete configurações. O resultado negativo da regularização de Fisher é definitivo ao longo de 8 ordens de grandeza de λ ; ver discussão na Seção 7.

10 Análise crítica e tese central (revisada após Mês 9)

10.1 Árvore de decisão para o cenário do trabalho

No início do quarto mês de pesquisa, foram definidos três **cenários possíveis** — perfis de resultado que o estudo poderia encontrar — para guiar a direção das análises seguintes. A idéia de uma “árvore de decisão” é simples: começa-se pela pergunta mais básica (“o drift é real ou é confundimento?”), e dependendo da resposta, avalia-se a próxima (“o modelo degrada muito ou pouco?”). Cada ramo leva a um cenário com hipóteses e conclusões diferentes.

- **Cenário 1**: o drift aparente seria um *confundimento* entre divisões — isto é, a piora do modelo seria causada não por mudança real no estilo de jogo, mas pelo fato de o dataset de 2019 misturar jogos de Divisão A e B enquanto os datasets de 2022–2025 são todos Divisão A. **Não se aplica**: os dados disponíveis não incluem Divisão B, de modo que não é possível testar essa hipótese; além disso, $\Delta_A \approx 0,40$ mm é positivo e persistente, indicando drift real.

- **Cenário 2:** o drift seria severo o suficiente para que adaptação simples (fine-tuning com poucos exemplos) não fosse suficiente, exigindo retreino completo ou arquitetura nova. **Não se aplica:** o excesso de ADE é pequeno (0,4mm em uma escala de dezenas de mm) e a recuperação mediana por importance weighting é $\approx 63\%$ — o modelo degrada, mas moderadamente, e a adaptação parcial é viável.
- **Cenário 3** (descritivo + detecção operacional): o drift é real e mensurável, a adaptação é *parcialmente* eficaz, e o trabalho concentra-se em **caracterizar** o drift (quais features mudam, em que período, com que magnitude) e em desenvolver um **detetor online operacional** que avise o engenheiro quando retreinar. **Escolhido.**

10.2 Tese central — Cenário 3 reformulado

A **tese central** é a afirmação principal que este trabalho defende, suportada por evidências numéricas convergentes. Em resumo:

O drift no Seq2Seq para trajetórias SSL é **real, intra-divisão** (ocorre dentro da Divisão A, ou seja, não é um artefato de misturar divisões) e **direcionalmente covariate** (o tipo de drift em que as características de entrada do modelo mudam, não a relação entre entrada e saída): o classifier two-sample test rejeita a hipótese de igualdade distribucional em todos os anos ($p < 10^{-3}$), e a fração do excesso de ADE atribuível a esse covariate shift está entre 9 e 72% (LSIF 59–72%, RuLSIF 9–23%, classifier cov. 22–51% — Tabela 4), com `accel_p99` como a feature dominante segundo o LSIF marginal (o percentil 99 da aceleração — representa os movimentos mais bruscos). Contudo, a adaptação por fine-tuning leve é severamente prejudicada por *catastrophic interference* (o modelo “esquece” o que aprendeu antes ao ser retreinado com dados novos). **O Mês 9 acrescenta uma camada operacional:** sobre o stream de 288k trajetórias com pré-processamento robust-z, identifica-se um detetor online viável (`ADWIN_S2S`, $\text{FAR}_{2019}=0,042/1k$, $\text{year_coverage}=5/5$).

Reposicionamento após Mês 6 e revisão final após Mês 9. Uma nota sobre **granularidade** é essencial para entender a evolução das conclusões. **Granularidade per-game** significa que cada ponto de dados é a média de ADE de um jogo inteiro: com 6 jogos por ano e 5 anos de dados (2020–2025), temos apenas 36 pontos no total. **Granularidade per-trajetória** significa que cada ponto é uma trajetória individual: com $\approx 57\,600$ trajetórias por ano, temos 288 000 pontos. Detectores estatísticos precisam de muitos pontos para funcionar — 36 é pouco demais.

A leitura inicial dos Mês 1–5 apoiava a tese central *também* em detecção formal de changepoint (`ADWIN/PH` online + Pelt offline). O Mês 6 mostrou que essas detecções não sobrevivem ao escrutínio formal *na granularidade per-game*. O Mês 9 mostrou que *na granularidade per-trajetória com pré-processamento adequado* elas funcionam bem. **A reposição final é:**

1. Per-game (36 pontos/ano): detecção formal não funciona; a tese descritiva apóia-se em decomposição IW (importance weighting — estimação de quanto cada tipo de trajetória está sobre- ou sub-representada em cada ano).
2. Per-trajetória (288k stream): detecção funciona bem com `ADWIN+robz` $w=200$ (Mês 9).
3. Ensemble AND temporal entre `ADWIN` e `PH` é estruturalmente inviável ($\Delta_{\min}=214 > W=200$; cobertura $\geq 2/5$ só com $W \geq 10\,000$) — testado, caracterizado e rejeitado (os dois detectores disparam com atrasos de milhares de trajetórias entre si, o que torna o AND de coincidência inoperante para janelas razoáveis).

10.2.1 Suporte numérico amalgamado

1. $\Delta_A \approx 0,40$ mm positivo e persistente em 2022–2025 (Tabela 6).
2. Recovery LSIF mediana $\approx 65\%$ (range 59–72%); RuLSIF mediana $\approx 12\%$ (range 9–23%); classifier cov. mediana $\approx 31\%$ (range 22–51%) — os três estimadores concordam em sinal

mas variam em magnitude (Tabela 4). Todos os anos significativos a $p < 10^{-3}$ pelo classifier two-sample test.

3. **accel_p99** lidera ranking de features (Tabela 5).
4. Fine-tuning ingênuo: **cat_ratio**=0,82 (Mês 3, Tabela 7); reduzido a 0,44 com EWC (Mês 7, Tabela 10).
5. Pelt mBIC monotônico: $K^*=0$ ao nível de 36 jogos (Figura 10).
6. **Detecção operacional viável** sobre stream per-trajetoria com robz $w=200$: FAR=0,042/1k, year_coverage=5/5 (Tabela 16, Mês 9).
7. **Ensemble AND inviável**: $\Delta_{\min}=214$; year_coverage=2 exige $W \geq 10\,000$ ($\sim 2,5$ meses) — absurdo (Tabela 17, Mês 9).

10.3 Leitura cruzada das figuras

- **O drift afeta a cauda mais do que a média**: Figura 2, Figura 3 e Tabela 2 concordam.
- **Aceleração é a feature dominante**: Figura 6 (regressão), Figura 8 (Spearman), Figura 5 (KS por janela), e Tabela 5 (LSIF marginal). *Quatro evidências independentes alinhadas.*
- **Concept drift residual existe**: recovery LSIF $\leq 72\%$ (Figura 11b; teto RuLSIF mais baixo, 23 %, Tabela 4) + fine-tuning falha em recuperar o restante (Tabela 7).
- **Detecção online funciona no stream completo**: Figura 30 (top-5 streams ADWIN+robz), Figura 33 (3 canais), Tabela 16 (master table). *Três ângulos confirmam o detetor primário.*
- **AND temporal é inviável**: Figura 34 (desalinhamento) + Figura 32 (sweep) + Tabela 17 ($W=10\,000 \sim 2,5$ meses).

11 Limitações honestas

- **Divisão B ausente**. O RoboCup SSL tem duas divisões de competição: Divisão A (times mais avançados) e Divisão B (times em desenvolvimento). Nenhum dos conjuntos de dados disponíveis corresponde à Divisão B; todos os dados de 2022–2025 são Divisão A. Isso impede testar se o drift observado seria diferente entre divisões — uma questão que permanece em aberto.
- **Detectores online sem poder estatístico ao nível per-game**. A calibração por ARL (Mês 6) confirmou que ADWIN e PH não conseguem rejeitar H_0 (a hipótese nula de “sem drift”) no stream por jogo ($n=36$ pontos, um por jogo). O problema é que 36 pontos são poucos para qualquer teste estatístico detectar drift gradual. O Mês 9 contorna isso operando *por trajetória* (288 k pontos), o que depende da hipótese de independência entre trajetórias consecutivas. **O Mês 10 testou essa hipótese** (Seção 9): a autocorrelação é real ($\rho_1 \approx 0,54$; $n_{\text{eff}} \approx 7\,762$ no sinal robz, 16 % de n), e o IC de Wilson conservador da FAR alarga para $[0,071; 0,939]/1k$. Porém o teste por permutação mostra que a dependência serial *infla* a FAR (streams 2019 i.i.d.-izados produzem zero alarmes): a violação é conservadora na direção certa e a admissibilidade do detetor não é artefato dela. O que permanece como limitação é a largura honesta do IC, consequência dos 6 jogos de 2019.
- **ADE single-mode, não min-ADE_K**. O Seq2Seq neste trabalho produz *uma única* previsão determinística por trajetória (single-mode). Modelos estocásticos modernos como Trajectron++ [20] e AgentFormer [21] geram K trajetórias hipotéticas diferentes e são avaliados pelo **min-ADE_K** — a distância mínima entre as K previsões e a trajetória real. Essa métrica é mais justa para modelos multimodo. Resultados com min-ADE_K não são diretamente comparáveis aos com ADE single-mode.

- **EWC reduz `cat_ratio` via `lr`, não via Fisher.** O ganho real em relação ao Mês 3 advém da taxa de aprendizado muito baixa ($lr = 10^{-5}$) e do congelamento do encoder — não da regularização EWC propriamente dita. A diagonal de Fisher é não nula ($\text{mean diag} \approx 6,7 \times 10^{-7}$), mas o sweep completo $\lambda \in \{0, 50, 500, 5000, 10^5, 10^6, 10^7\}$ (Mês 7 e 10) mostra `cat_ratio` estável em 0,44–0,45 em todo o intervalo: a regularização de Fisher não adiciona sinal em nenhum regime testado. O resultado negativo é definitivo para esta arquitetura/conversão; a alternativa por explorar é replay ou retreino do encoder.
- **Tamanho amostral pequeno.** Apenas 6 jogos por ano — com este volume, os ICs são amplos, especialmente para anos com baixo excesso (2022, excesso = 0,23 mm). O IID bootstrap original reamostrava trajetórias individualmente, ignorando a correlação intra-jogo; o **block bootstrap** (bloco = jogo, implementado em iteração pós-Mês 9) revelapag ICs honestamente mais largos e compatíveis com o tamanho real da amostra. Para 2022, cujo excesso é próximo de zero, o block bootstrap é instável (denominador da recovery tende a zero); o IC IID é reportado para esse ano. Mais jogos por ano reduziriam a incerteza significativamente.
- **Concept drift residual não é isolado por feature.** A fração do excesso de ADE não explicada por covariate shift nas features cinemáticas medidas (velocidade, aceleração, curva) depende do estimador: $\approx 28\text{--}41\%$ sob LSIF, $\approx 77\text{--}91\%$ sob RuLSIF; o intervalo plausível para a componente residual é $[28; 91]\%$. Não sabemos se esse residuo é: (a) covariate shift em features não medidas; (b) concept drift real (mudança nas estratégias de jogo); ou (c) simplesmente ruído amostral. A discordância LSIF/RuLSIF reforça a hipótese de que parte do que LSIF atribuíu a covariate shift está, na verdade, na cauda extrema de `accel_p99` onde os pesos LSIF são numericamente instáveis.
- **Monocultura de evidência no covariate shift.** O número “59–72 %” tinha uma única fonte metodológica (LSIF). Em iteração pós-Mês 9, foram implementados dois validadores independentes: (i) **RuLSIF** [15], estimador robusto com pesos limitados por $1/\alpha$, que reduz a variância sob cauda pesada e é comparado ao LSIF; (ii) teste de duas amostras via **classificador** [16], que detecta covariate shift sem assumir forma para a razão de densidades. Os resultados são discutidos na Tabela 4.
- **Ensemble AND inviável entre escalas distintas (Mês 9); viável mas subordinado na mesma escala (Mês 10).** A inviabilidade da Fase 3 é estrutural à configuração ADWIN-por-trajetoria + PH-por-janela. O teste de mesma escala do Mês 10 (dois ADWIN por trajetória: ADE + `accel_p95`) confirma o diagnóstico: $\Delta_{\min}$ cai de 214 para 3 trajetórias e o AND passa a produzir alarmes confirmados já com $W=50$. Porém o AND de mesma escala reduz a cobertura (3/5 anos com $W \leq 1000$) porque o detector de aceleração é individualmente inadmissível ($\text{FAR} = 0,271/1k$) e os anos de drift fraco (2022, 2025) não geram coincidências — o ADWIN_S2S standalone segue sendo a recomendação pela regra de decisão (cobertura primeiro).
- **Pré-processamento robust-z assume estacionariedade local.** O robust-z com janela $w=200$ funciona porque o drift é *gradual*: a distribuição muda lentamente ao longo de meses. Isso significa que dentro de uma janela de 200 trajetórias ($\approx$ algumas horas de jogo), a distribuição é aproximadamente estável — o que justifica normalizar por ela. Sob drift *abrupto* (mudança instantânea), a janela de 200 trajetórias faria a normalização absorver o próprio sinal de drift, tornando-o indetectável — o robusto-z “engoliria” exatamente o que tenta detectar.

12 Próximos passos

1. (*Concluído Mês 6*) Calibração de ADWIN/PH por ARL sintético; curva de latência corrigida; validação do Pelt por BOCPD + gamma.
2. (*Concluído Mês 7*) EWC [17] com sweep $\lambda \in \{0, 50, 500\}$: `cat_ratio` $0,82 \rightarrow 0,44$.
3. (*Concluído Mês 8*) Reescrita das seções introdutórias e *related work* integrando ~ 35 referências.
4. (*Concluído Mês 9*) Otimização em 4 fases: (i) pré-processamento `robz_w200`, (ii) grid 174 configs $\times$ 2 modelos $\times$ 3 frentes, (iii) ensemble AND/OR + sweep W , (iv) métrica $\text{SNR}_{\text{smooth}}$, master table, inviabilidade do AND. Detetor primário: ADWIN+robz com $\text{FAR}_{2019}=0,042/1k$, `year_coverage=5/5`.
5. (*Concluído pós-Mês 9*) **RuLSIF** [15] implementado e comparado ao LSIF clássico; block bootstrap (bloco = jogo) substituí o IID bootstrap para ICs honestos; teste de duas amostras via classificador [16] implementado como validação independente. Resultados na Tabela 4.
6. (*Concluído Mês 10*) Fechamento das quatro pendências da revisão do artigo (Seção 9): latência de detecção por ano no stream real; validação da hipótese de independência do stream (ACF, n_{eff} , permutações em bloco); ensemble AND de mesma escala temporal (Δ_{min} : $214 \rightarrow 3$); EWC com $\lambda \in \{10^5, 10^6, 10^7\}$ (negativo definitivo).
7. (*Pendente*) Implementação do detetor primário como serviço online: integrar ADWIN+robz em um pipeline que dispare retreino com `cooldown=200`.
8. (*Pendente*) Baseline de adaptação por *replay* (minibatches 2019+2021) como alternativa ao EWC.
9. (*Mestrado*) Cobrir Divisão B (alvo: 2024+, equipes competidoras) e **minADE_K** ao lado do ADE single-mode atual, alinhando com Salzmann et al. [20], Yuan et al. [21].

Referências

- [1] B. Efron. *Bootstrap methods: another look at the jackknife*. The Annals of Statistics 7(1), 1979, p. 1–26. DOI: 10.1214/aos/1176344552.
- [2] A. Bifet, R. Gavaldà. *Learning from time-changing data with adaptive windowing*. SDM, 2007. DOI: 10.1137/1.9781611972771.42.
- [3] E. S. Page. *Continuous Inspection Schemes*. Biometrika 41(1–2), 1954, p. 100–115. DOI: 10.1093/biomet/41.1-2.100.
- [4] C. Raab, M. Heusinger, F.-M. Schleif. *Reactive Soft Prototype Computing for Concept Drift Streams*. Neurocomputing 416, 2020. DOI: 10.1016/j.neucom.2020.01.119.
- [5] H. Mouss, D. Mouss, N. Mouss, L. Sefouhi. *Test of Page-Hinkley, an approach for fault detection in an agro-alimentary production system*. Asian Control Conference, 2004.
- [6] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, A. Bouchachia. *A survey on concept drift adaptation*. ACM Computing Surveys 46(4), 2014. DOI: 10.1145/2523813.
- [7] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, G. Zhang. *Learning under concept drift: a review*. IEEE TKDE 31(12), 2019. DOI: 10.1109/TKDE.2018.2876857.
- [8] C. Truong, L. Oudre, N. Vayatis. *Selective review of offline change point detection methods*. Signal Processing 167, 2020. DOI: 10.1016/j.sigpro.2019.107299.

- [9] R. Killick, P. Fearnhead, I. A. Eckley. *Optimal detection of changepoints with a linear computational cost*. JASA 107(500), 2012. DOI: 10.1080/01621459.2012.737745.
- [10] R. P. Adams, D. J. C. MacKay. *Bayesian Online Changepoint Detection*. arXiv:0710.3742, 2007.
- [11] S. Aminikhanghahi, D. J. Cook. *A survey of methods for time series change point detection*. KAIS 51, 2017, p. 339–367. DOI: 10.1007/s10115-016-0987-z.
- [12] M. Sugiyama, T. Suzuki, S. Nakajima, H. Kashima, P. von Büna, M. Kawanabe. *Direct importance estimation for covariate shift adaptation*. Annals of the Institute of Statistical Mathematics 60(4), 2008. DOI: 10.1007/s10463-008-0197-x.
- [13] M. Sugiyama, M. Kawanabe. *Machine Learning in Non-Stationary Environments*. MIT Press, 2012.
- [14] T. Kanamori, S. Hido, M. Sugiyama. *A least-squares approach to direct importance estimation*. JMLR 10, 2009, p. 1391–1445.
- [15] M. Yamada, T. Suzuki, T. Kanamori, H. Hachiya, M. Sugiyama. *Relative density-ratio estimation for robust distribution comparison*. Neural Computation 25(5), 2013. DOI: 10.1162/NECO_a_00442.
- [16] D. Lopez-Paz, M. Oquab. *Revisiting Classifier Two-Sample Tests*. ICLR, 2017. arXiv:1610.06545.
- [17] J. Kirkpatrick, R. Pascanu, N. Rabinowitz *et al.* *Overcoming catastrophic forgetting in neural networks*. PNAS 114(13), 2017. DOI: 10.1073/pnas.1611835114.
- [18] T. Lesort, V. Lomonaco, A. Stoian, D. Maltoni, D. Filliat, N. Díaz-Rodríguez. *Continual learning for robotics*. Information Fusion 58, 2020. DOI: 10.1016/j.inffus.2019.12.004.
- [19] A. Alahi, K. Goel, V. Ramanathan, A. Robicquet, L. Fei-Fei, S. Savarese. *Social LSTM: Human trajectory prediction in crowded spaces*. CVPR, 2016. DOI: 10.1109/CVPR.2016.110.
- [20] T. Salzmann, B. Ivanovic, P. Chakravarty, M. Pavone. *Trajectron++: Dynamically-feasible trajectory forecasting with heterogeneous data*. ECCV, 2020. arXiv:2001.03093.
- [21] Y. Yuan, X. Weng, Y. Ou, K. Kitani. *AgentFormer: Agent-aware transformers for socio-temporal multi-agent forecasting*. ICCV, 2021. arXiv:2103.14023.
- [22] L. Steuernagel, M. R. O. A. Maximo. *Trajectory Prediction for SSL Robots Using Seq2seq Neural Networks*. In RoboCup 2022: Robot World Cup XXV, LNCS vol. 13561, pp. 27–39. Springer, 2023. DOI: 10.1007/978-3-031-28469-4_3.
- [23] T. Gu, G. Chen, J. Li, C. Lin, Y. Rao, J. Zhou, J. Lu. *Stochastic Trajectory Prediction via Motion Indeterminacy Diffusion*. CVPR, 2022. arXiv:2203.13777.